



Sheet 3 : Game Playing

ENG. ABDULRAHMAN
ELMADKHOUM

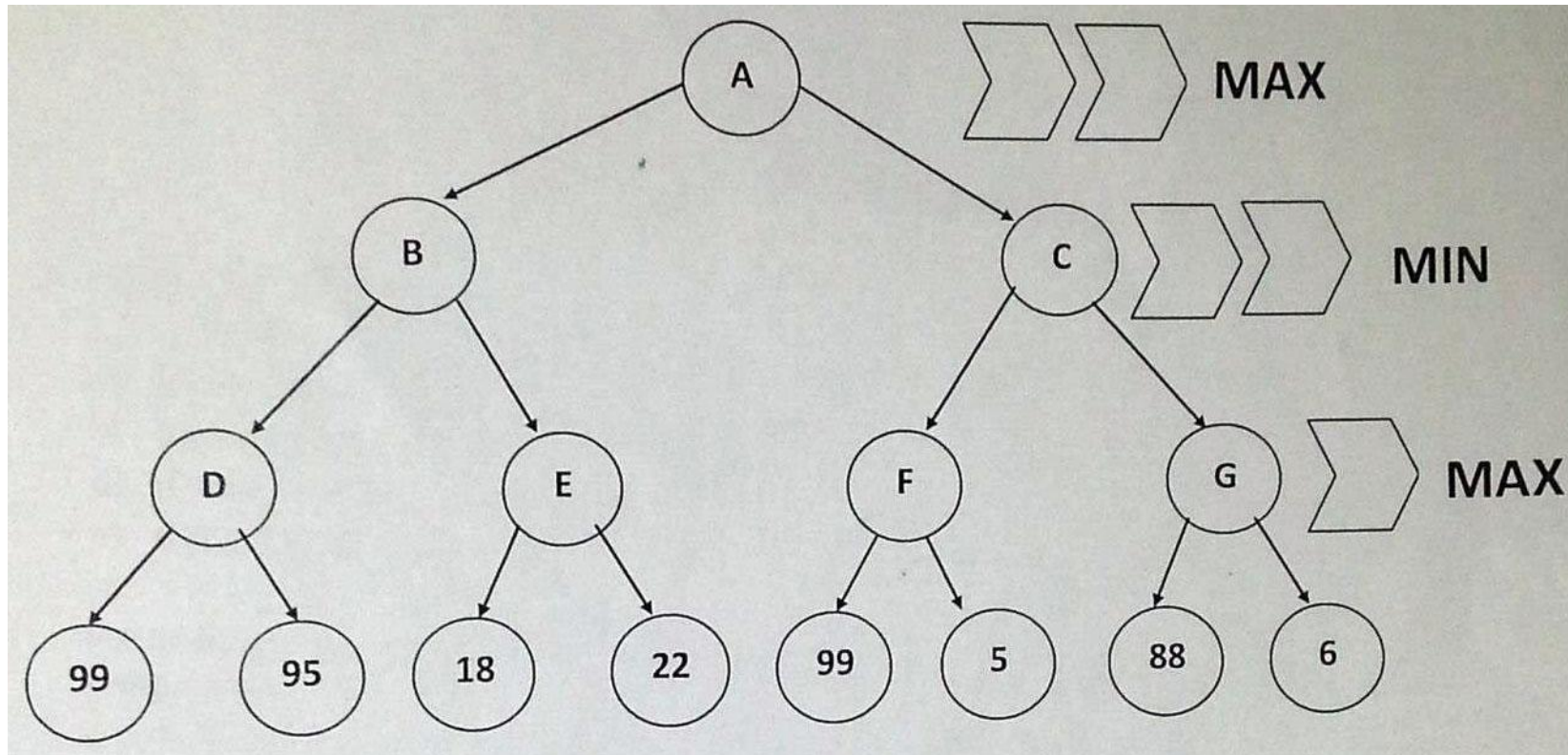
1- Min Max Algorithm

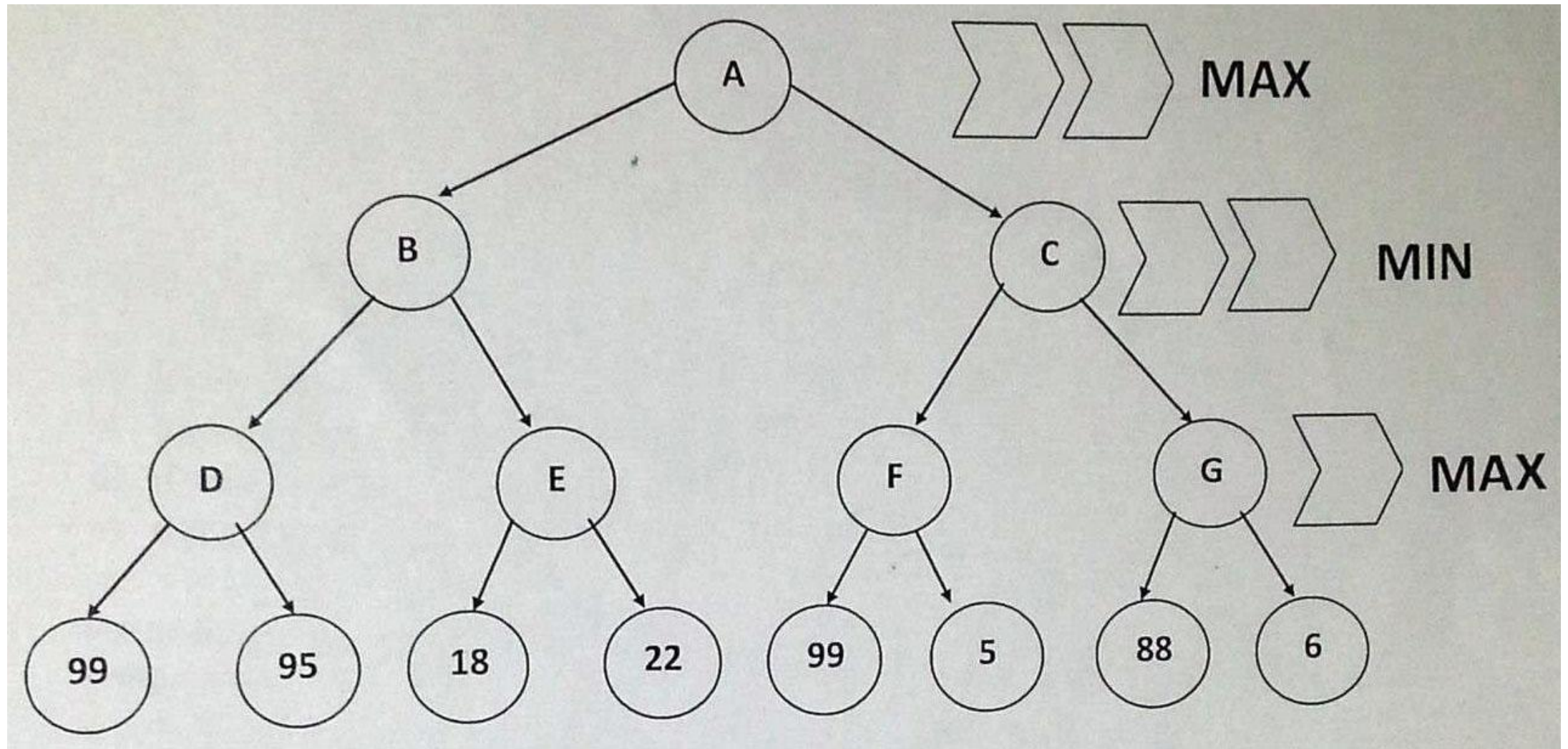
- The Minimax algorithm is a decision-making technique commonly used in game theory, specifically for two-player games like chess or tic-tac-toe.
- It involves a tree structure that represents all possible moves.

1- Min Max Algorithm

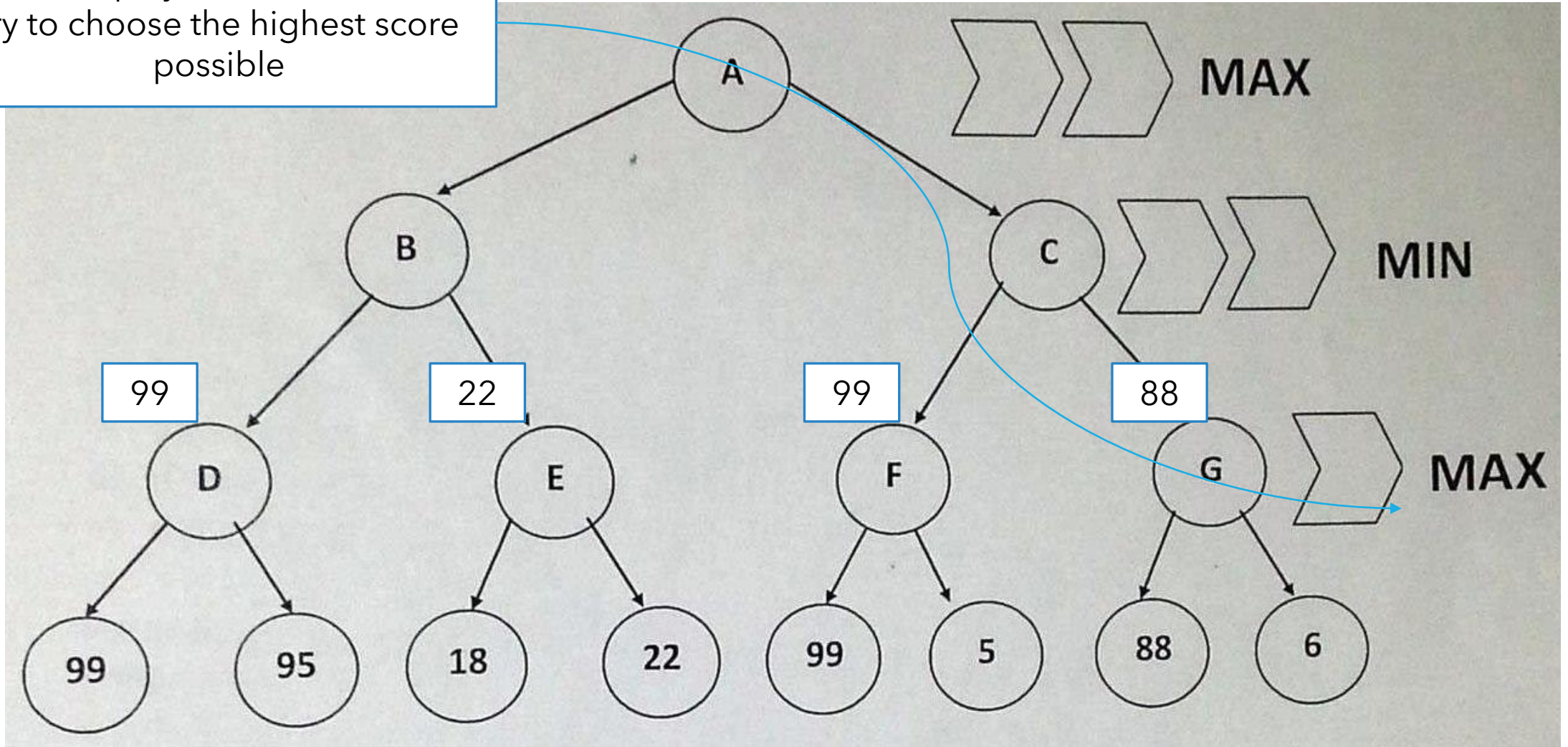
- In Minimax, one player is designated the "maximizer" aiming to get the highest possible score, while the other is the "minimizer," who seeks to lower the maximizer's score.
- The algorithm recursively explores possible game states down the tree, evaluating each position's score and backtracking to determine the optimal move. Minimax assumes both players are making the best possible moves.

Example : Get the Value of A after applying min max algorithm

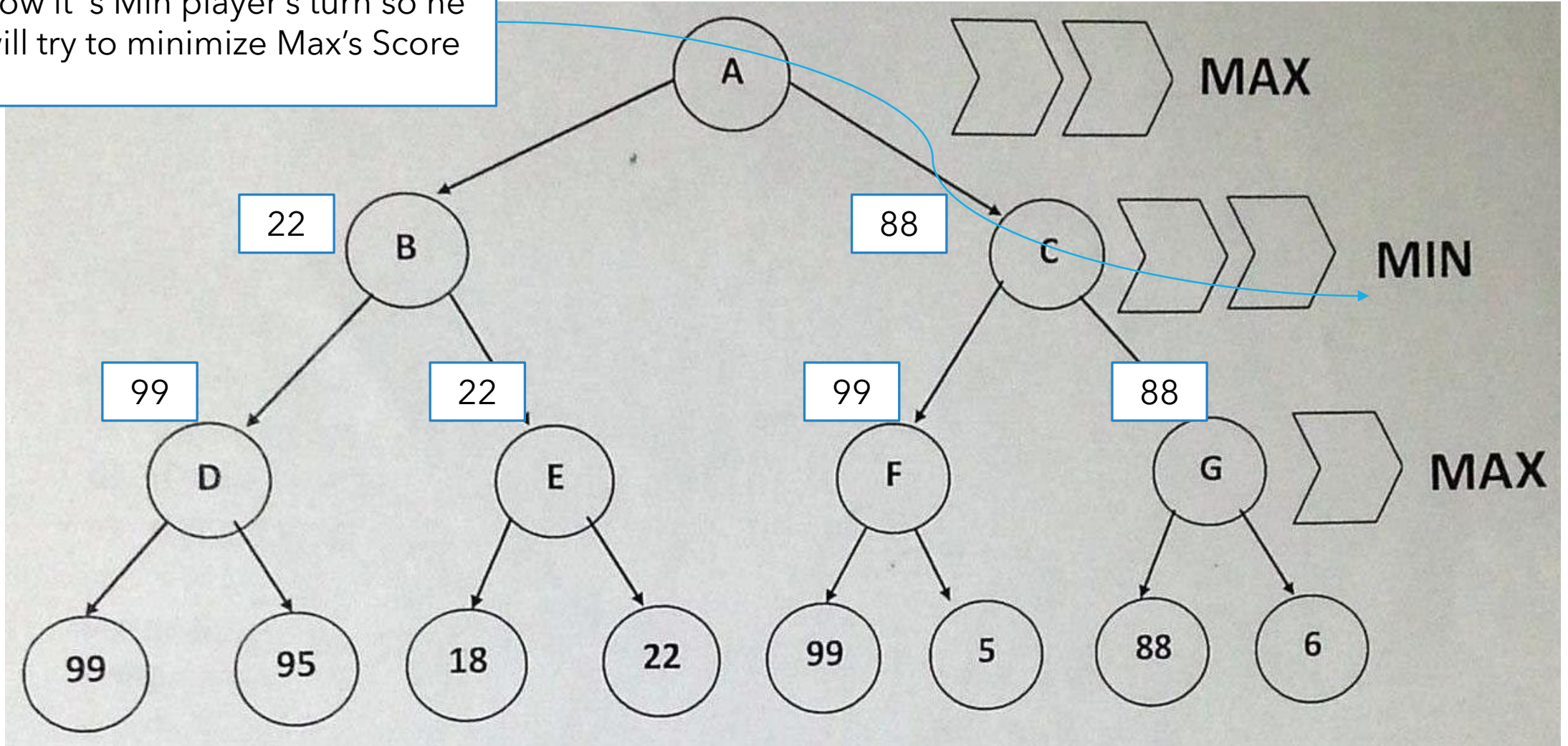




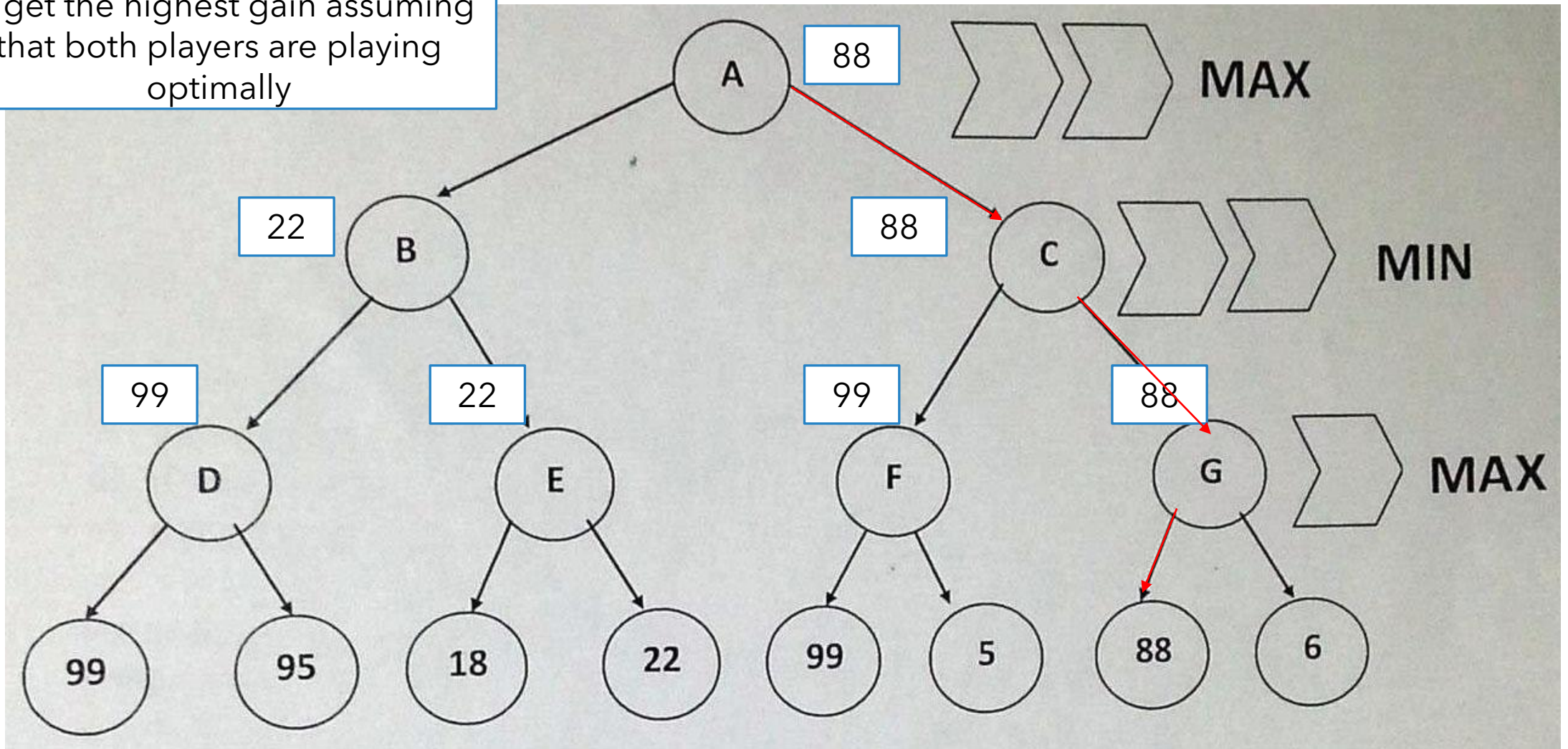
It is max player's turn so he will try to choose the highest score possible



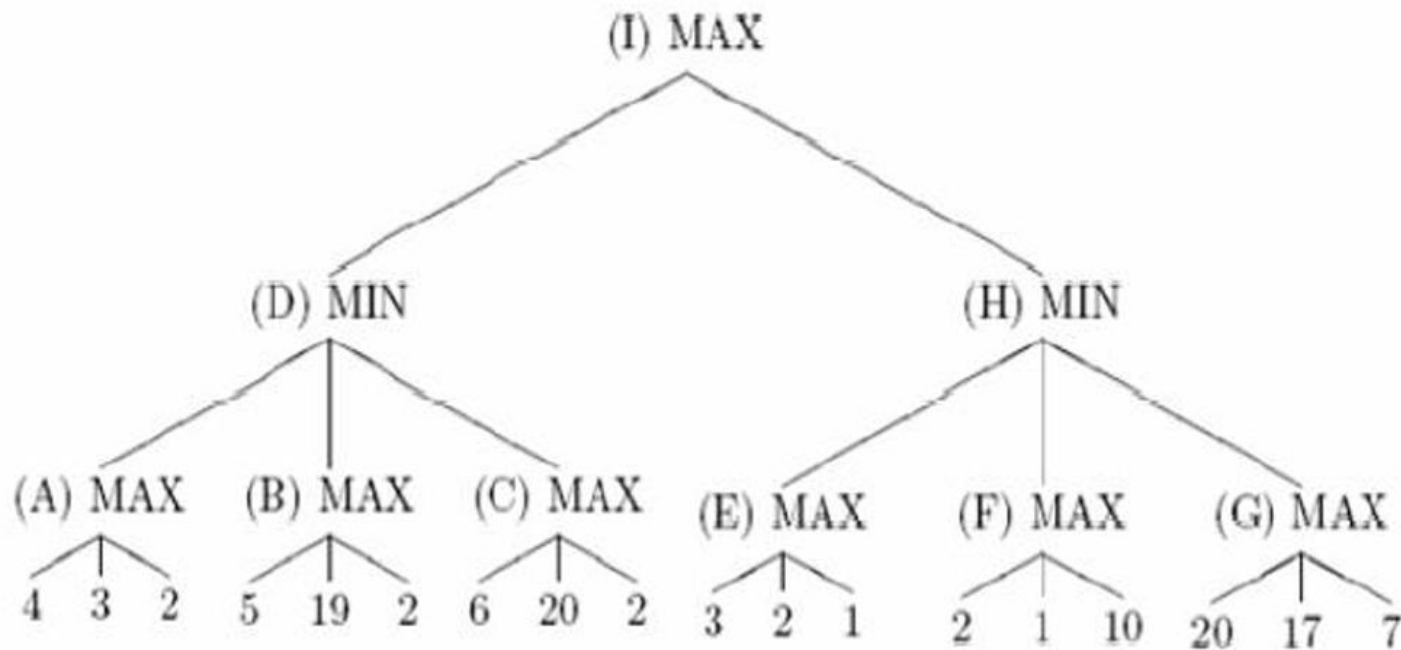
Now it's Min player's turn so he will try to minimize Max's Score

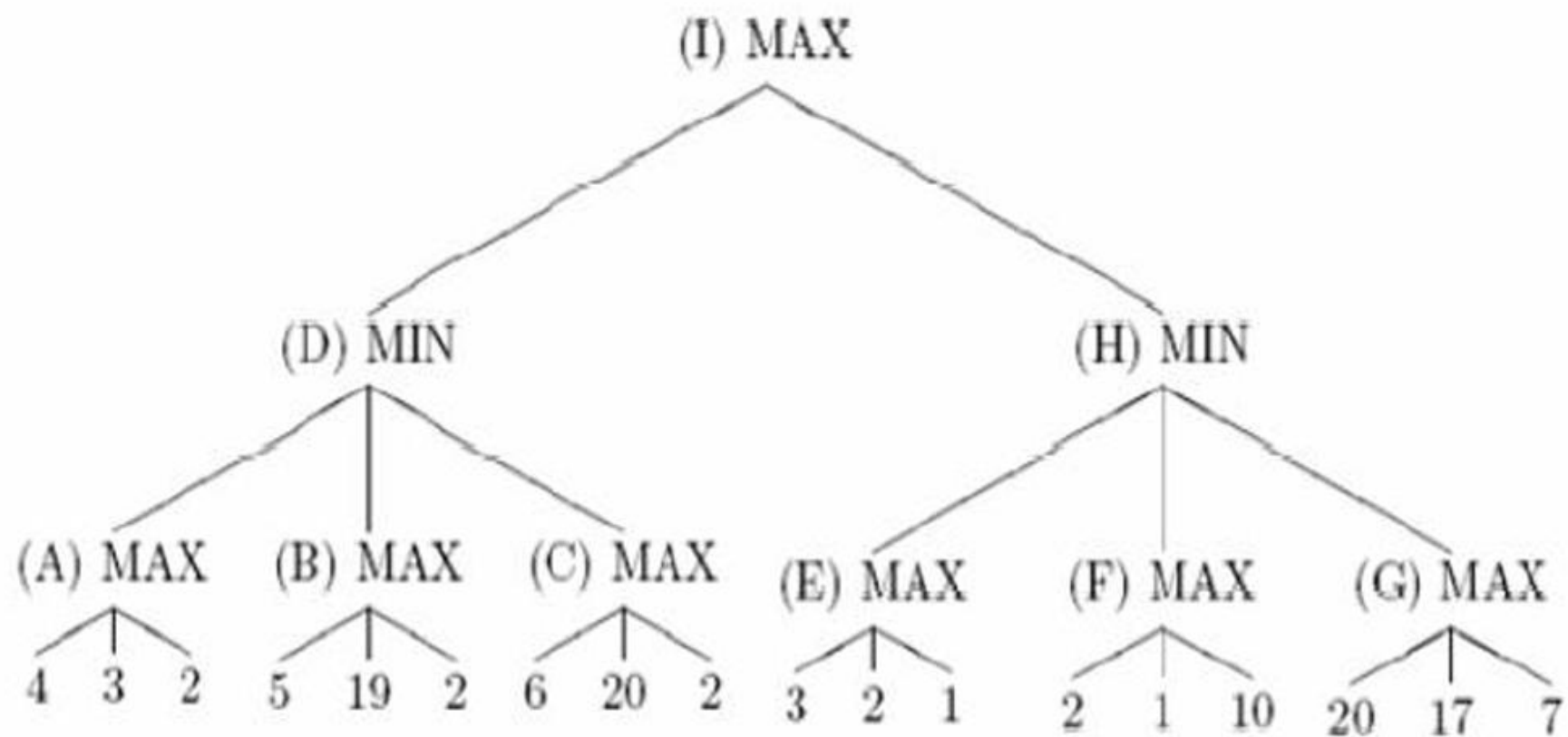


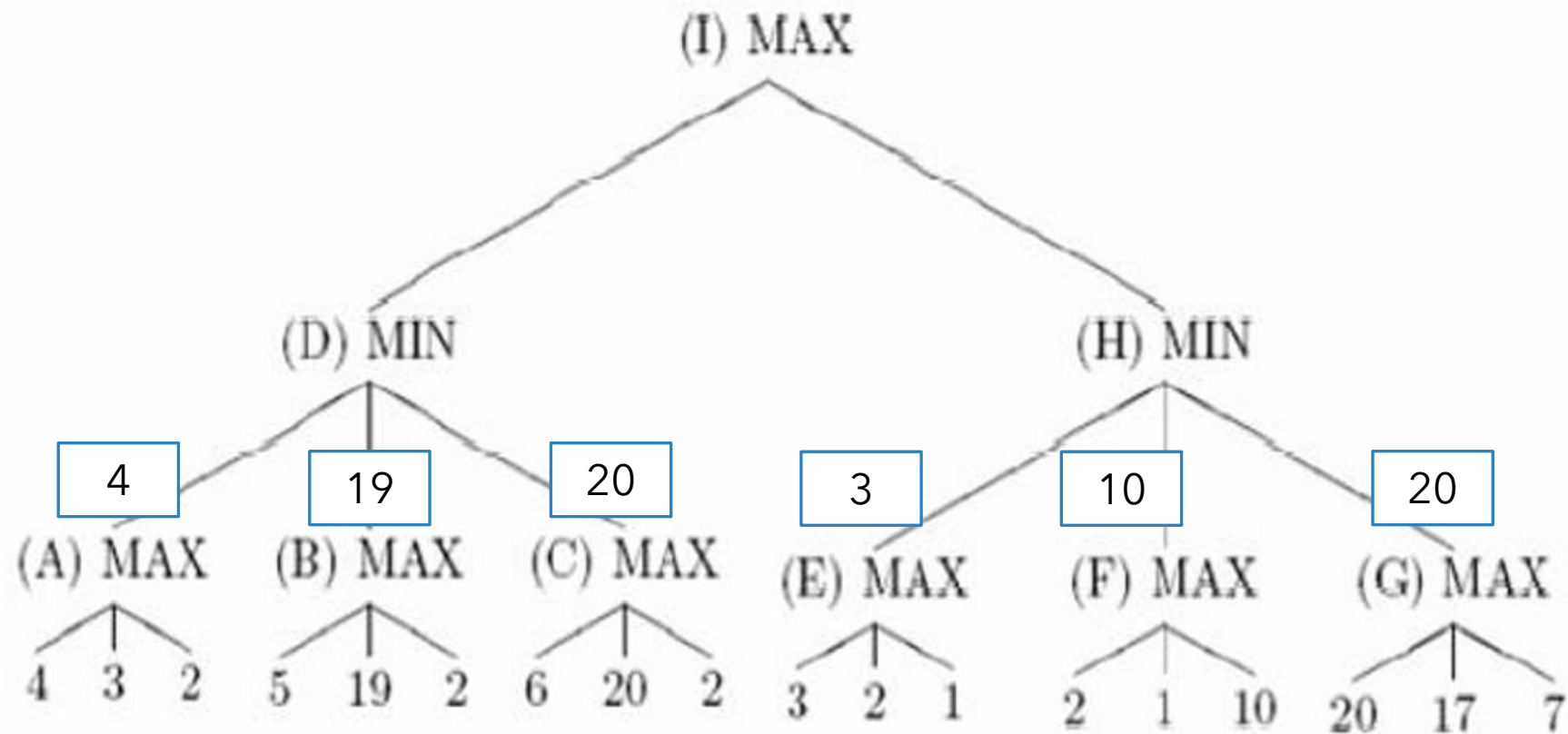
So This is the Optimal path for A to get the highest gain assuming that both players are playing optimally

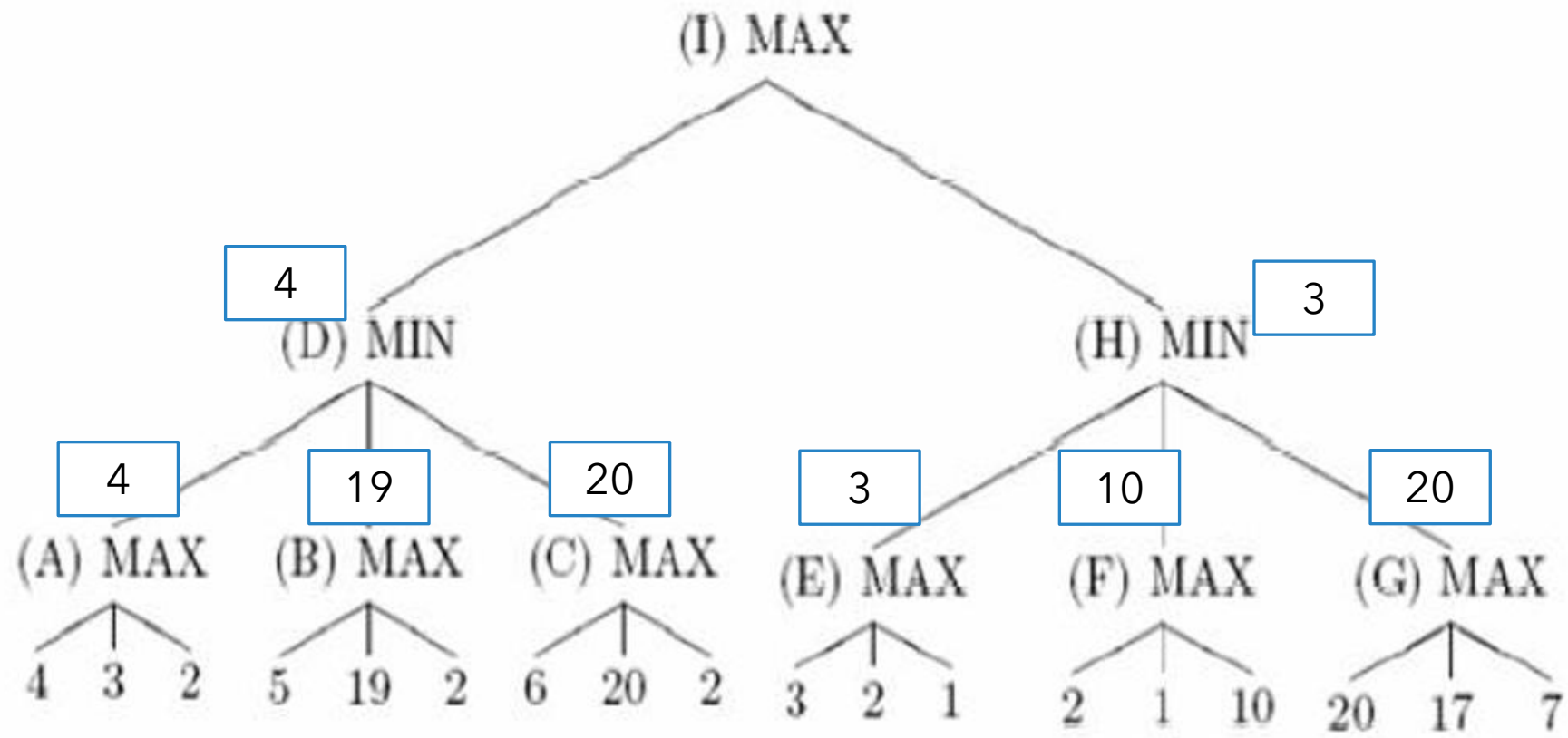


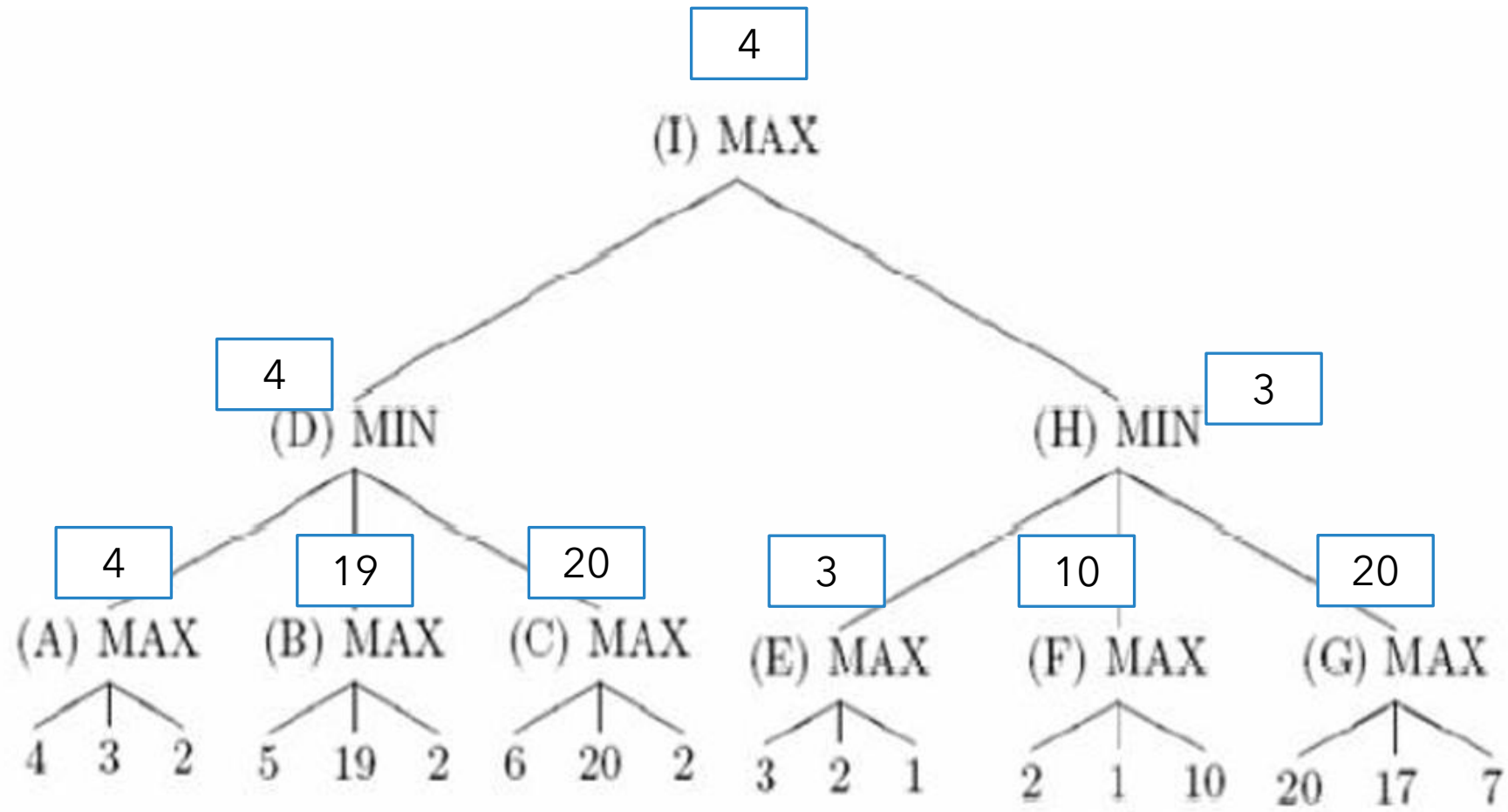
Example : Get the Value of I after applying min max algorithm



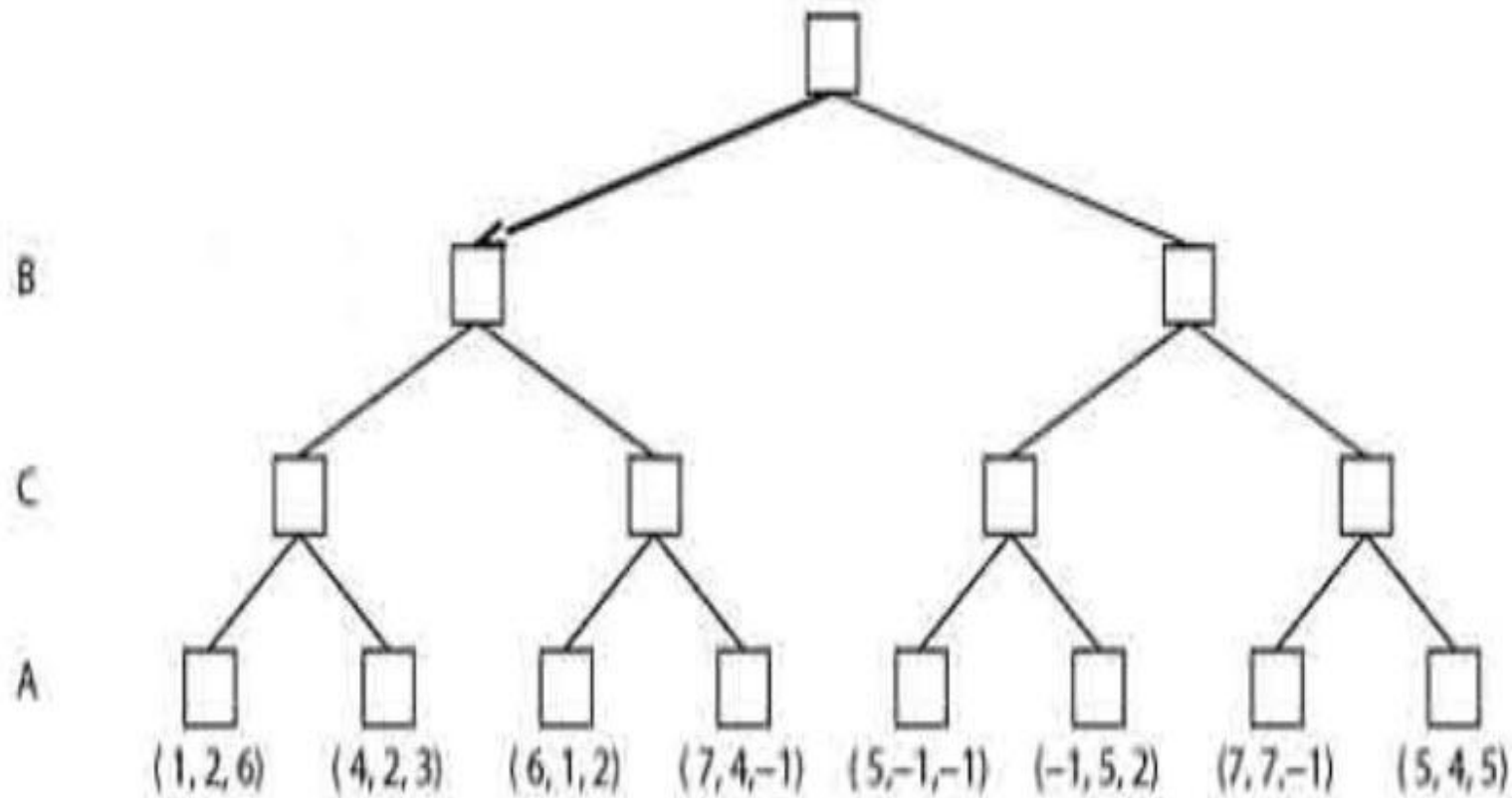


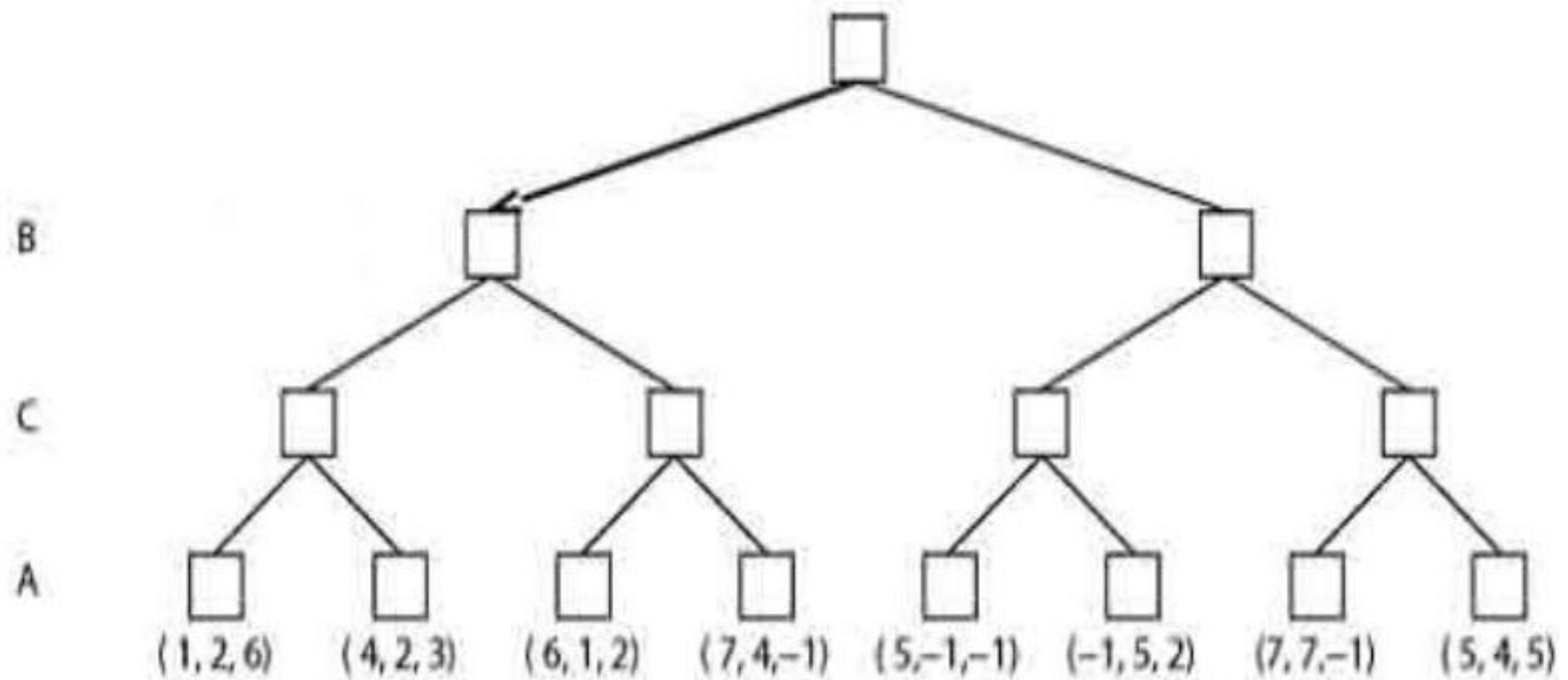


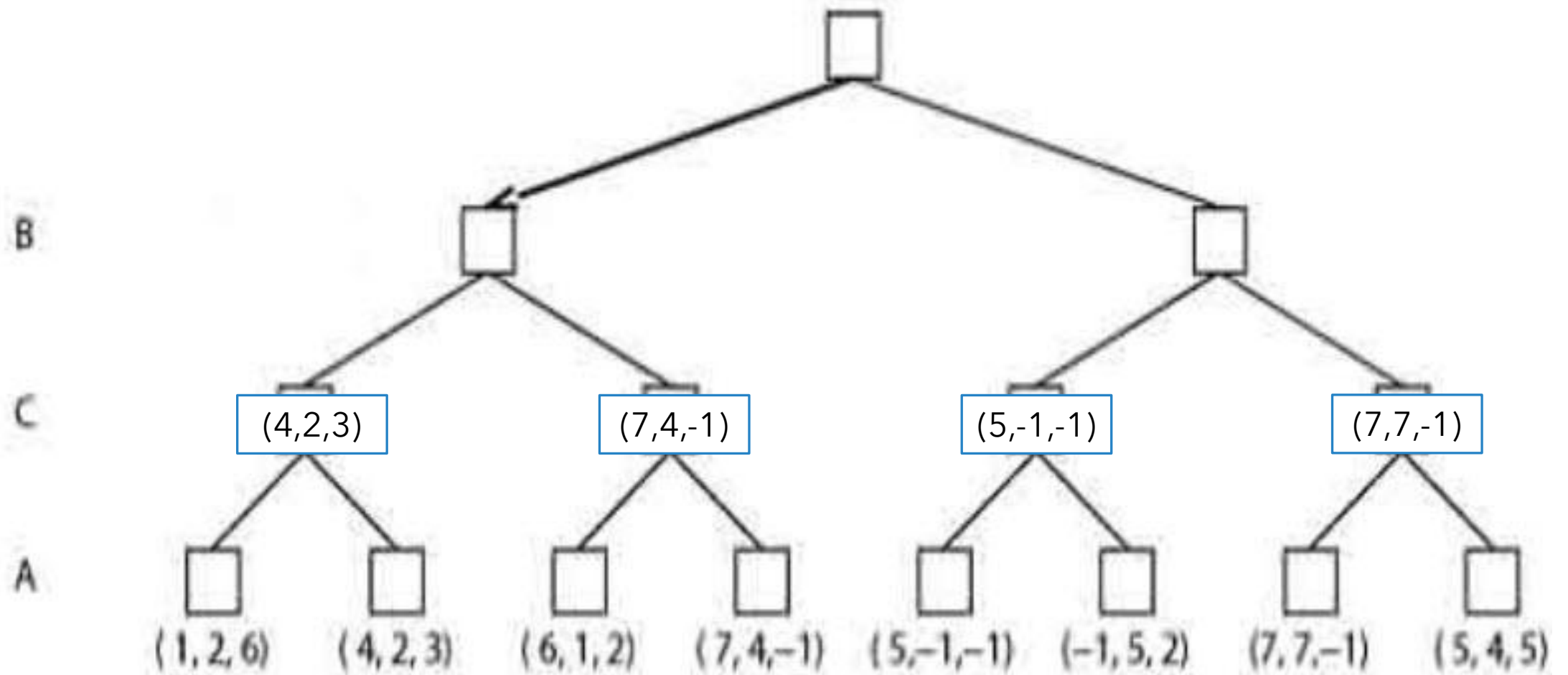


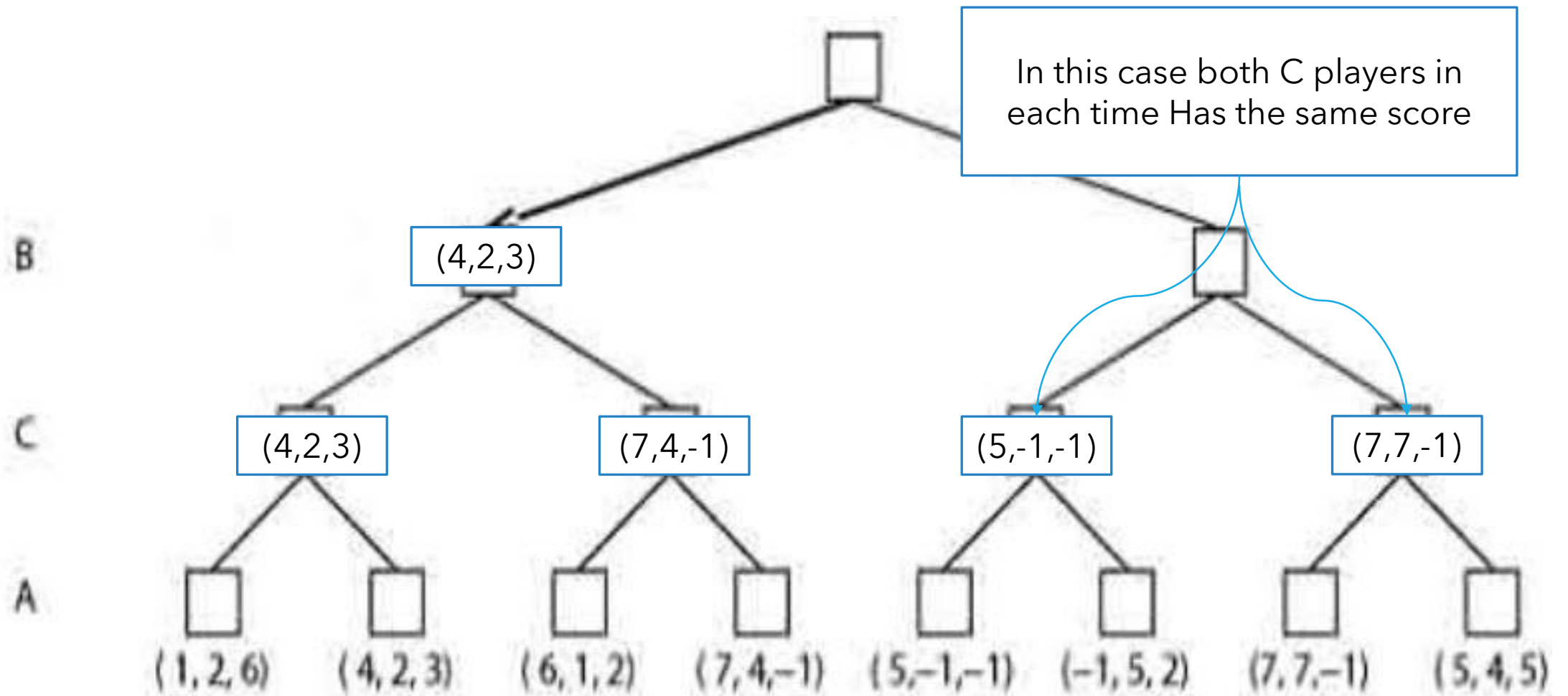


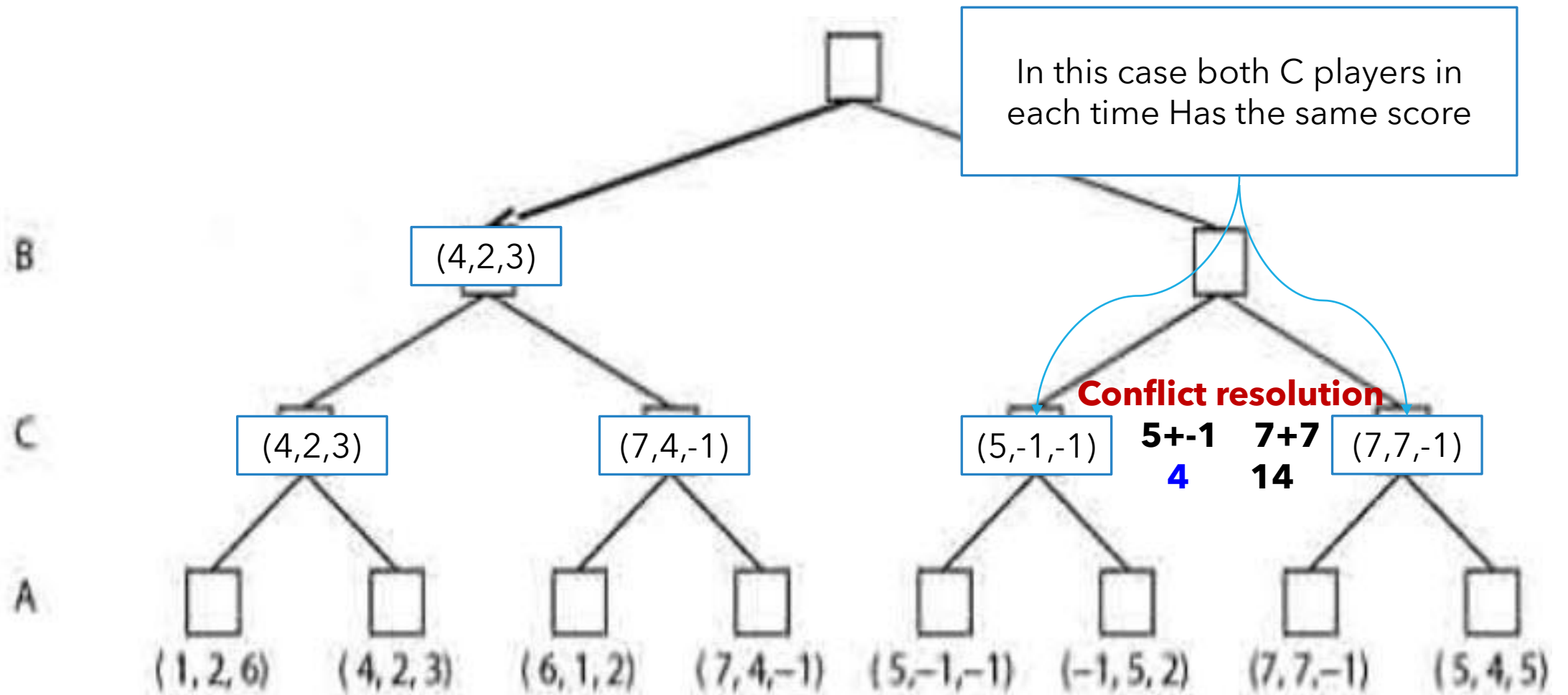
Solve the following 3 players game tree

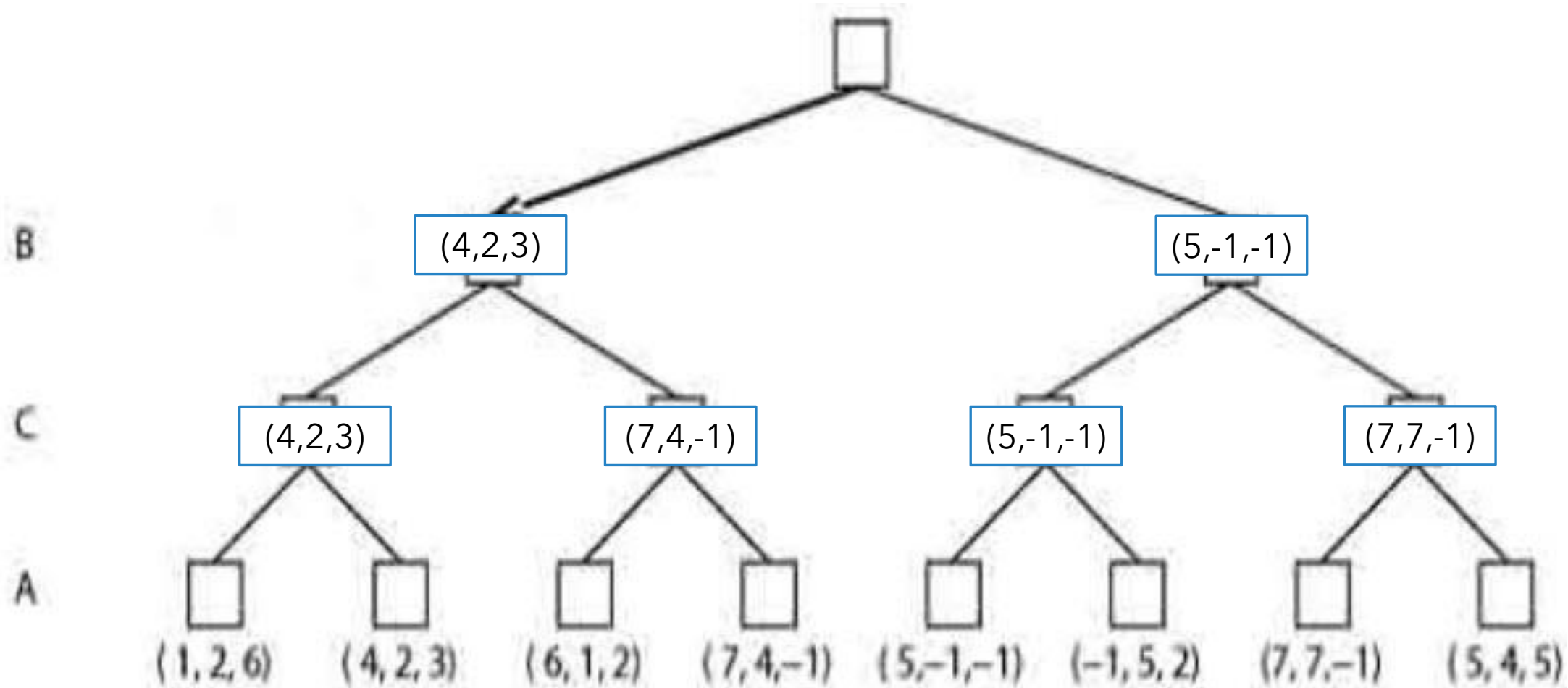


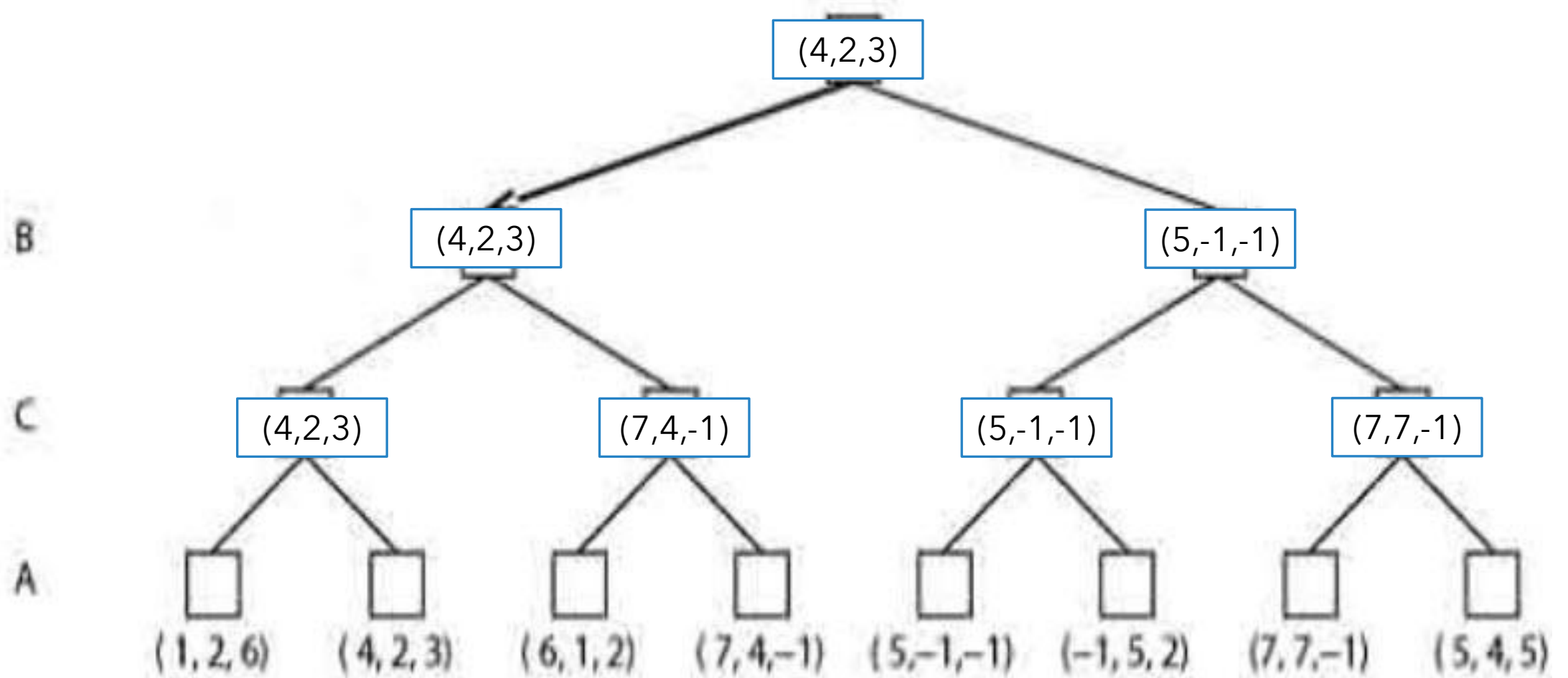






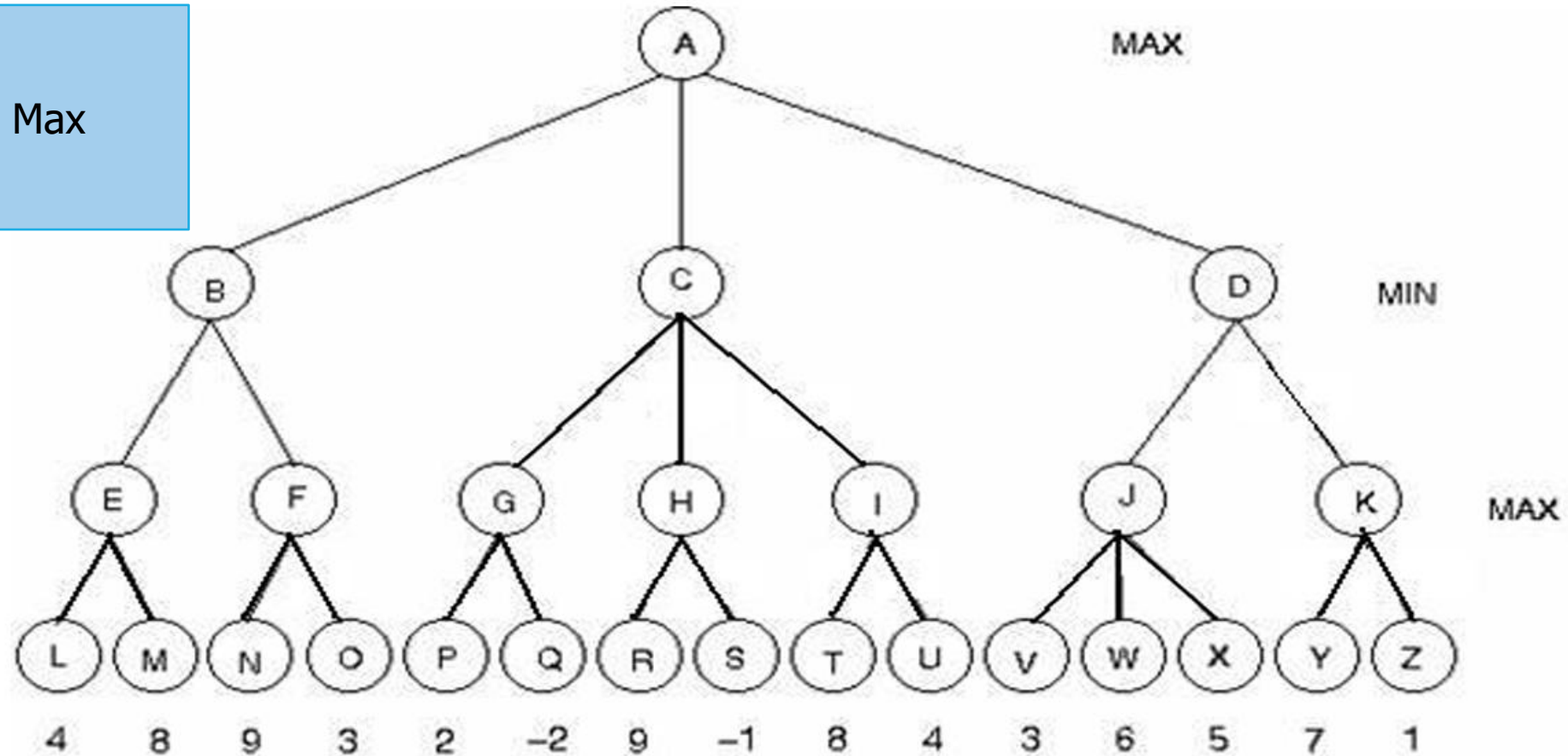


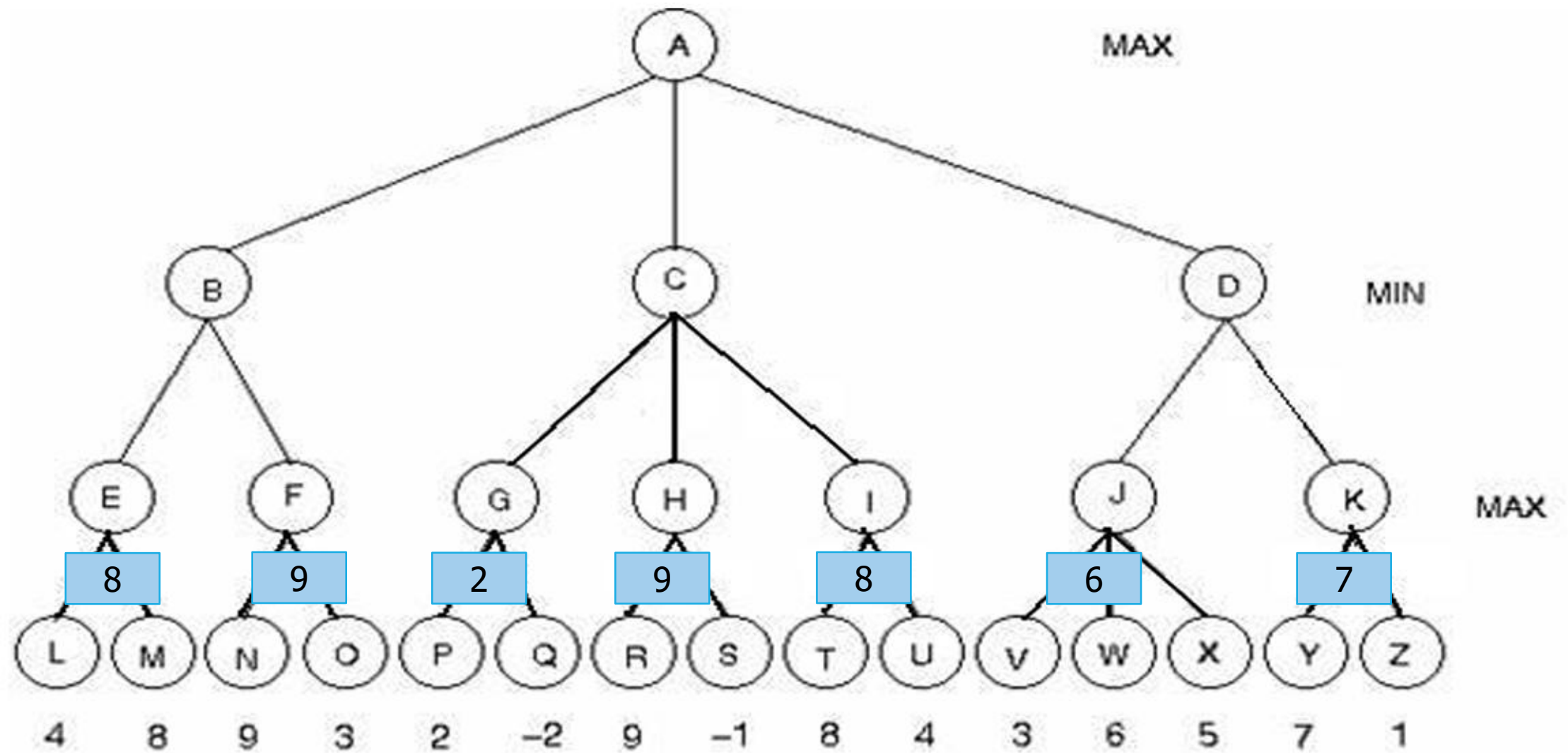


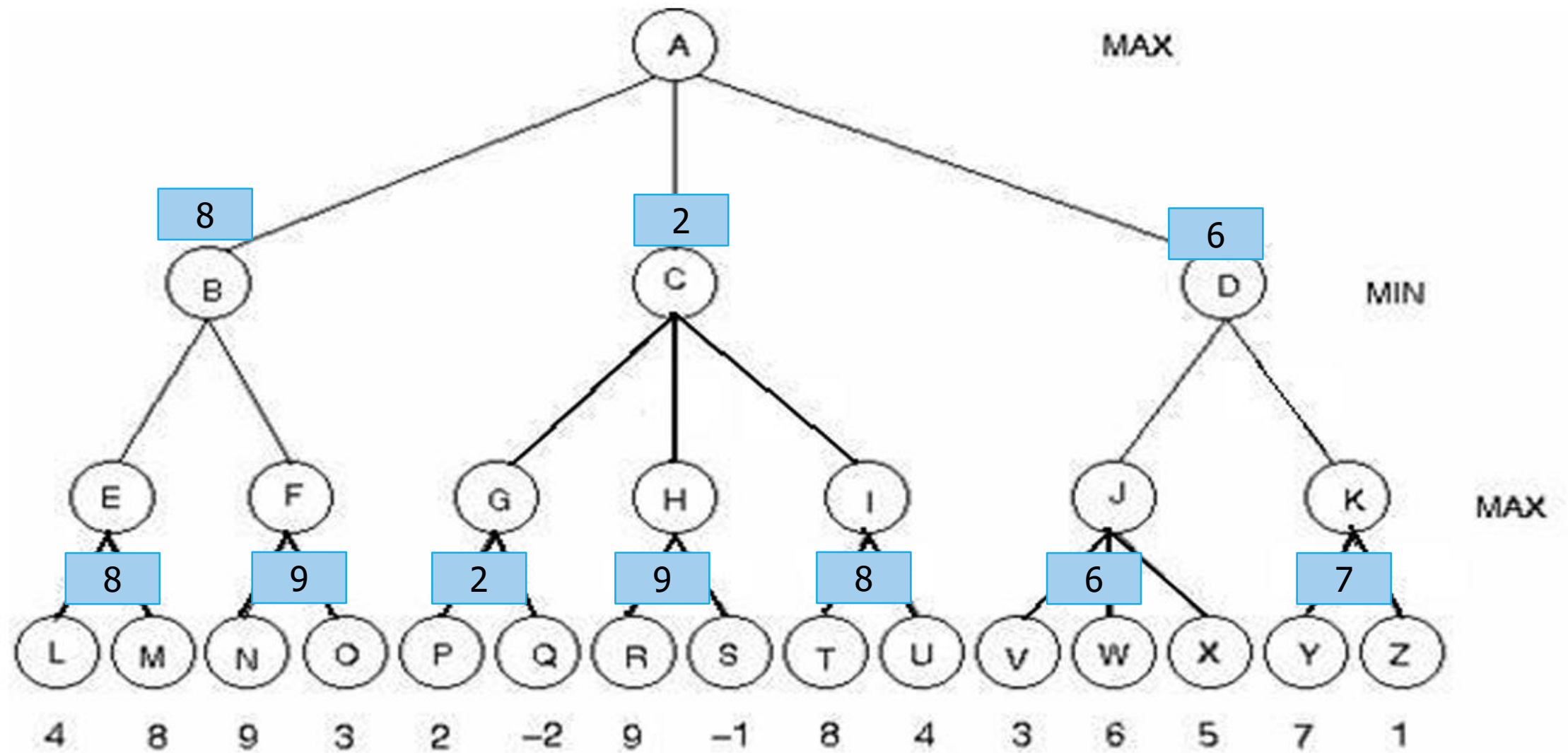


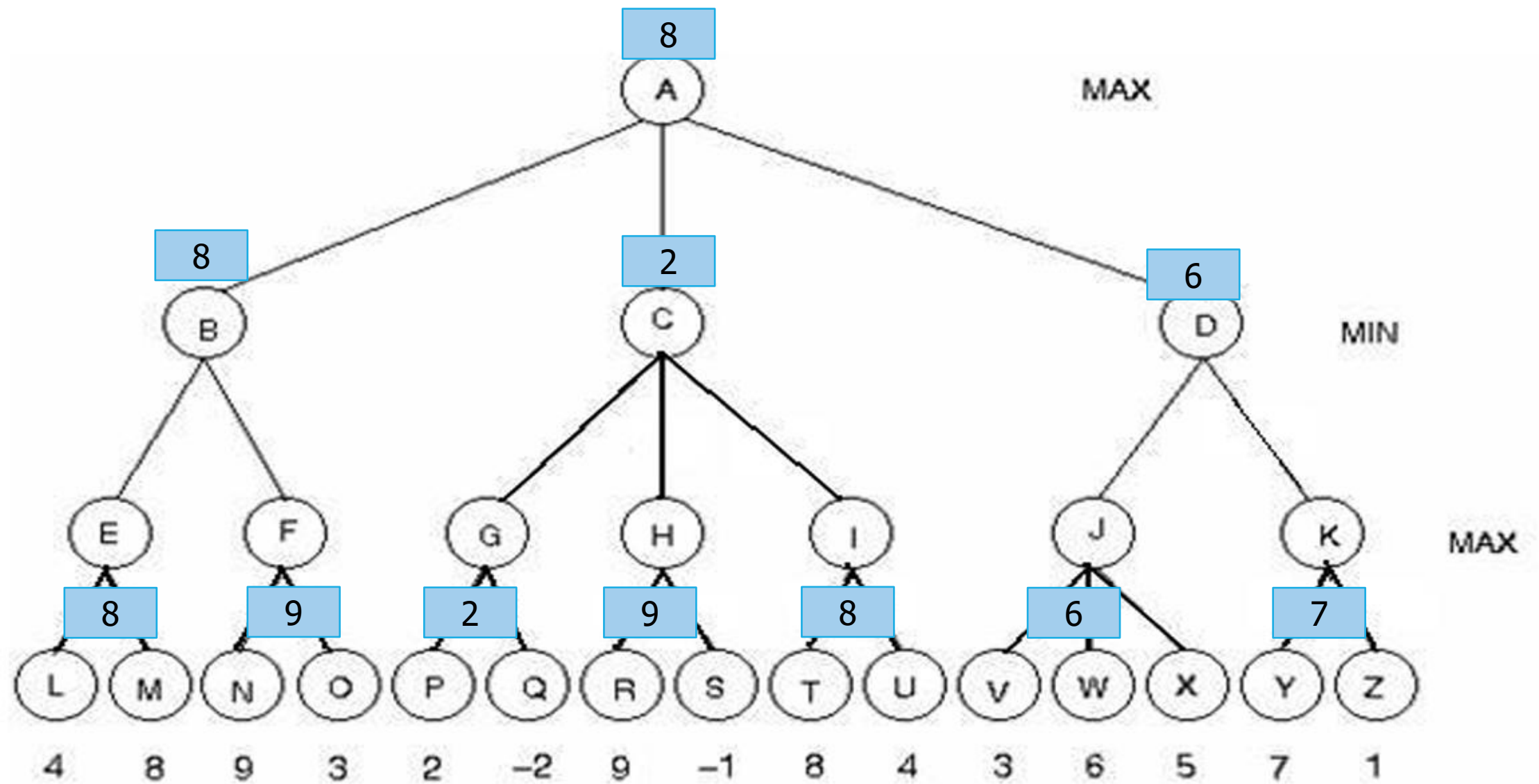
1. Apply the Min-Max Procedure and then the Alpha-Beta Pruning Algorithm for the following 2 players game trees.

First : Min Max









2- Alpha Beta Algorithm

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm that significantly reduces the number of nodes evaluated in the game tree.

It introduces two values, alpha (the best value the maximizer can guarantee) and beta (the best value the minimizer can guarantee).

2- Alpha Beta Algorithm

As the algorithm explores the tree, it "prunes" or cuts off branches that cannot affect the final decision, saving computational resources by skipping over irrelevant moves.

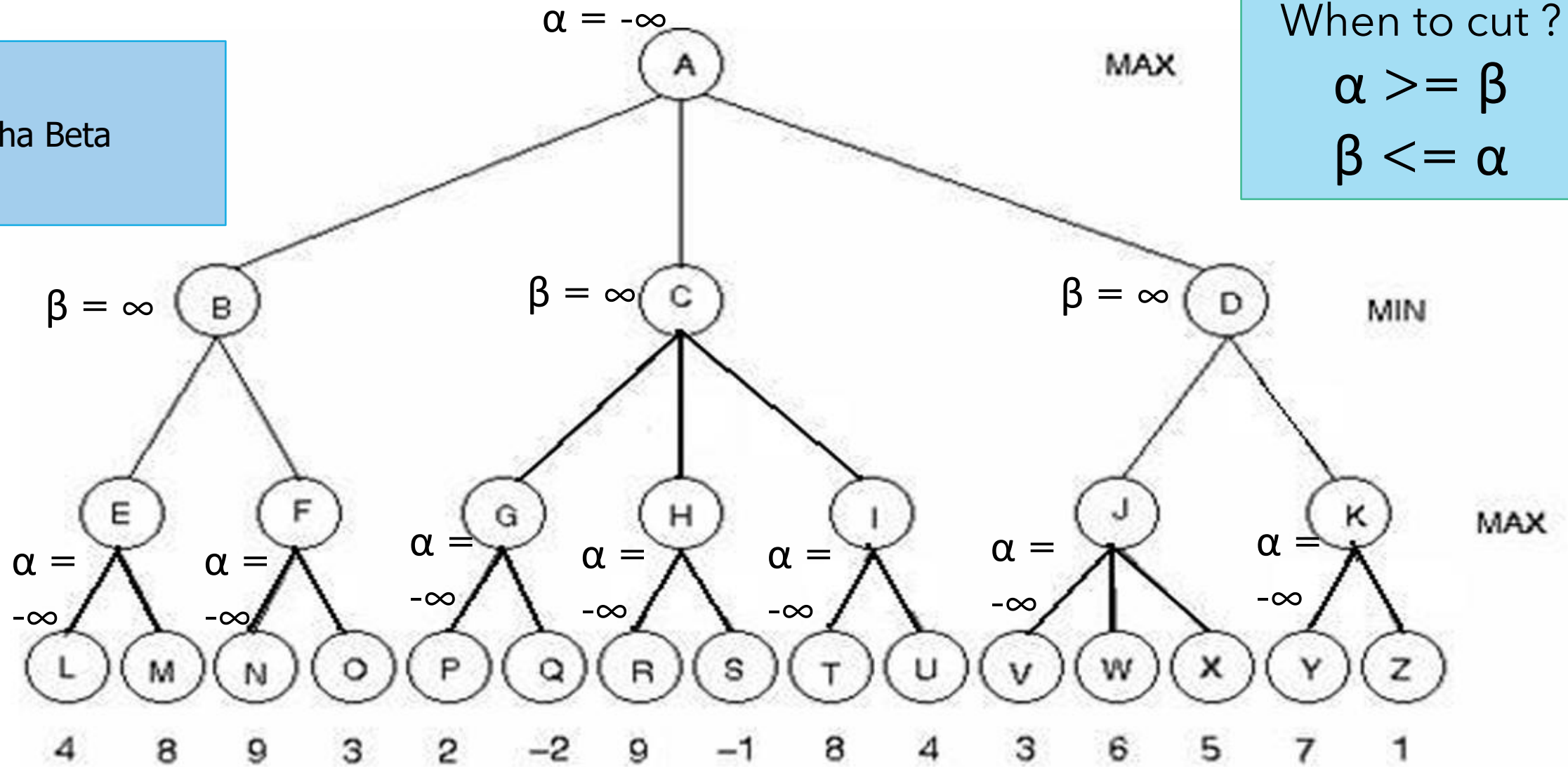
If a node's score exceeds the current alpha or beta values, further examination of that path stops since it won't influence the optimal choice. This technique makes the search process more efficient without compromising the outcome.

Alpha Beta

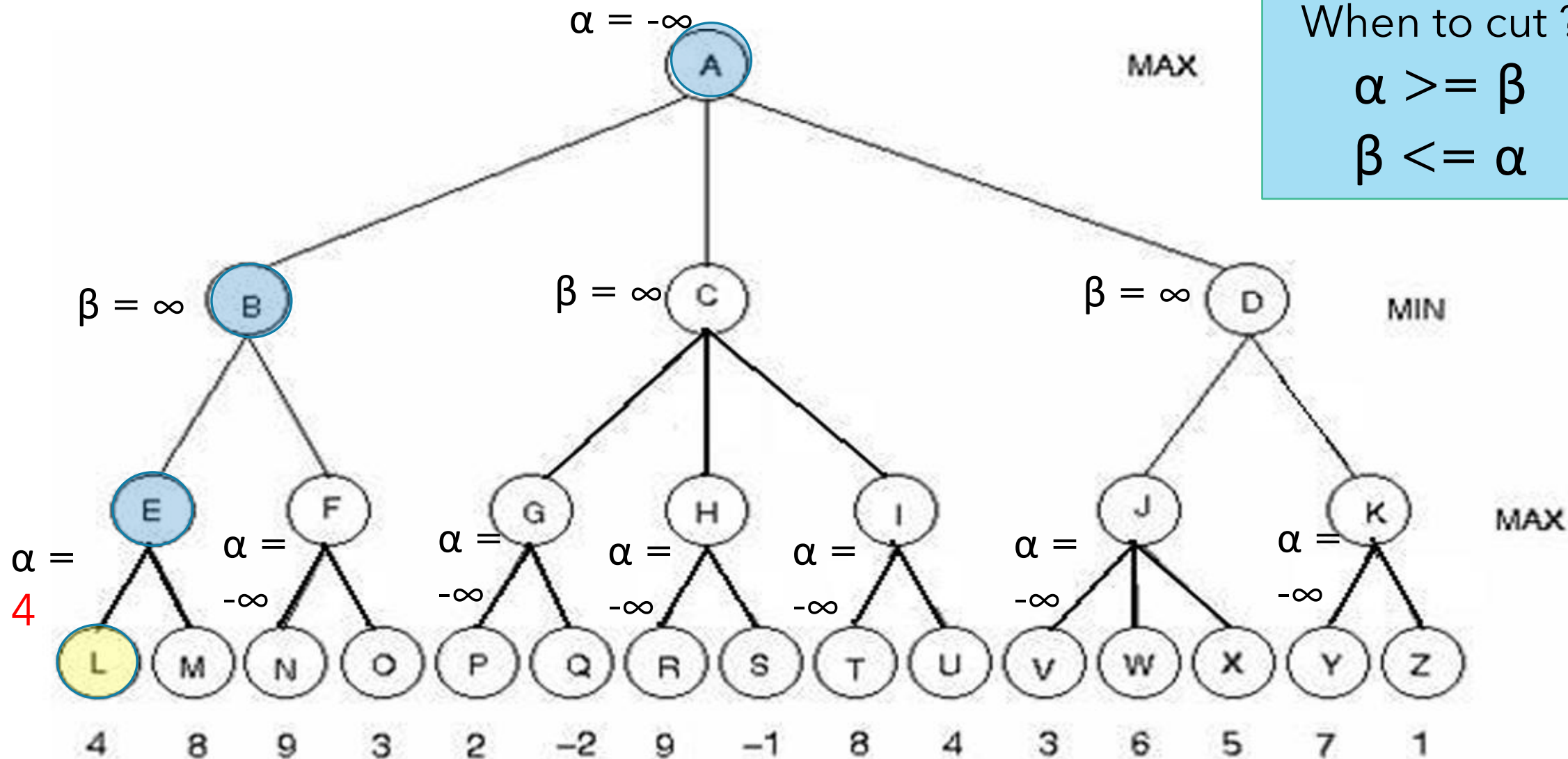
When to cut ?

$$\alpha \geq \beta$$

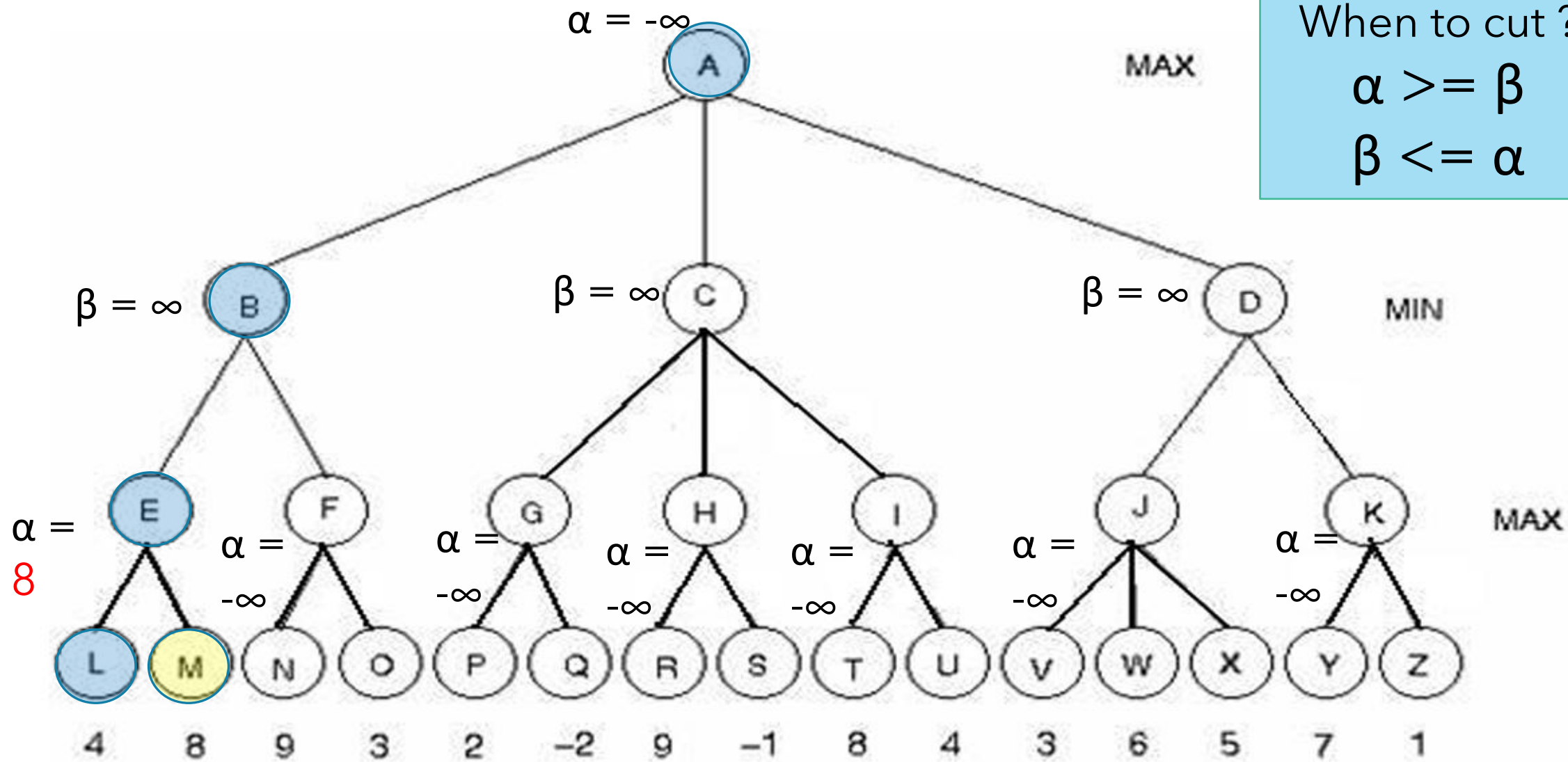
$$\beta \leq \alpha$$



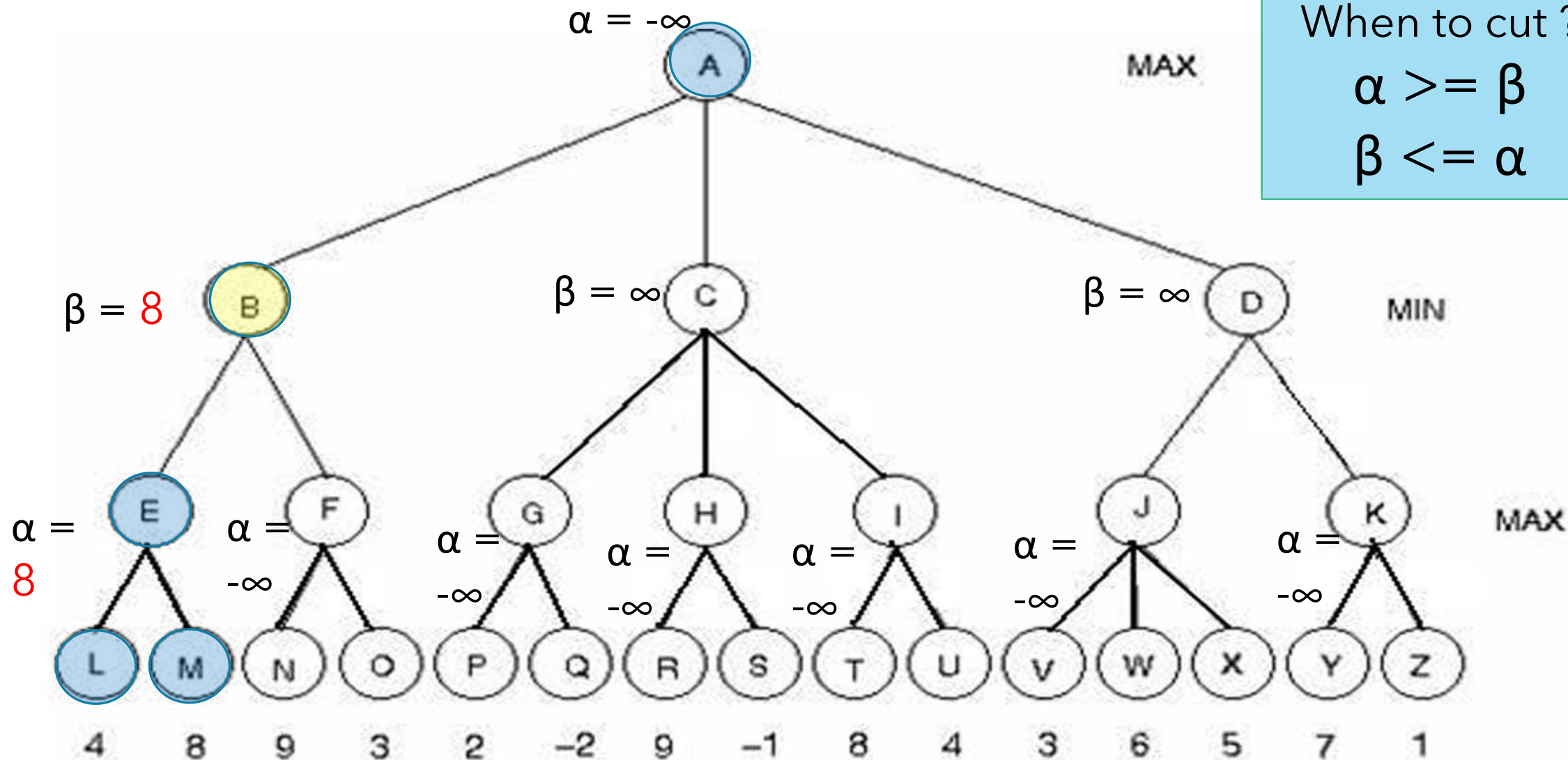
When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$



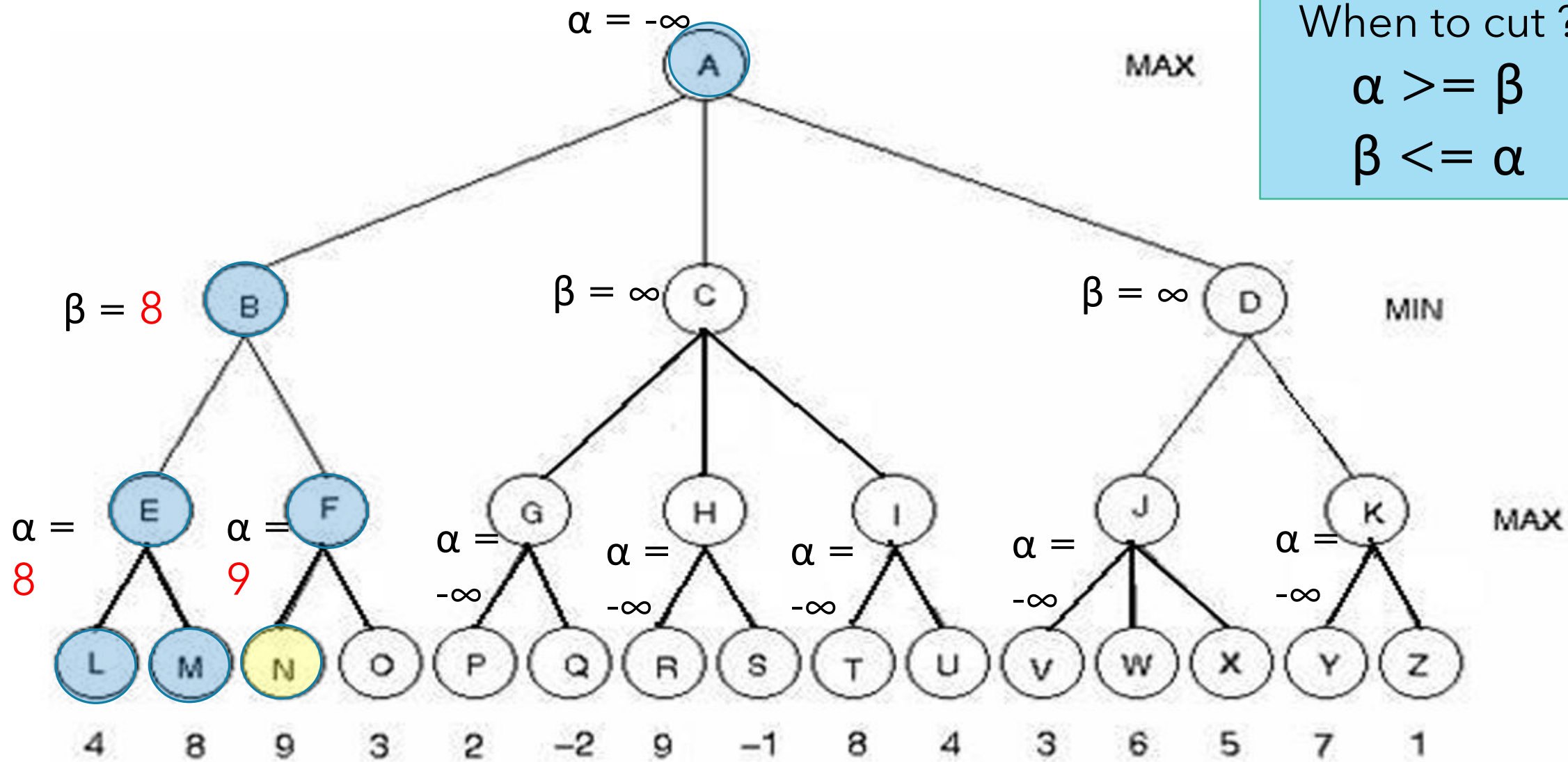
When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$



When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$

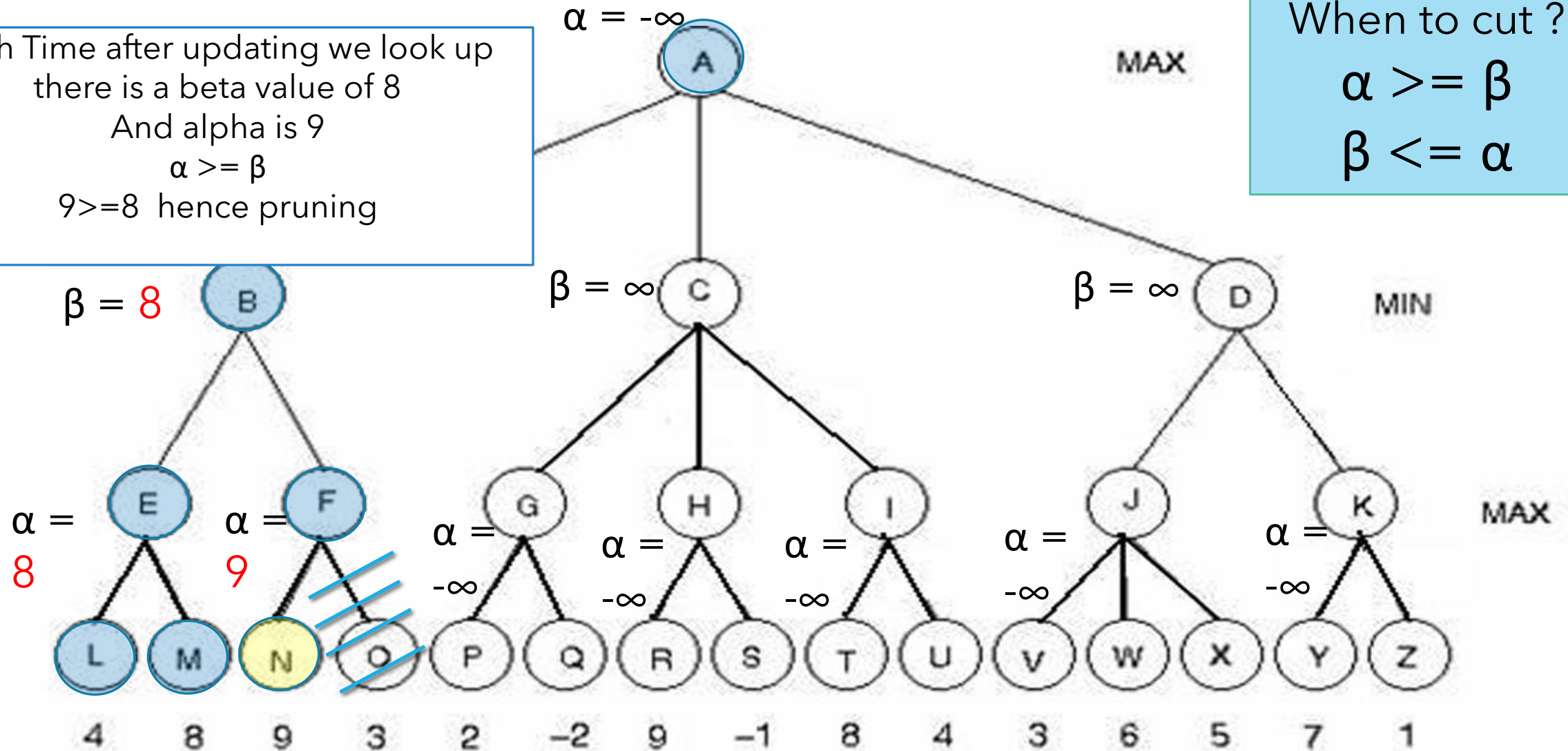


When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$

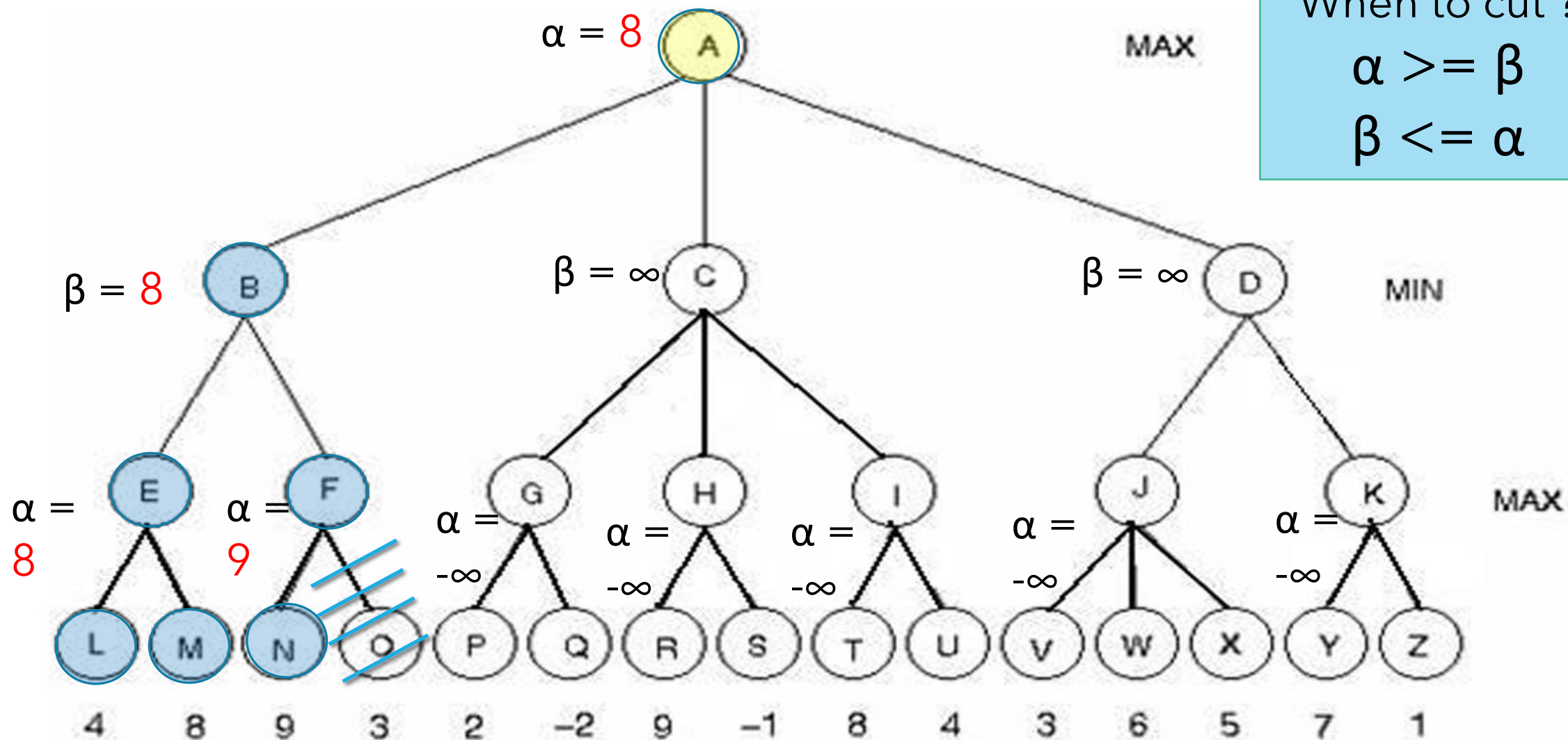


Each Time after updating we look up
there is a beta value of 8
And alpha is 9
 $\alpha \geq \beta$
 $9 \geq 8$ hence pruning

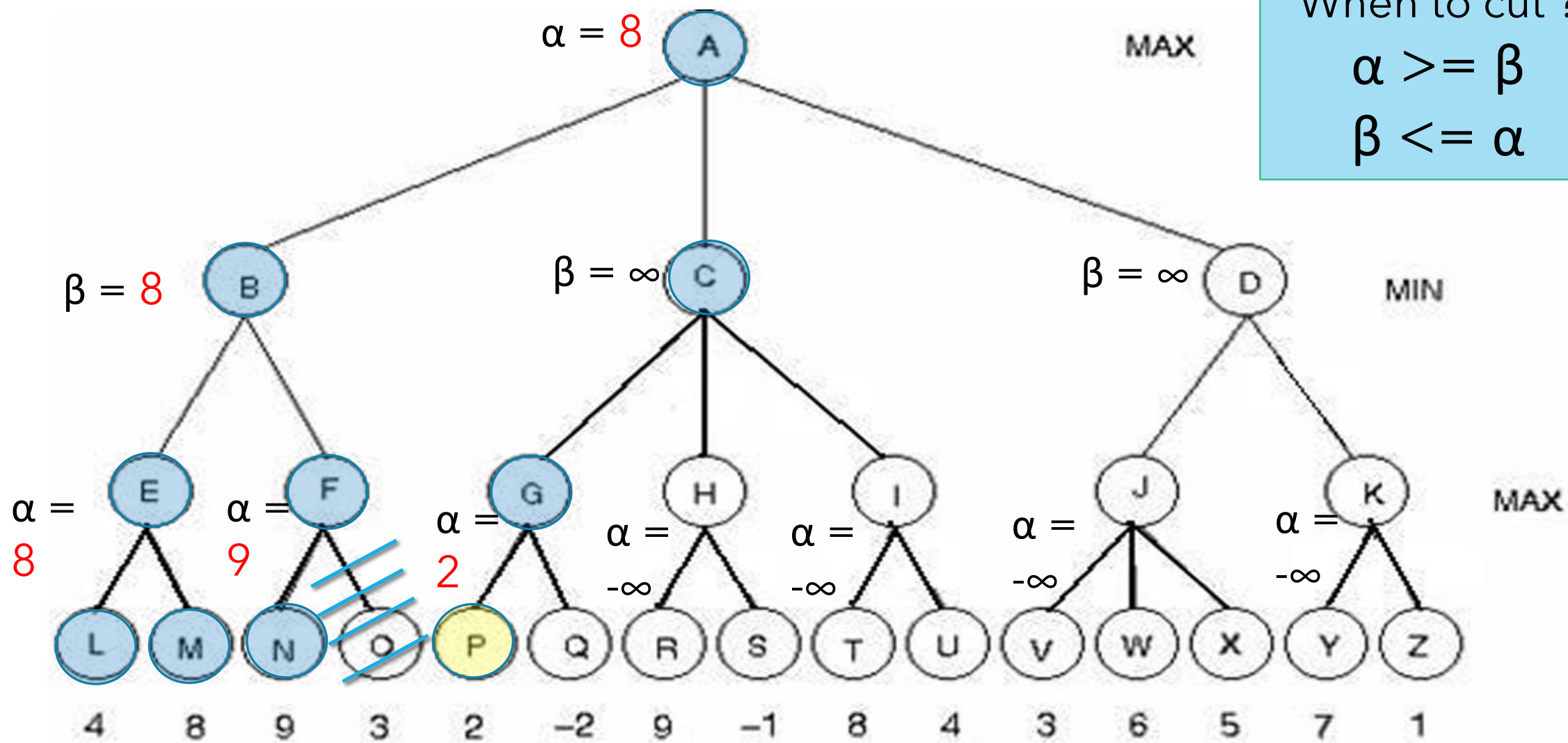
When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$



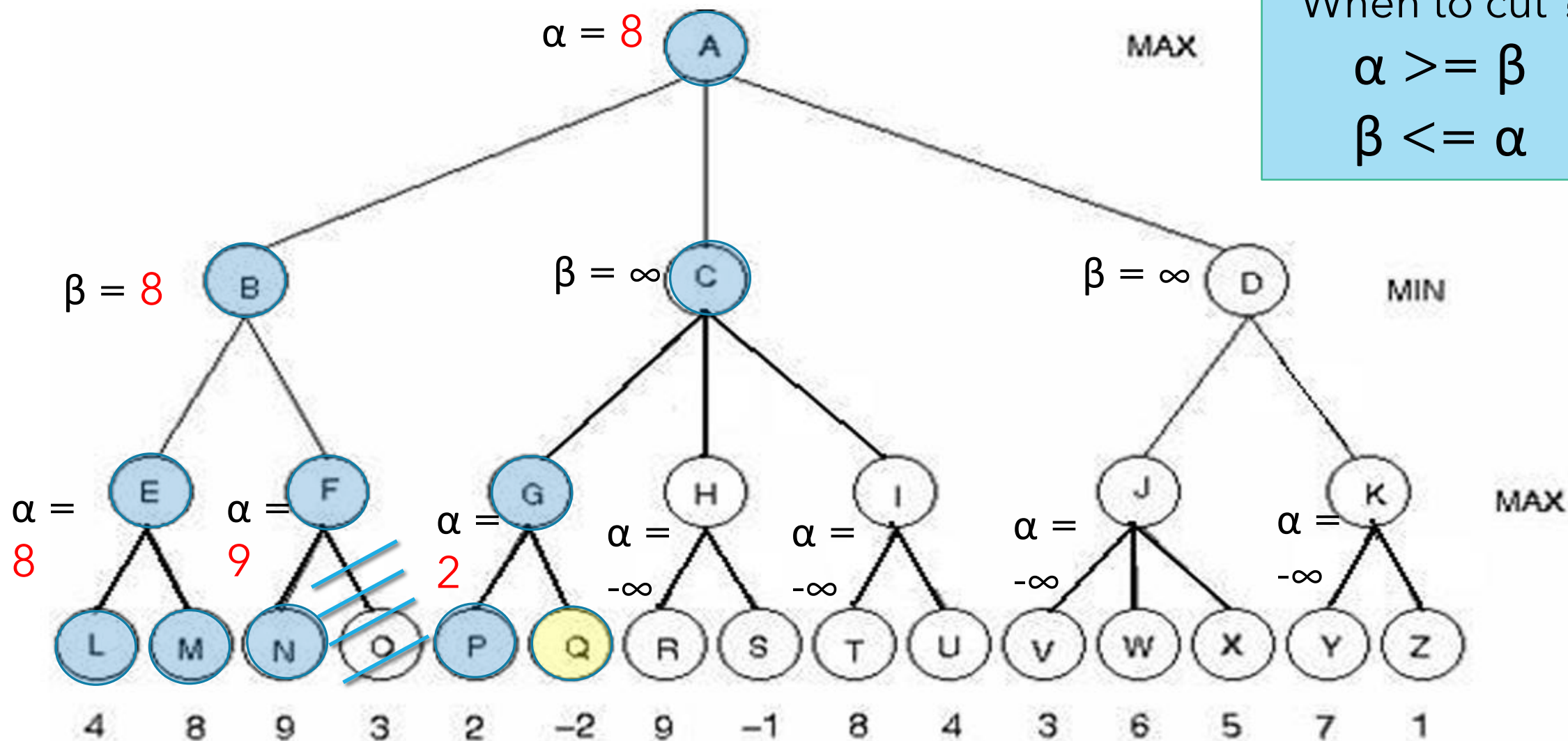
When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$



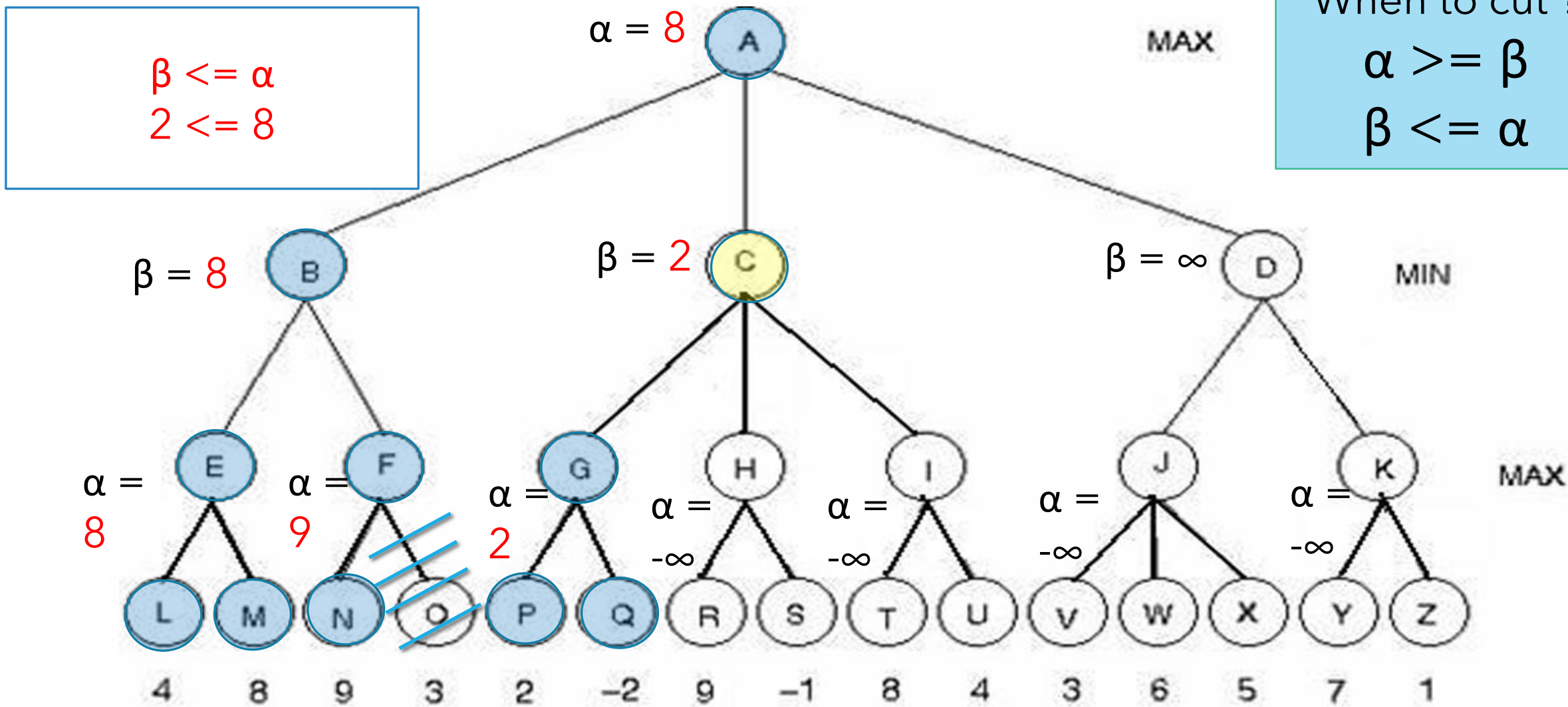
When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$



When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$



When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$

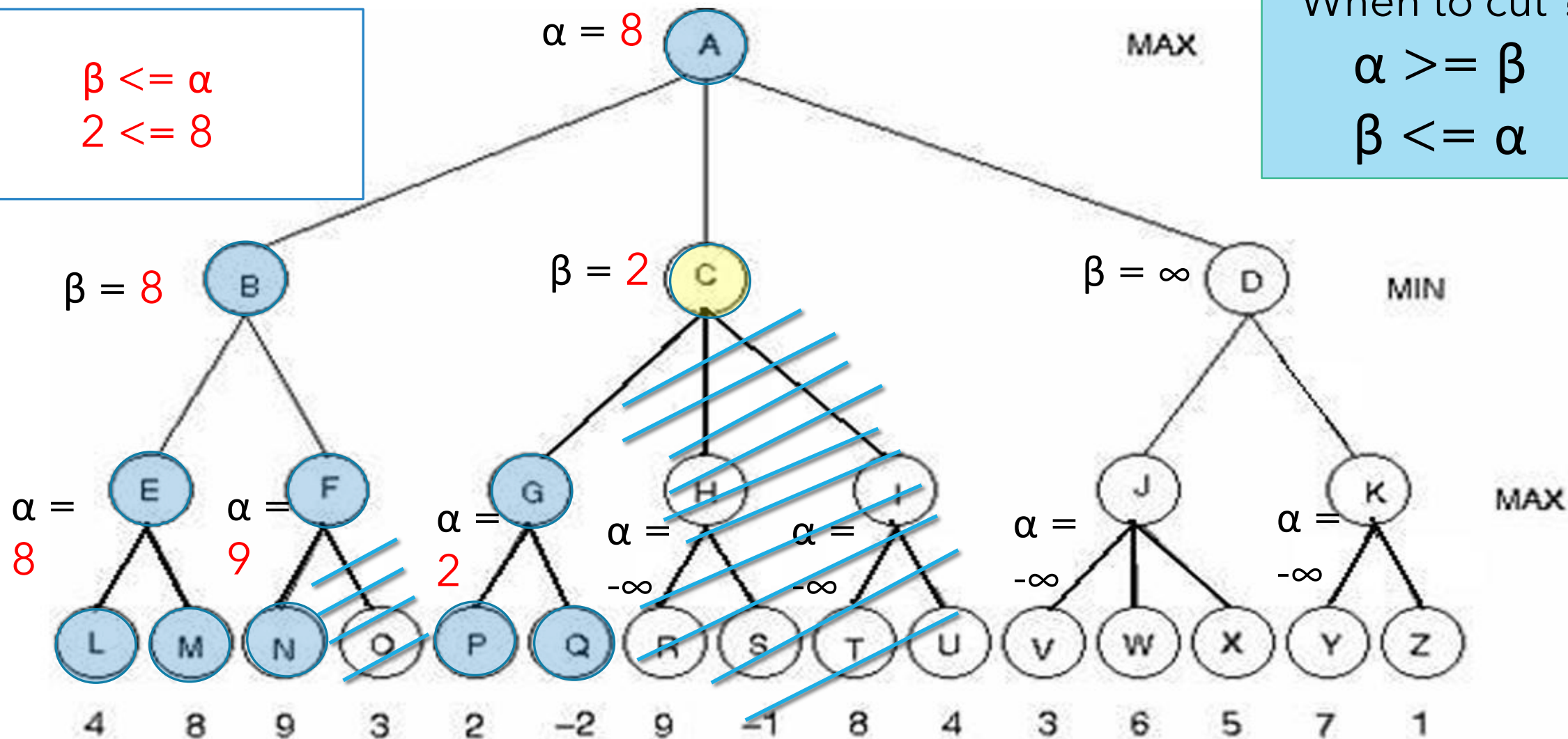


$$\beta \leq \alpha$$

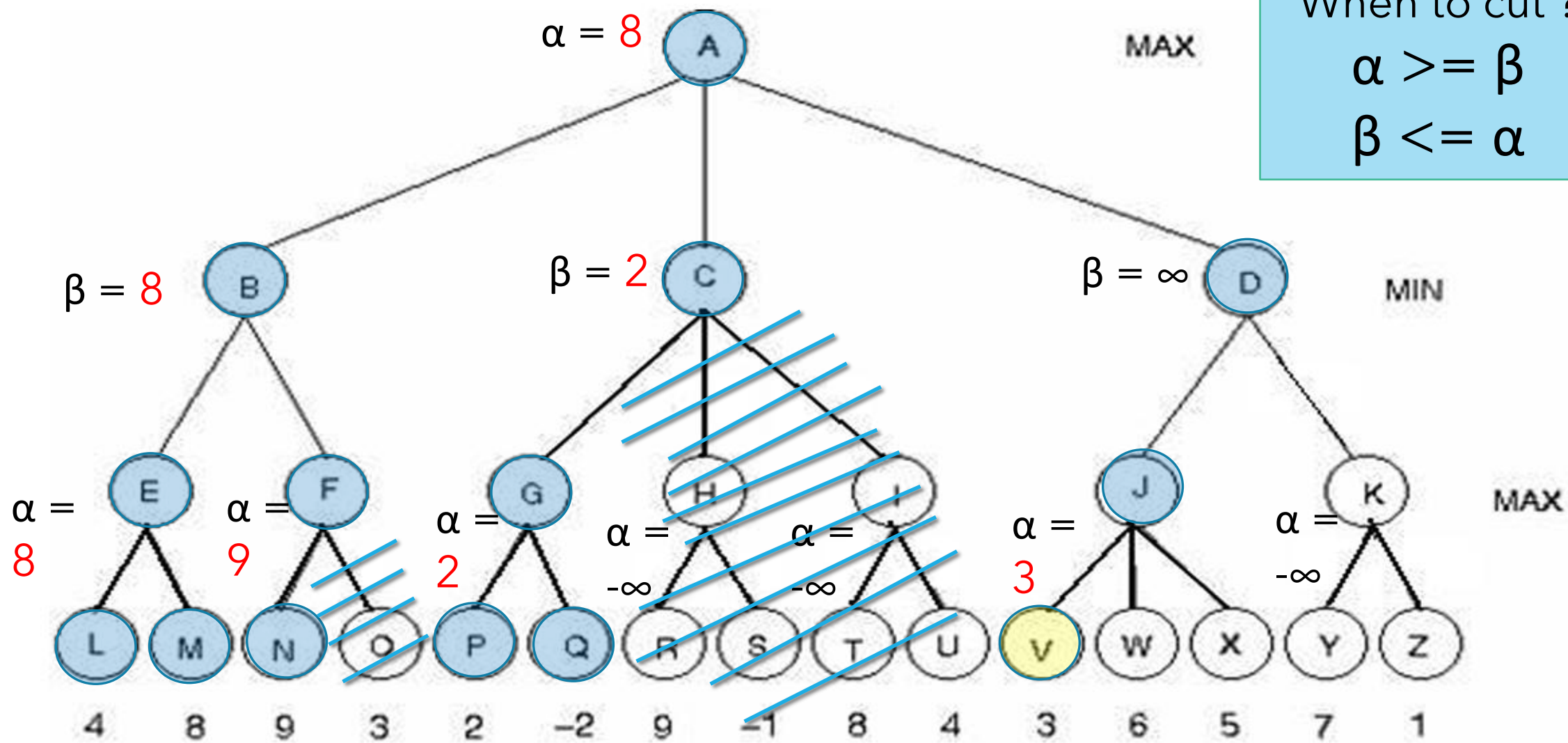
$$2 \leq 8$$

When to cut ?

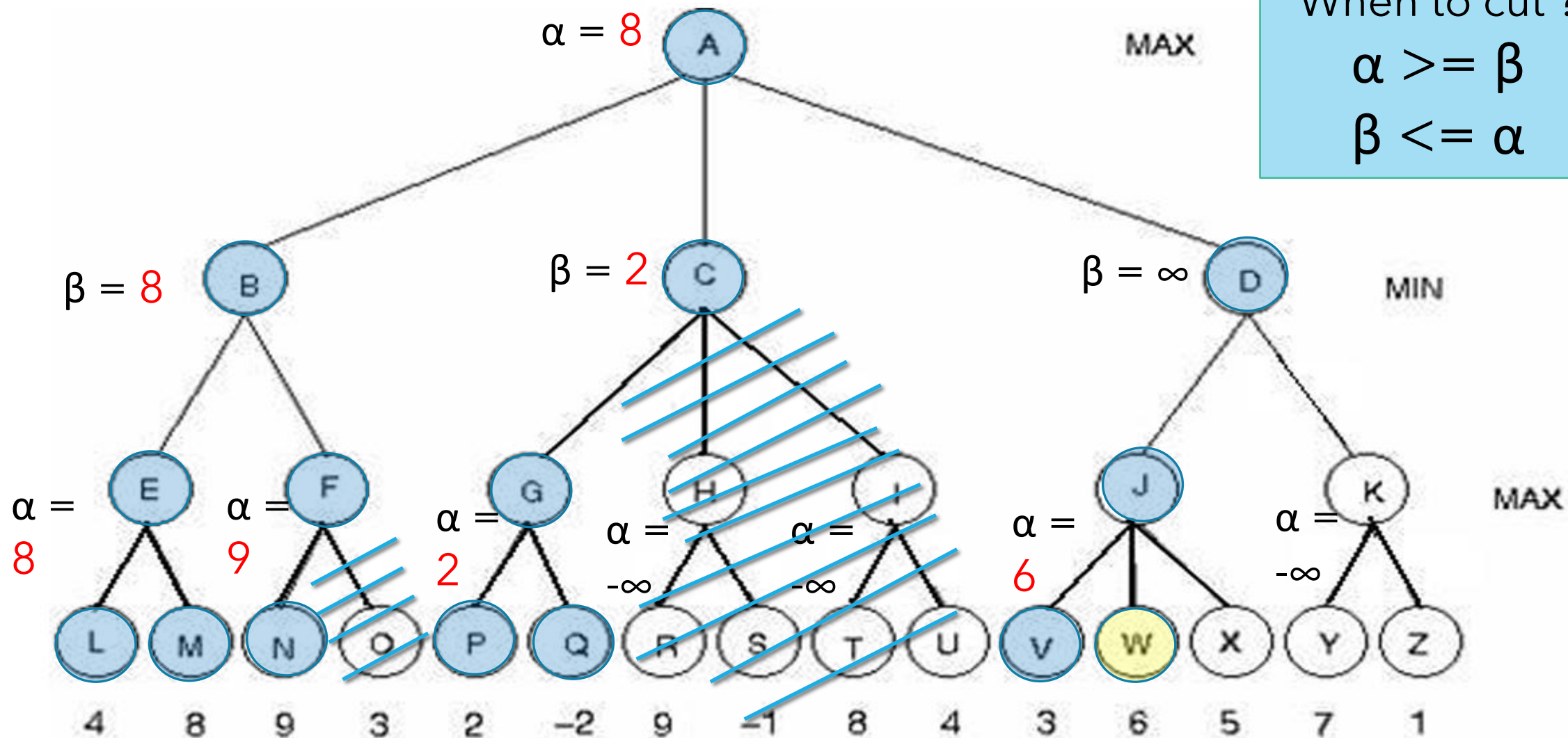
$$\alpha \geq \beta$$

$$\beta \leq \alpha$$


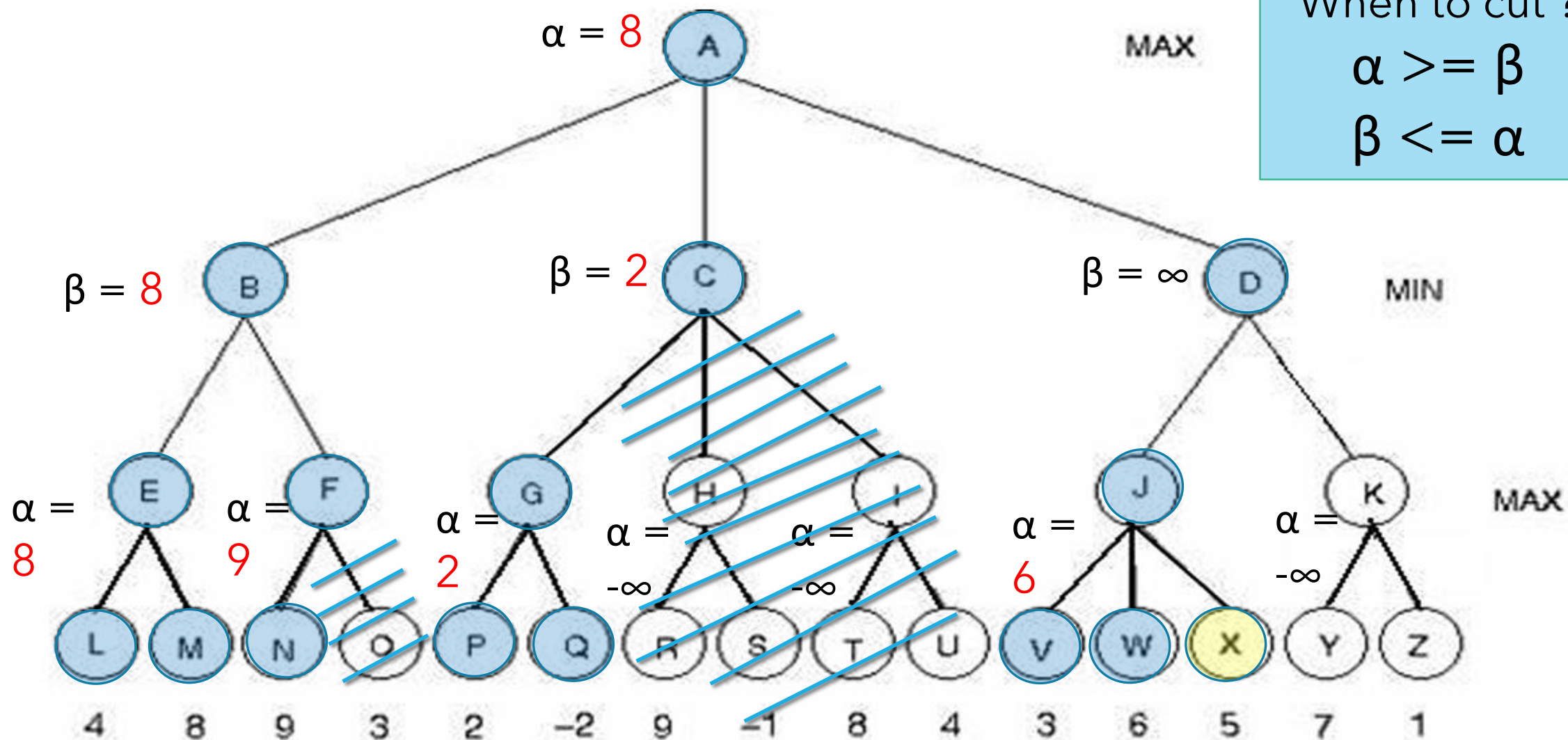
When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$



When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$



When to cut ?
 $\alpha \geq \beta$
 $\beta \leq \alpha$

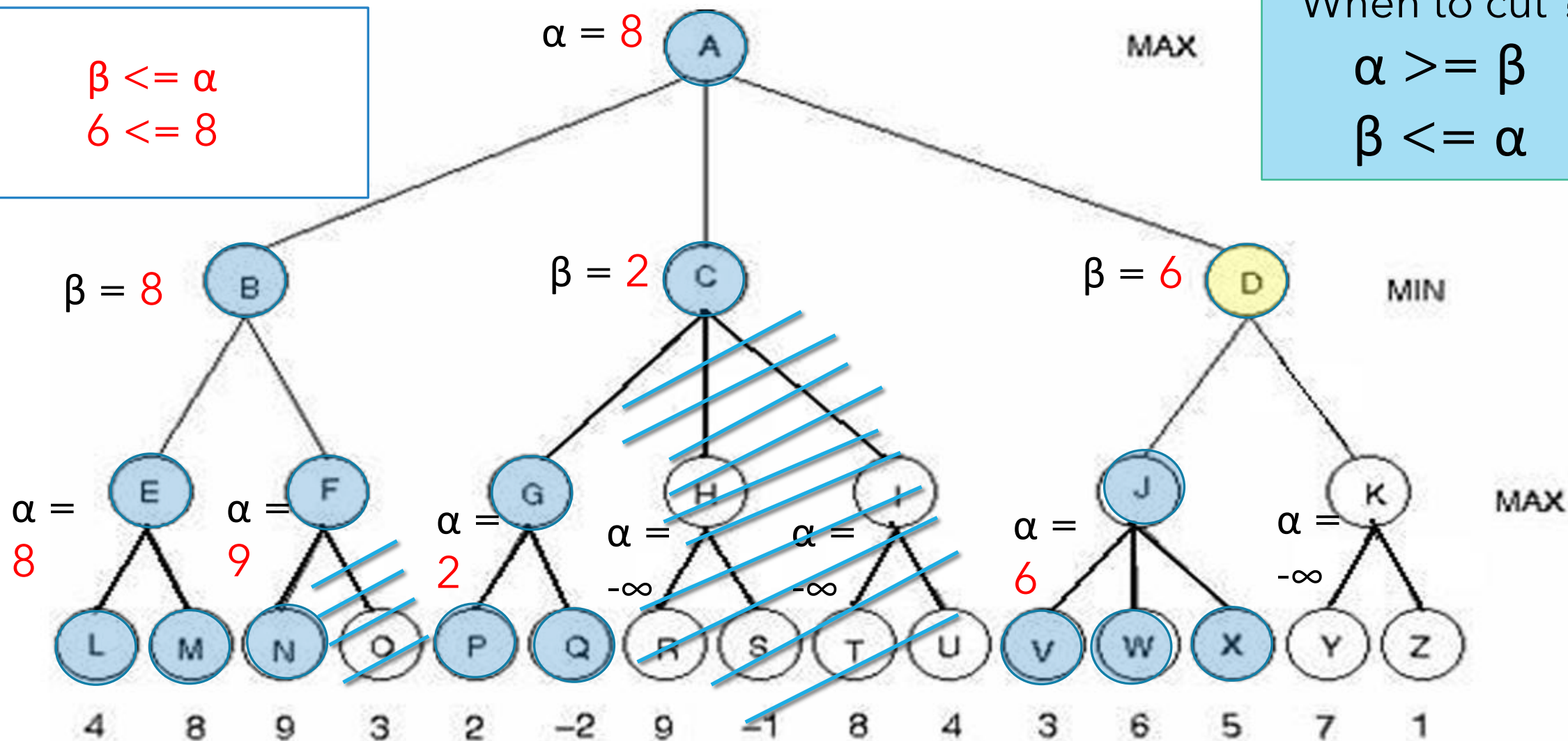


$$\beta \leq \alpha$$

$$6 \leq 8$$

When to cut ?

$$\alpha \geq \beta$$

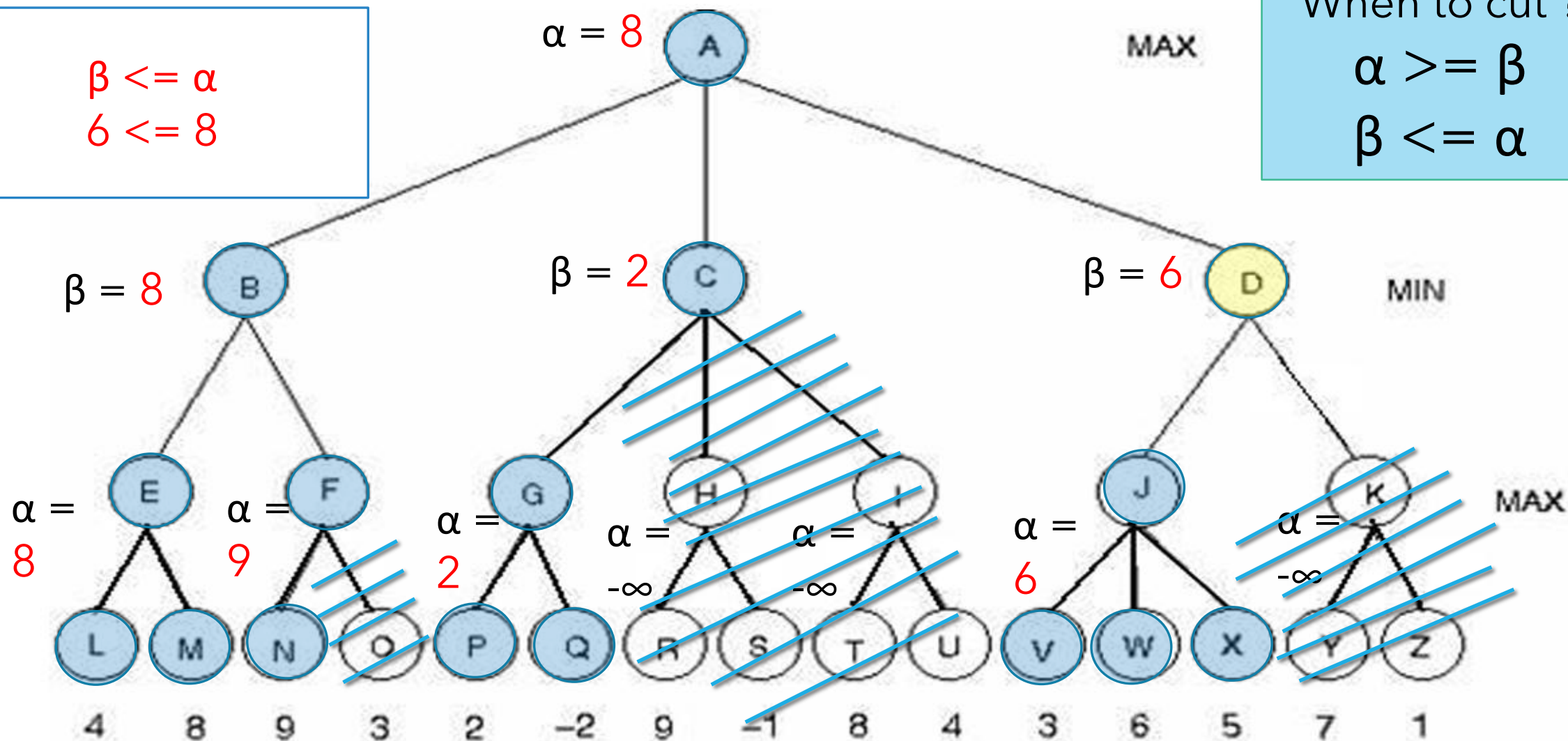
$$\beta \leq \alpha$$


$$\beta \leq \alpha$$

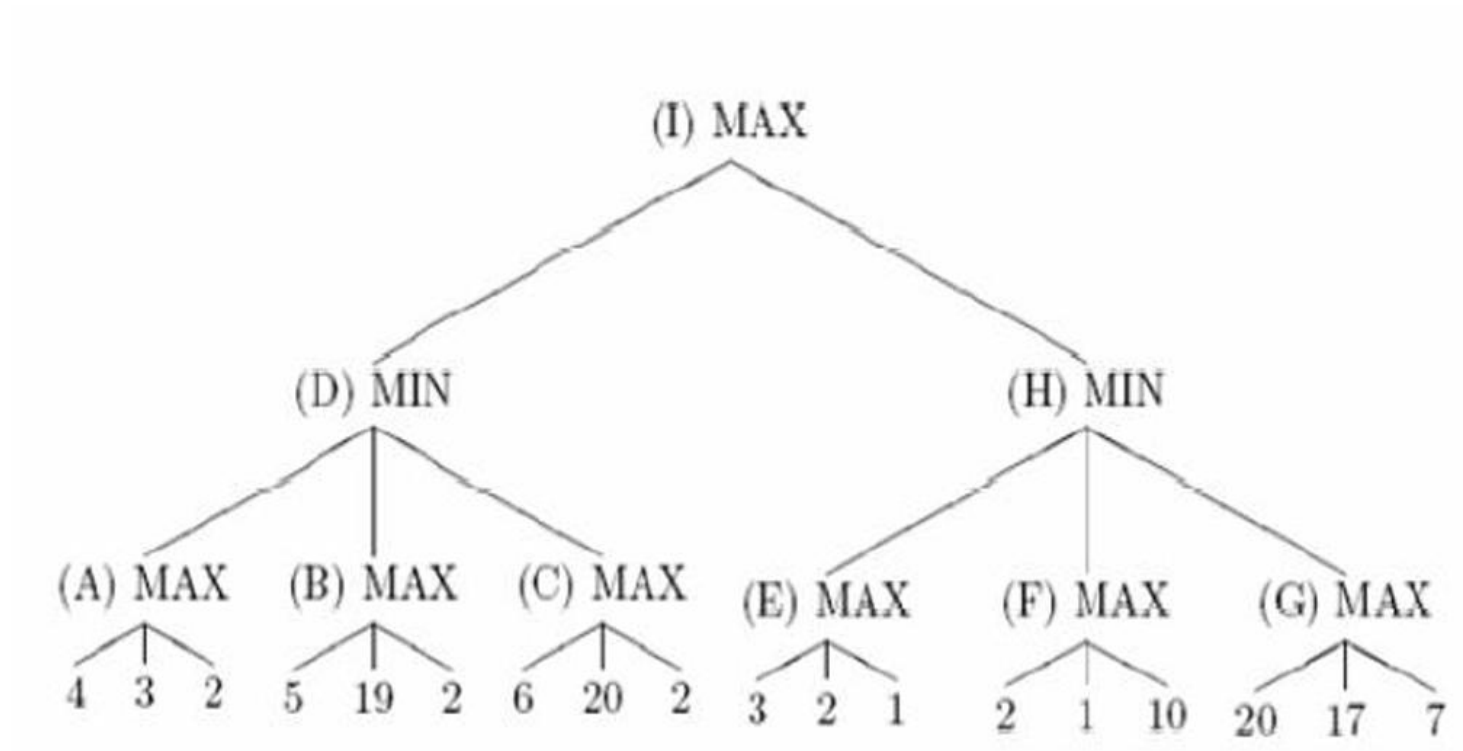
$$6 \leq 8$$

When to cut ?

$$\alpha \geq \beta$$

$$\beta \leq \alpha$$


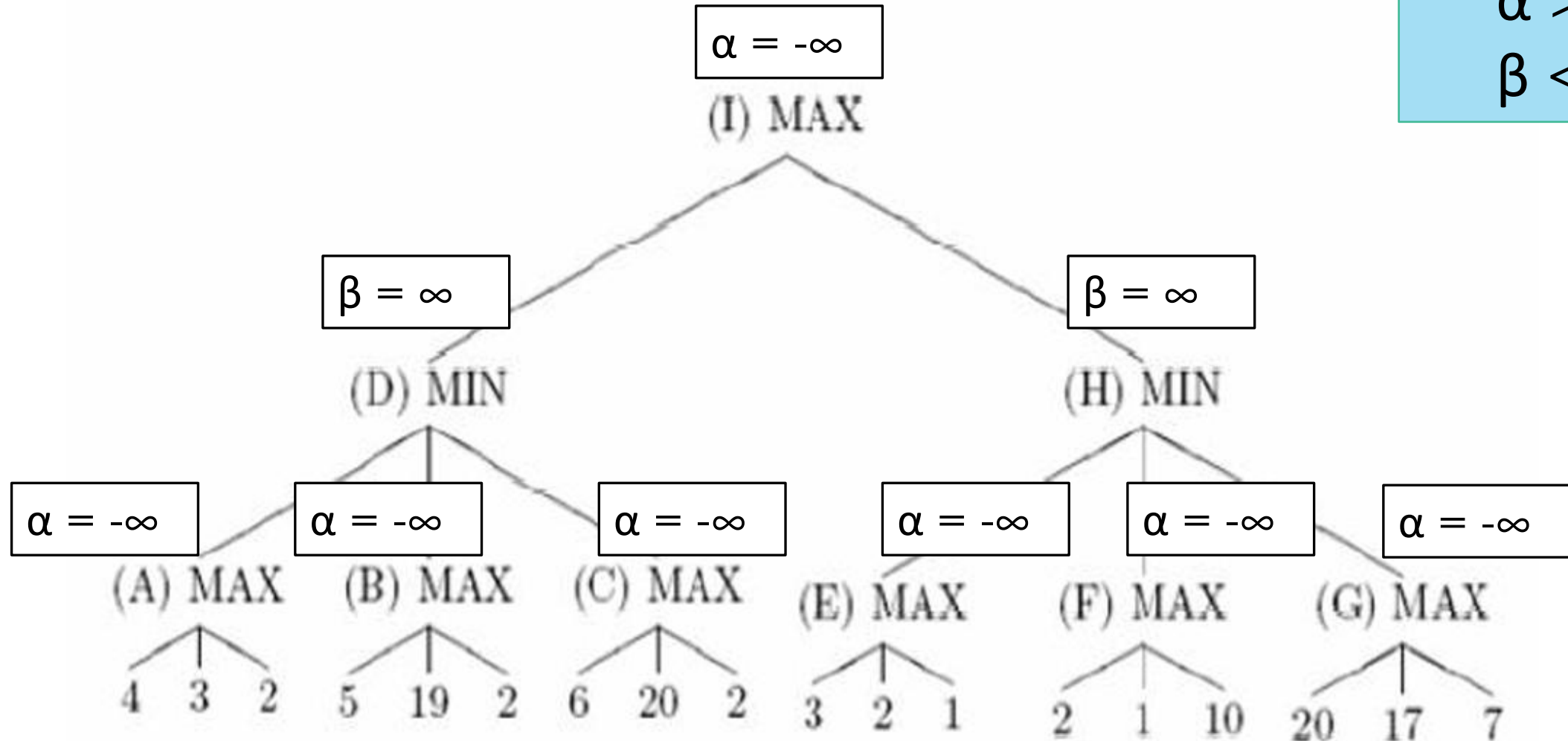
Example : Get the Value of I after applying alpha beta algorithm



When to cut ?

$$\alpha \geq \beta$$

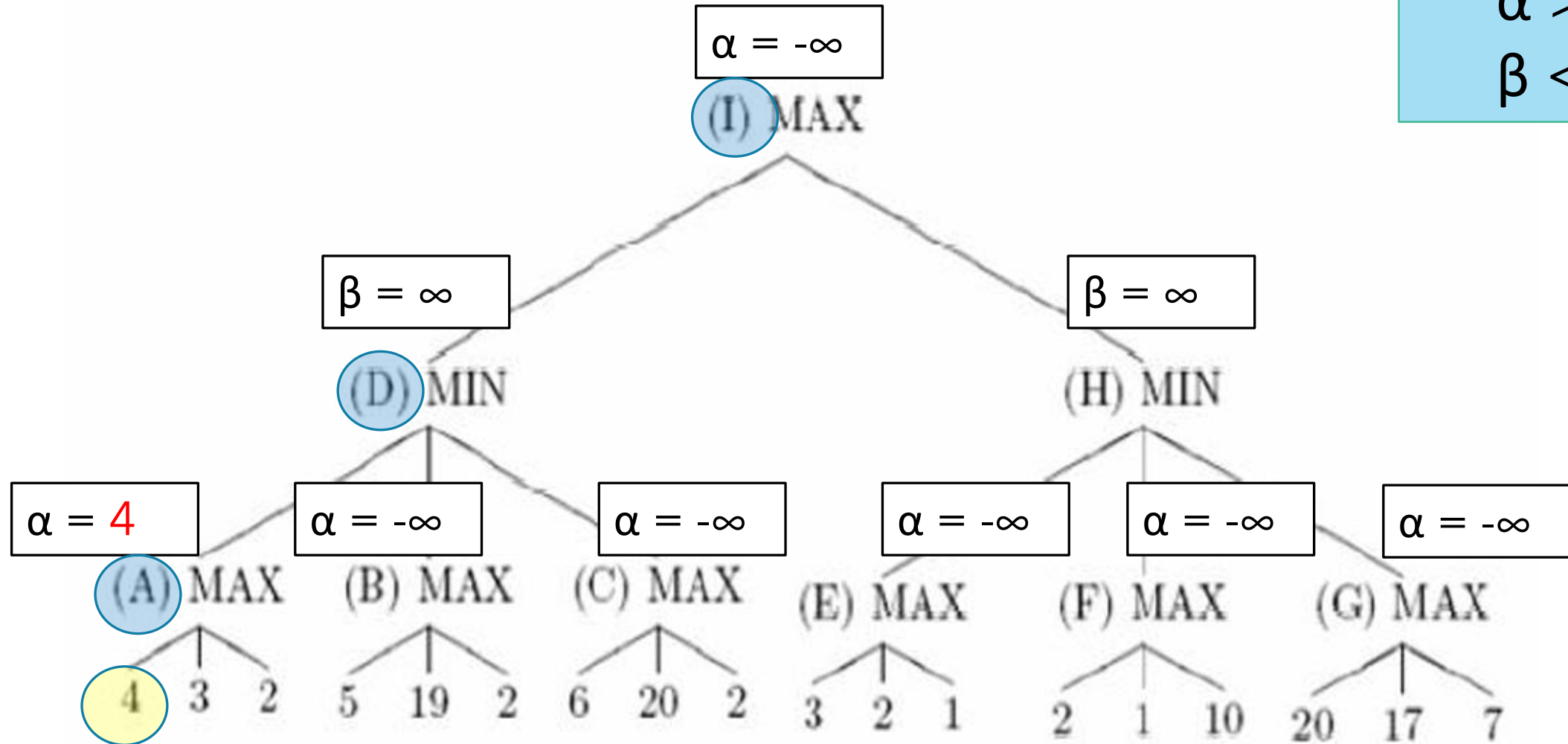
$$\beta \leq \alpha$$



When to cut ?

$$\alpha \geq \beta$$

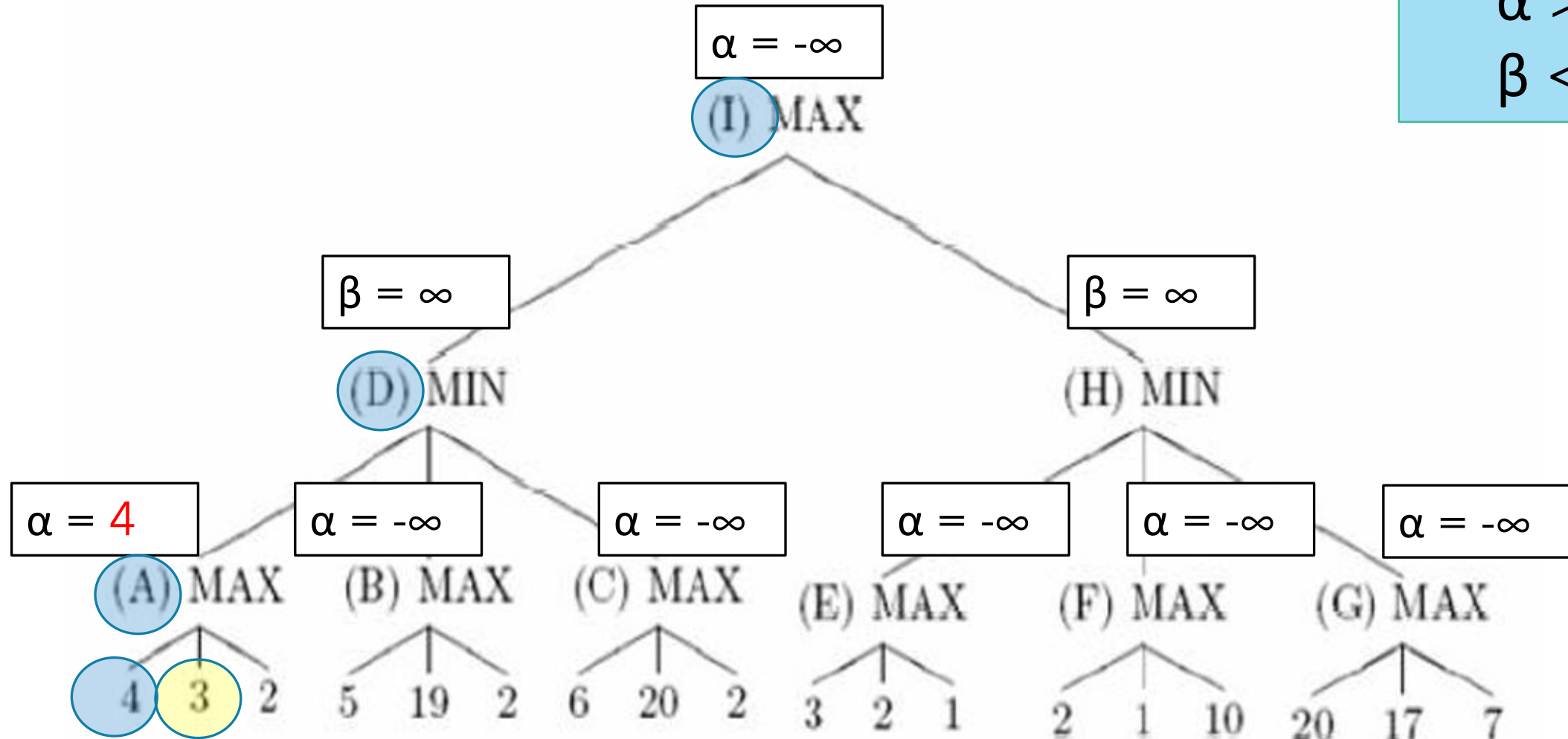
$$\beta \leq \alpha$$



When to cut ?

$$\alpha \geq \beta$$

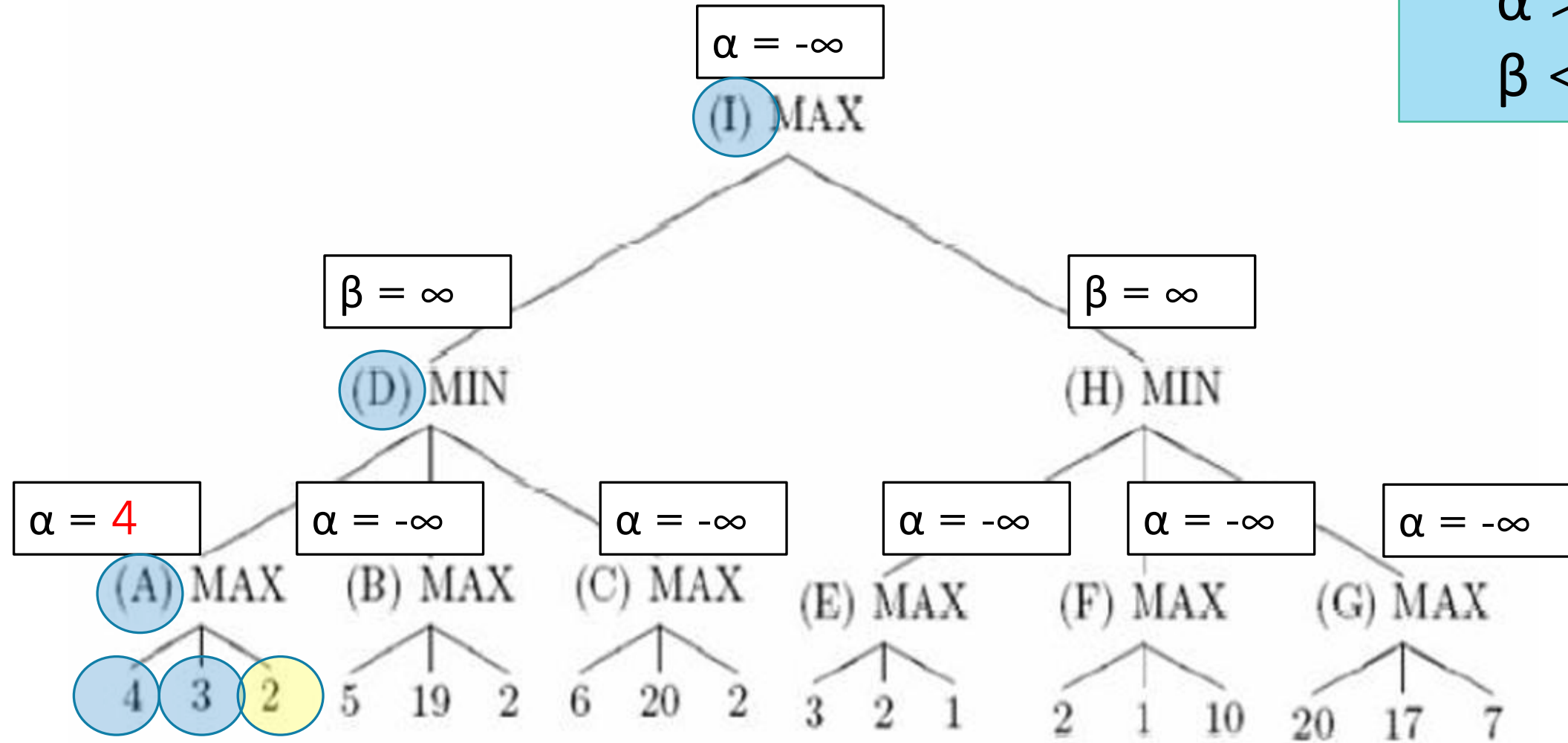
$$\beta \leq \alpha$$



When to cut ?

$$\alpha \geq \beta$$

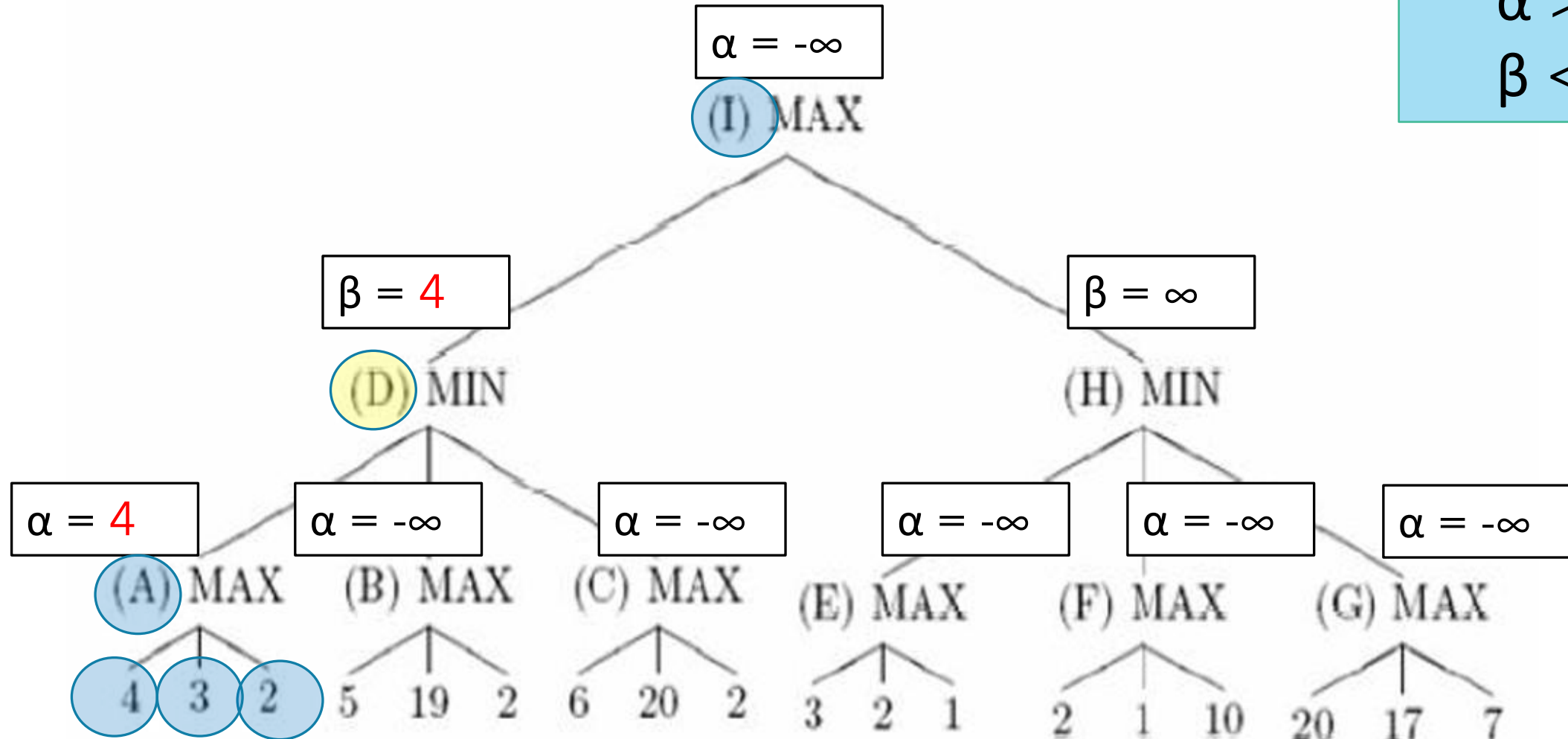
$$\beta \leq \alpha$$



When to cut ?

$$\alpha \geq \beta$$

$$\beta \leq \alpha$$



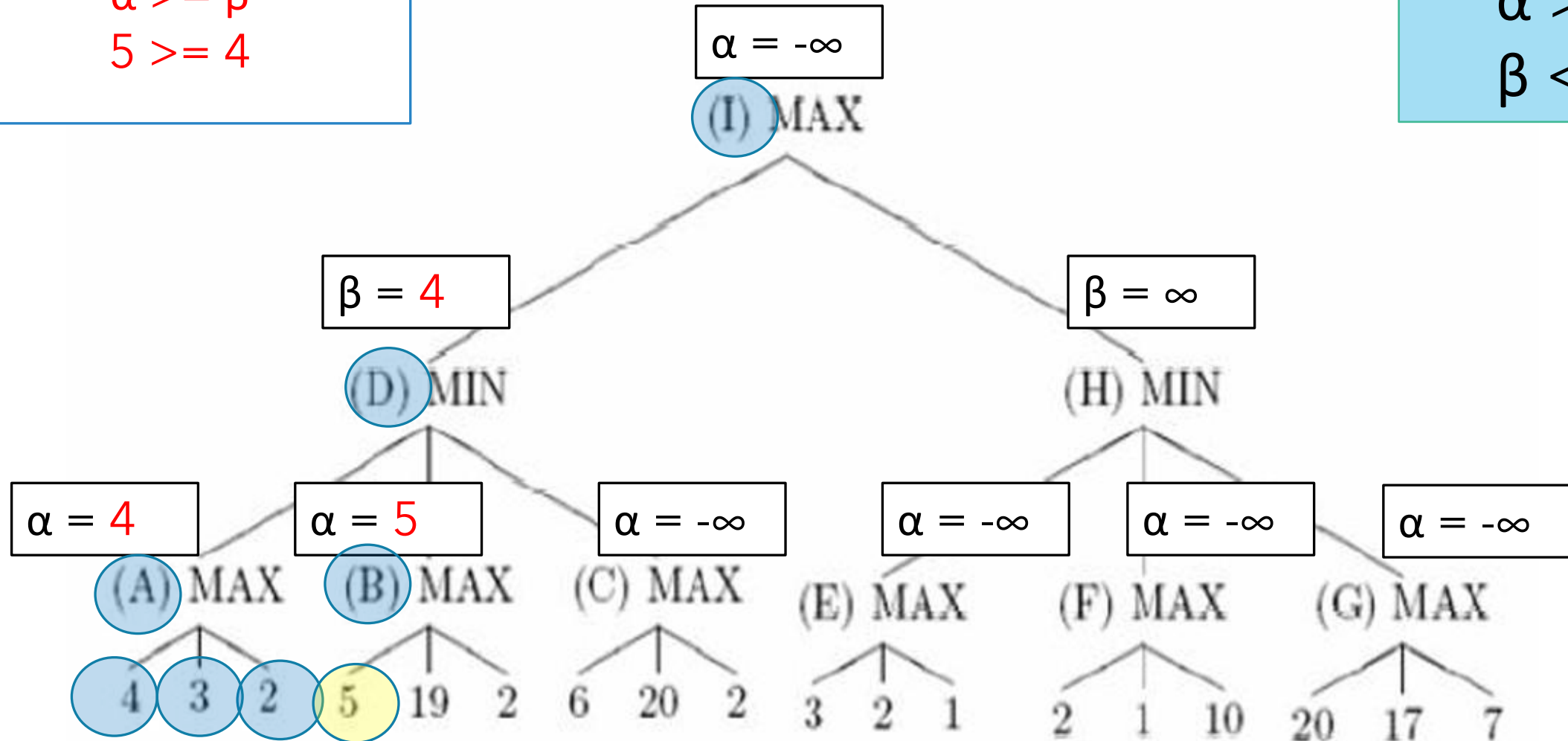
$$\alpha \geq \beta$$

$$5 \geq 4$$

When to cut ?

$$\alpha \geq \beta$$

$$\beta \leq \alpha$$



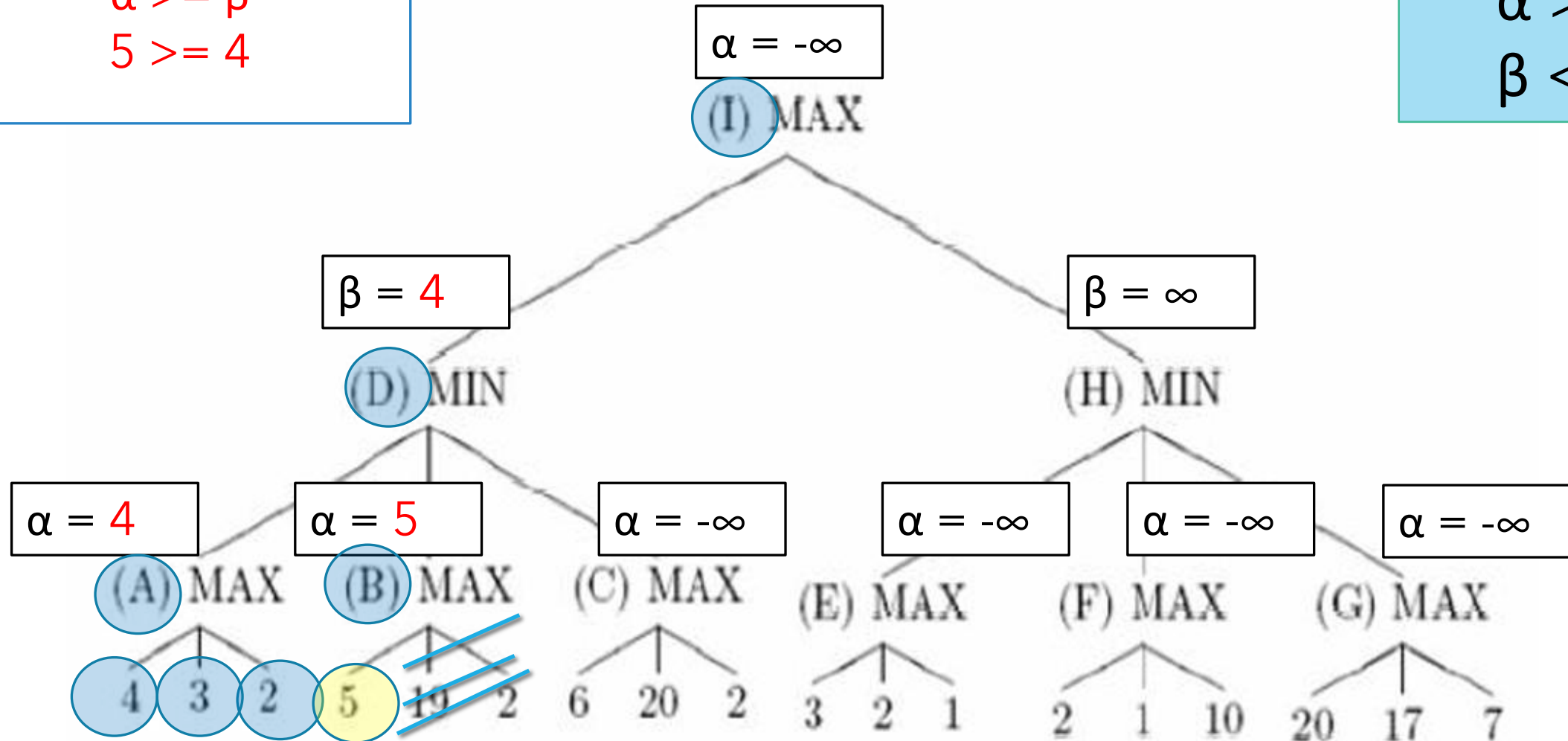
$$\alpha \geq \beta$$

$$5 \geq 4$$

When to cut ?

$$\alpha \geq \beta$$

$$\beta \leq \alpha$$



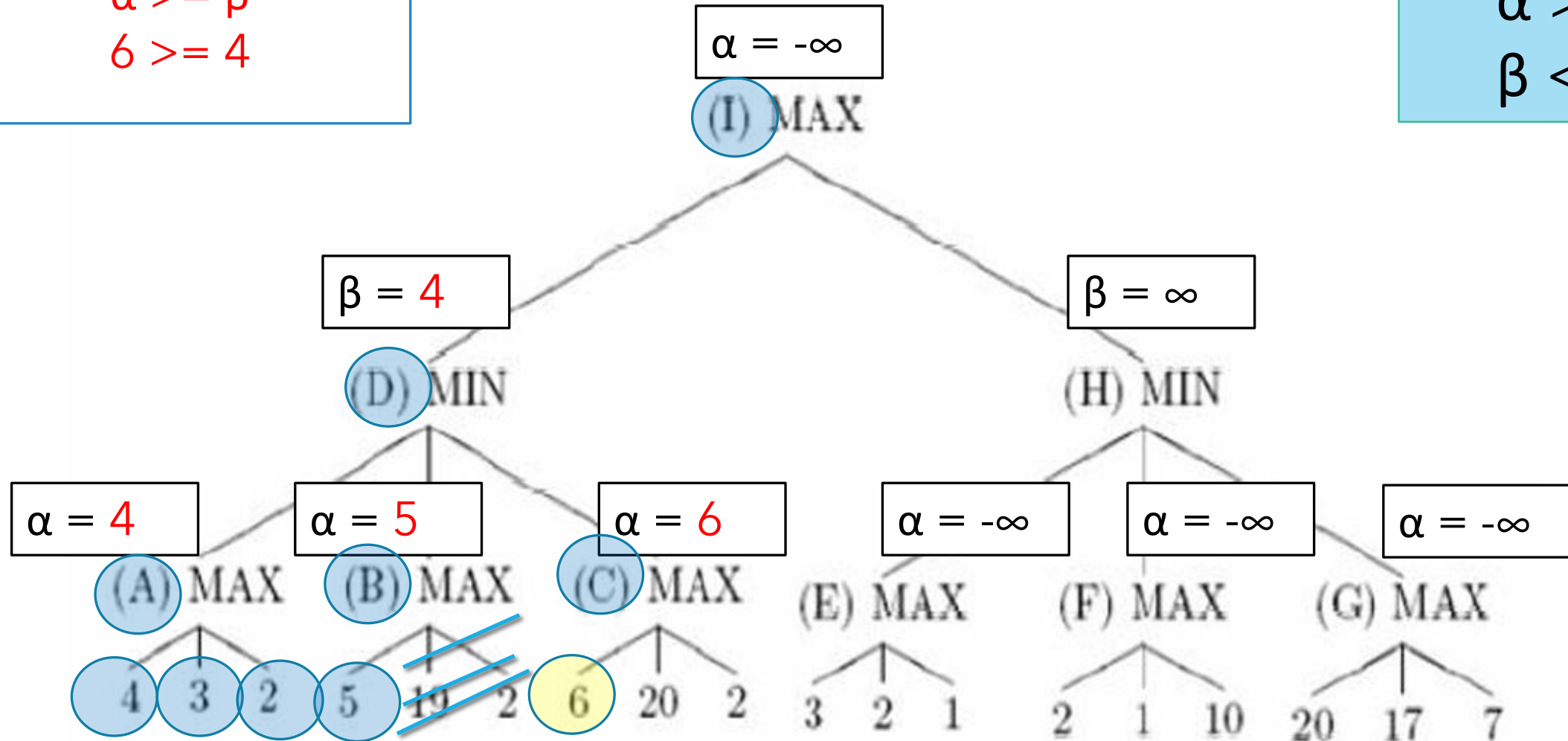
$$\alpha \geq \beta$$

$$6 \geq 4$$

When to cut ?

$$\alpha \geq \beta$$

$$\beta \leq \alpha$$



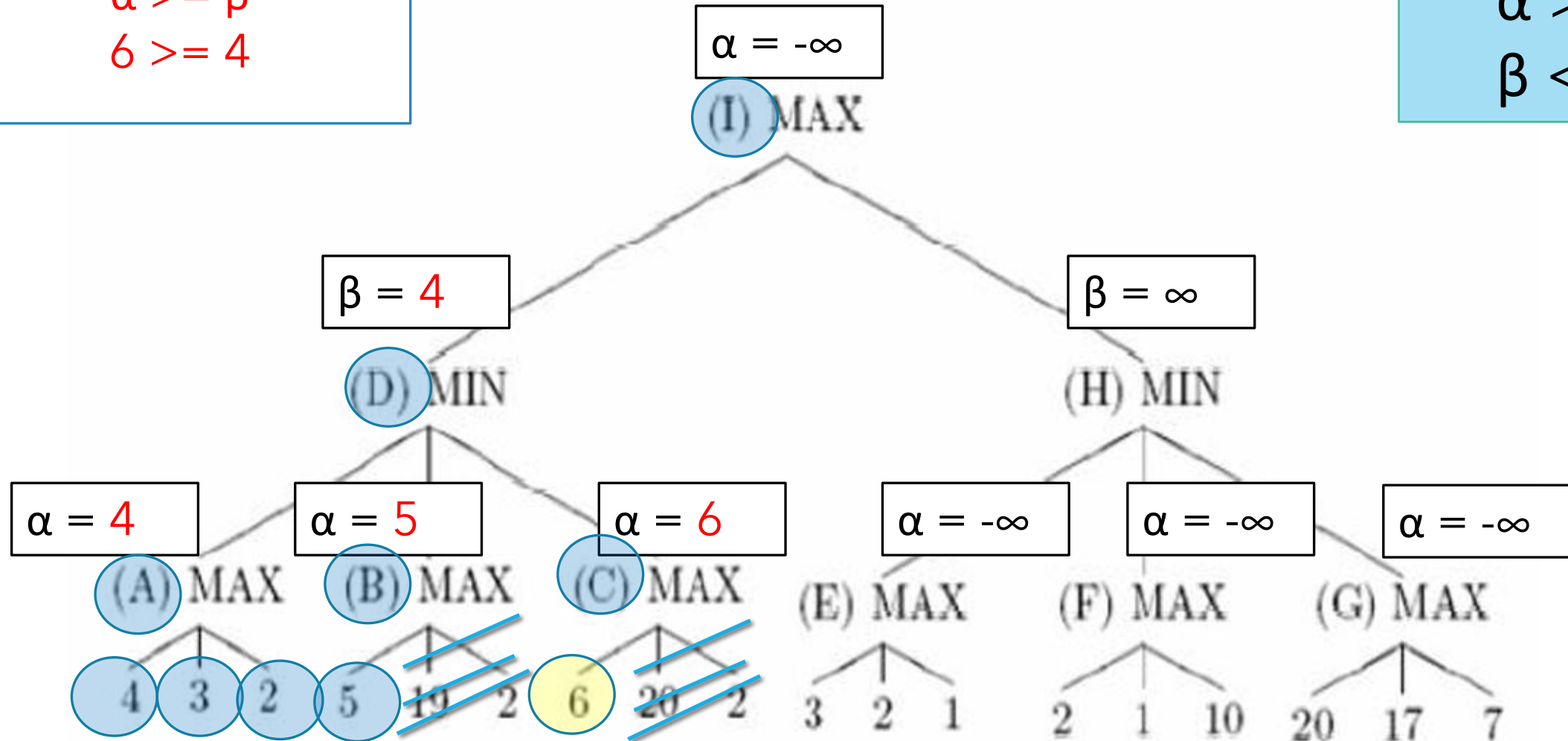
$$\alpha \geq \beta$$

$$6 \geq 4$$

When to cut ?

$$\alpha \geq \beta$$

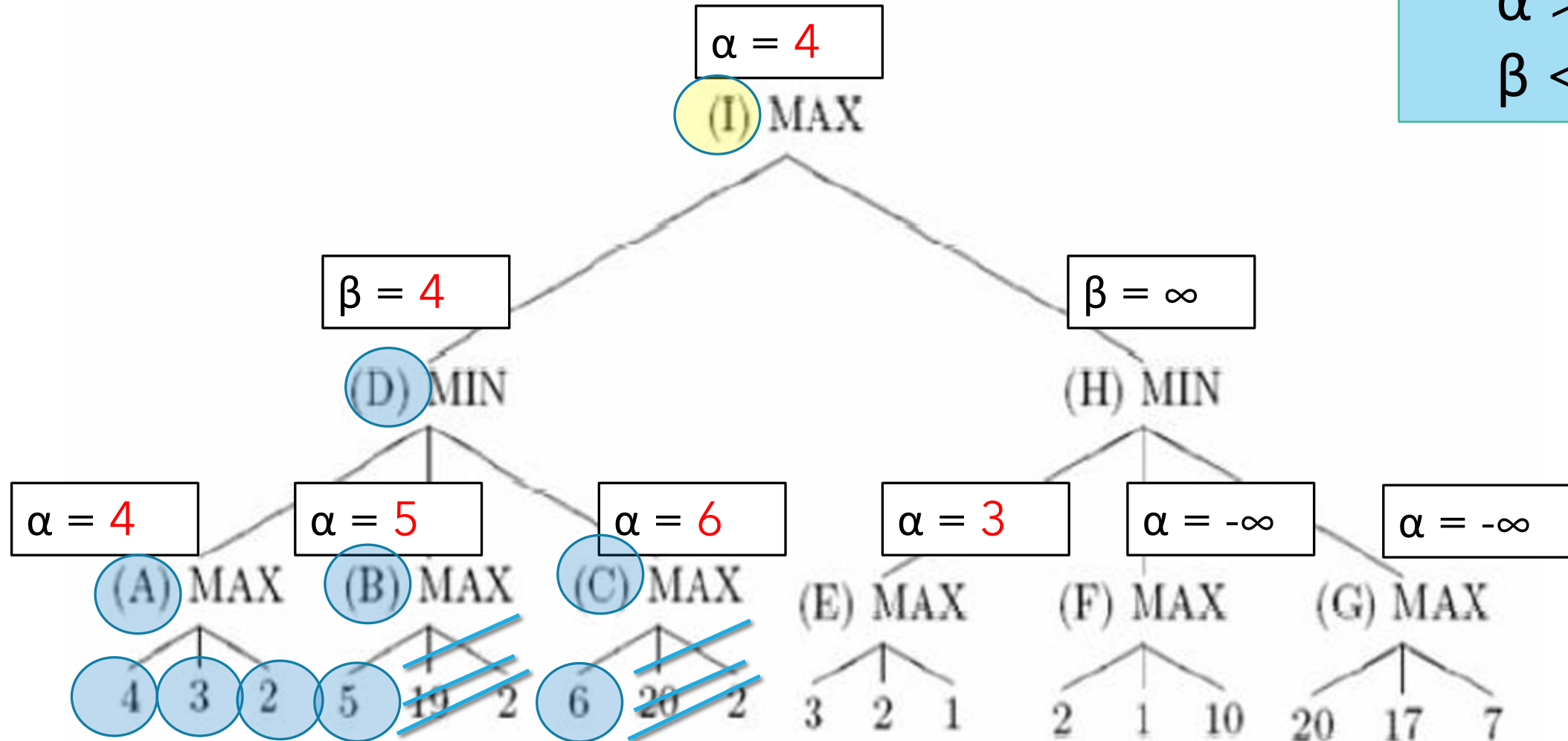
$$\beta \leq \alpha$$



When to cut ?

$$\alpha \geq \beta$$

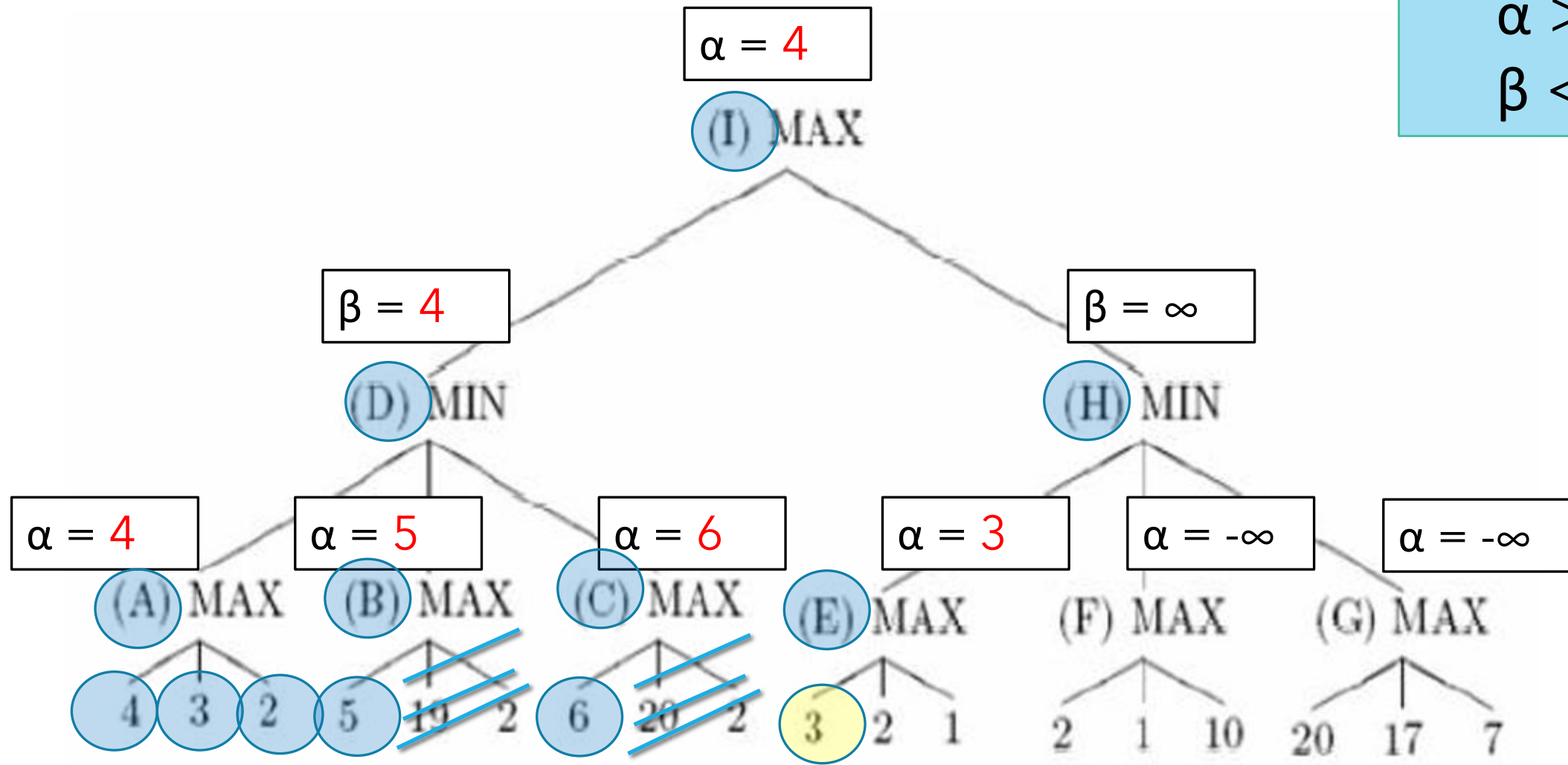
$$\beta \leq \alpha$$



When to cut ?

$$\alpha \geq \beta$$

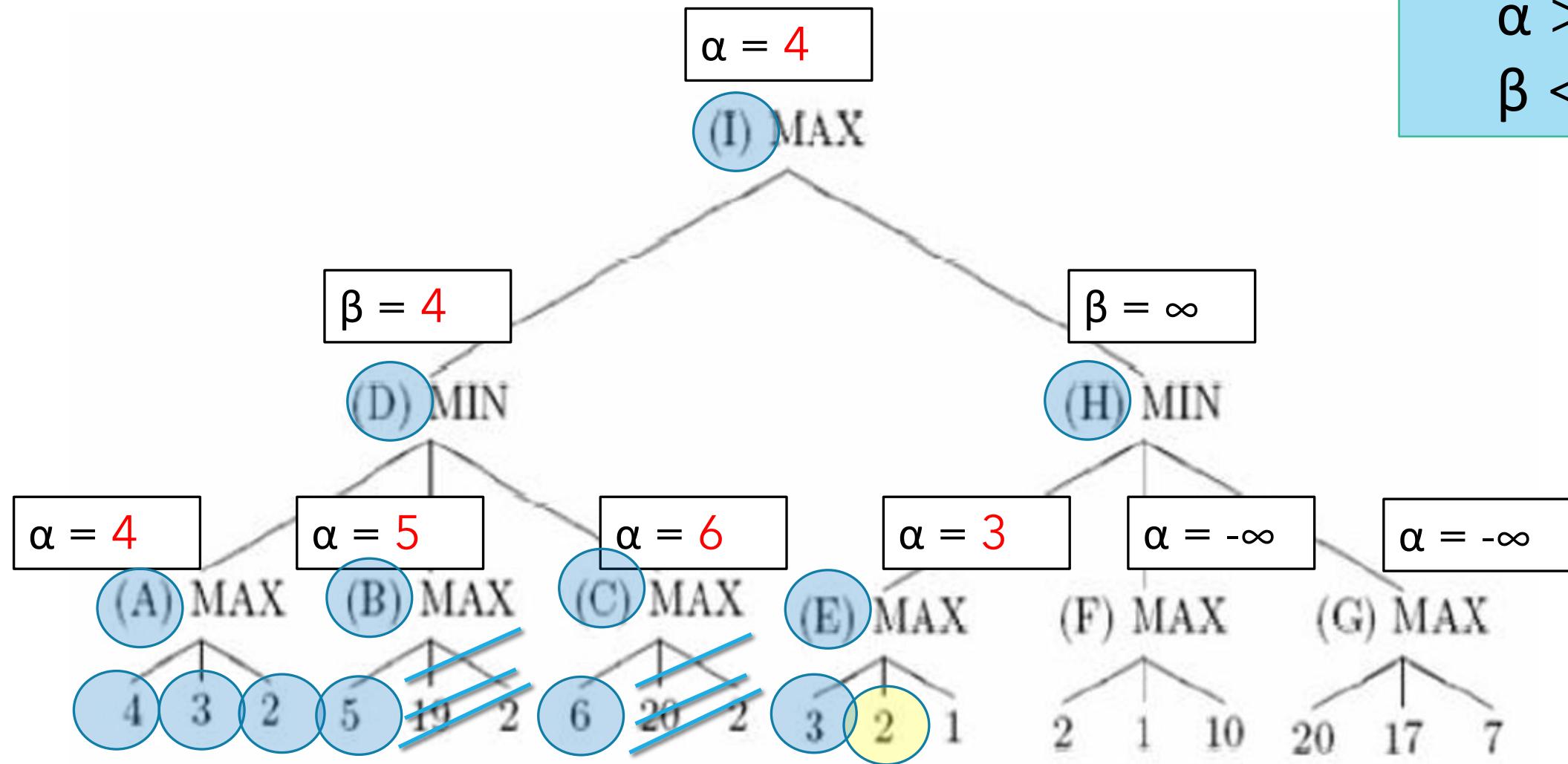
$$\beta \leq \alpha$$



When to cut ?

$$\alpha \geq \beta$$

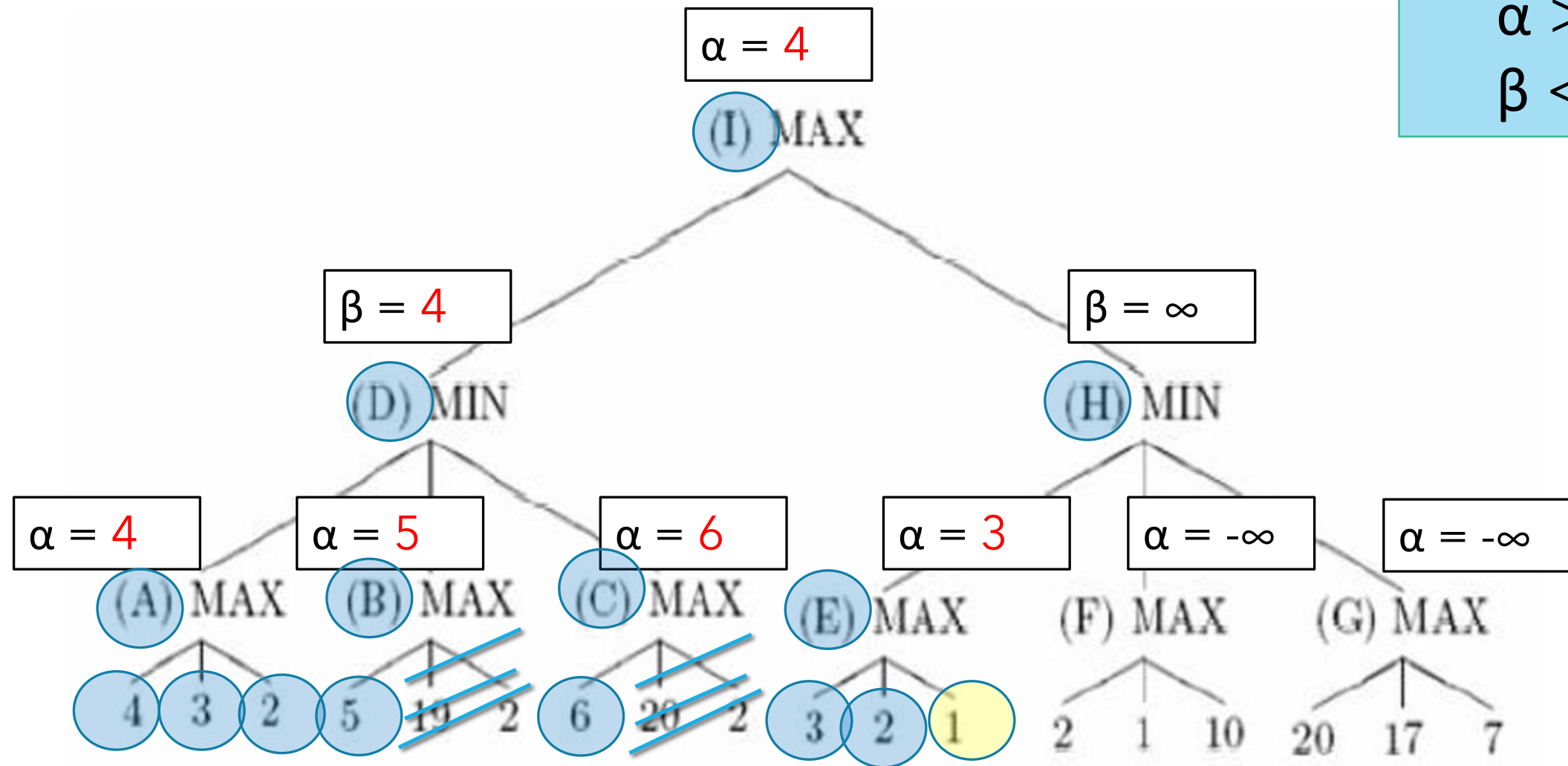
$$\beta \leq \alpha$$



When to cut ?

$$\alpha \geq \beta$$

$$\beta \leq \alpha$$



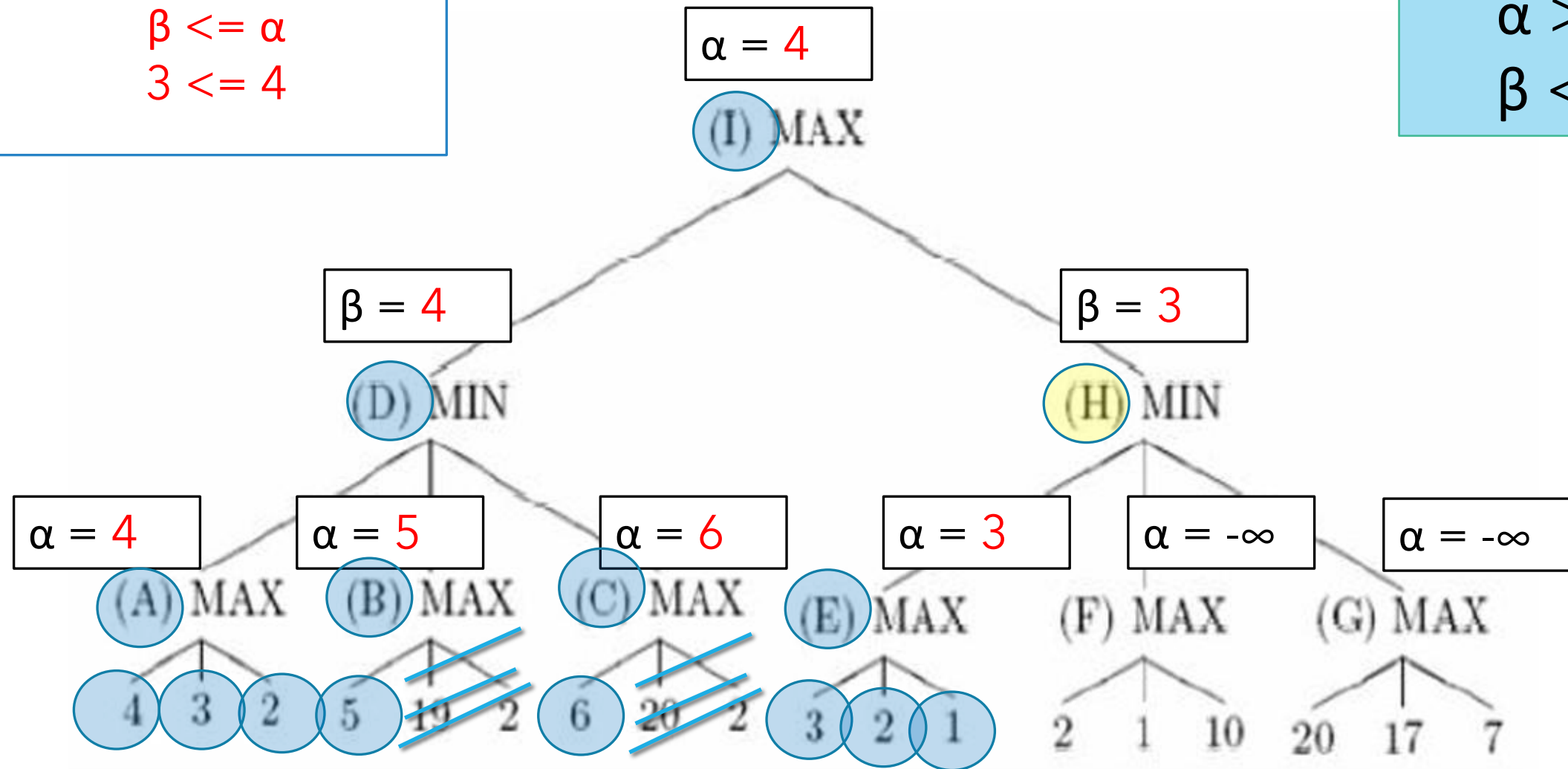
$$\beta \leq \alpha$$

$$3 \leq 4$$

When to cut ?

$$\alpha \geq \beta$$

$$\beta \leq \alpha$$



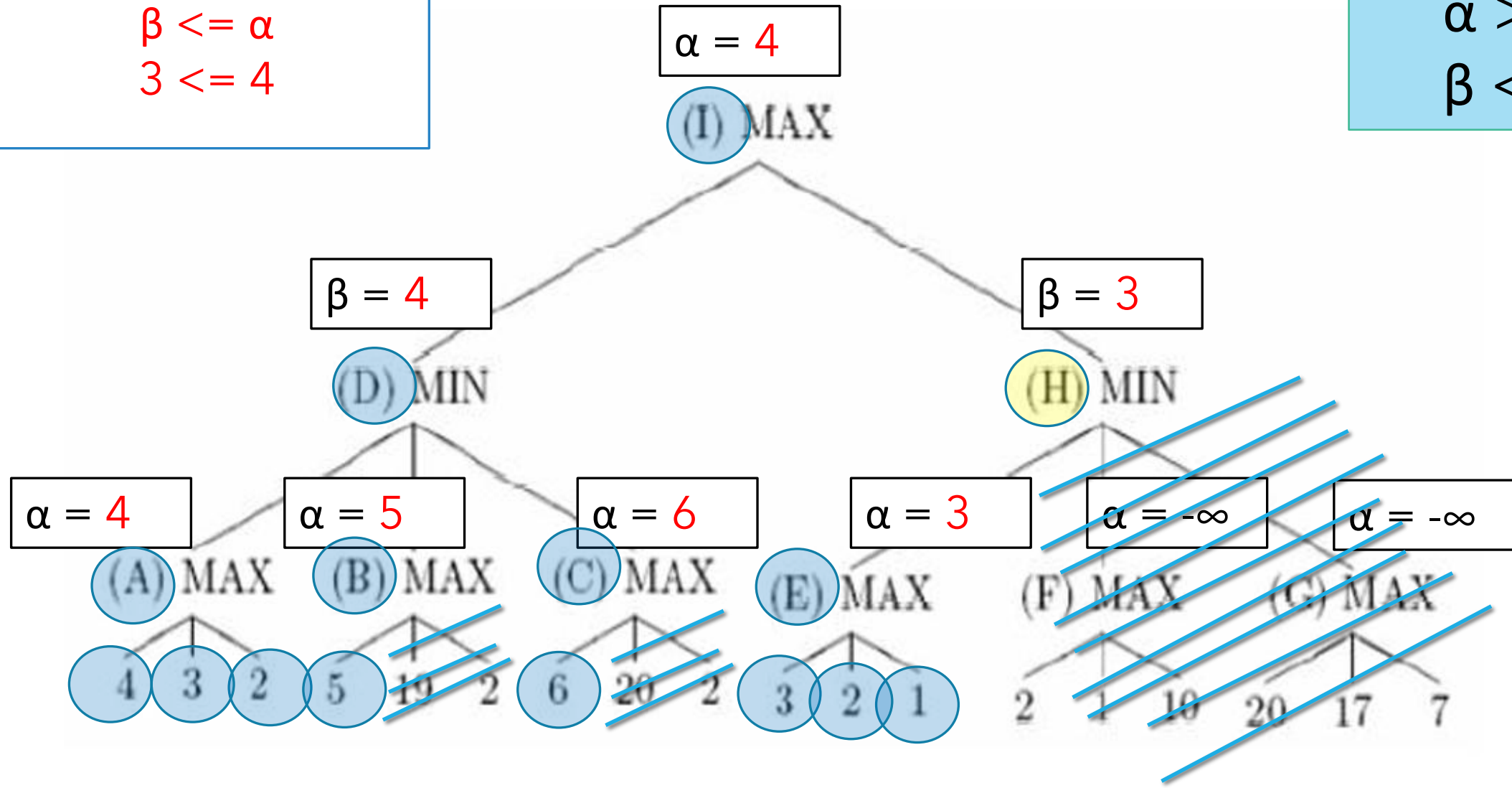
$$\beta \leq \alpha$$

$$3 \leq 4$$

When to cut ?

$$\alpha \geq \beta$$

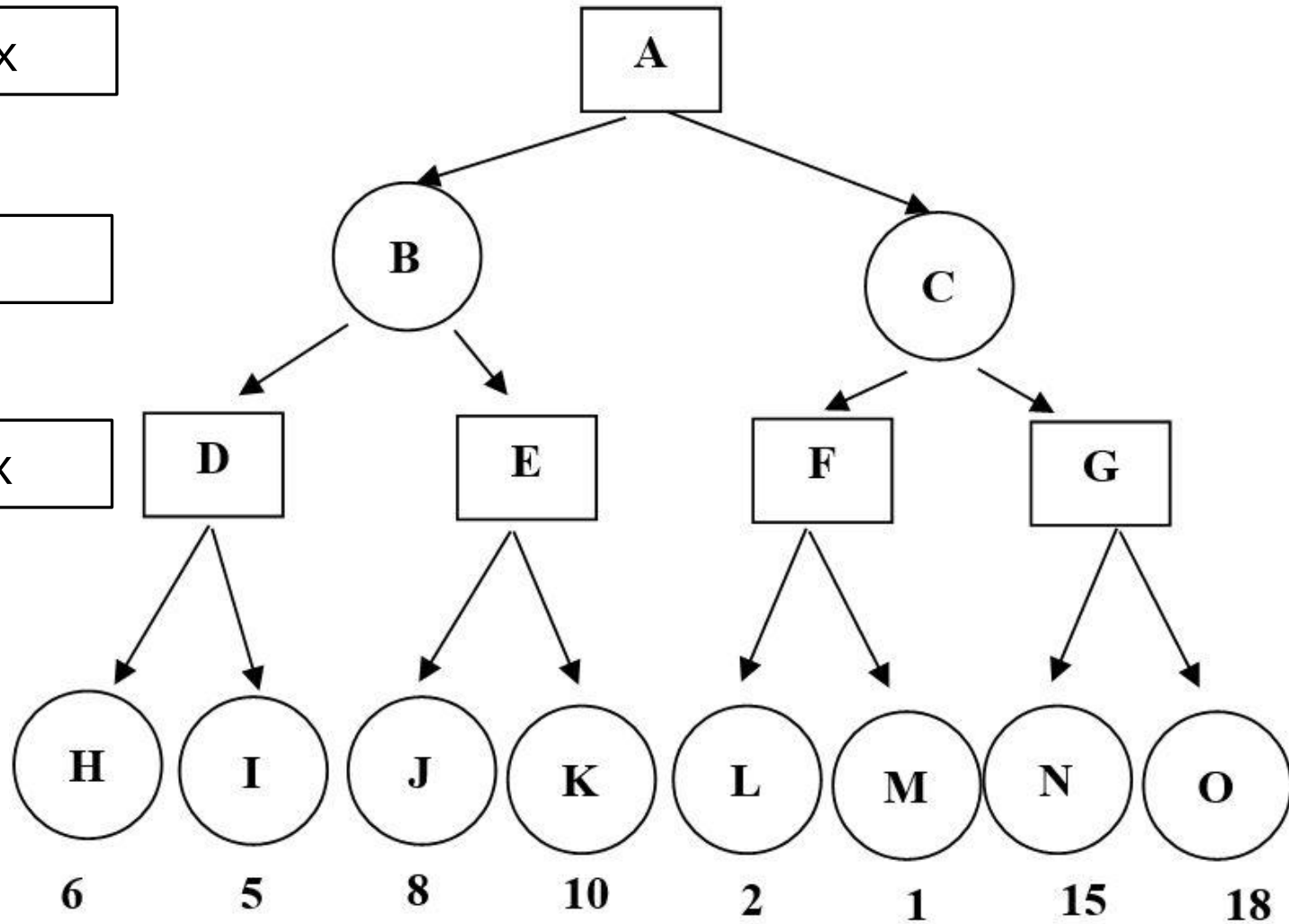
$$\beta \leq \alpha$$



max

min

max



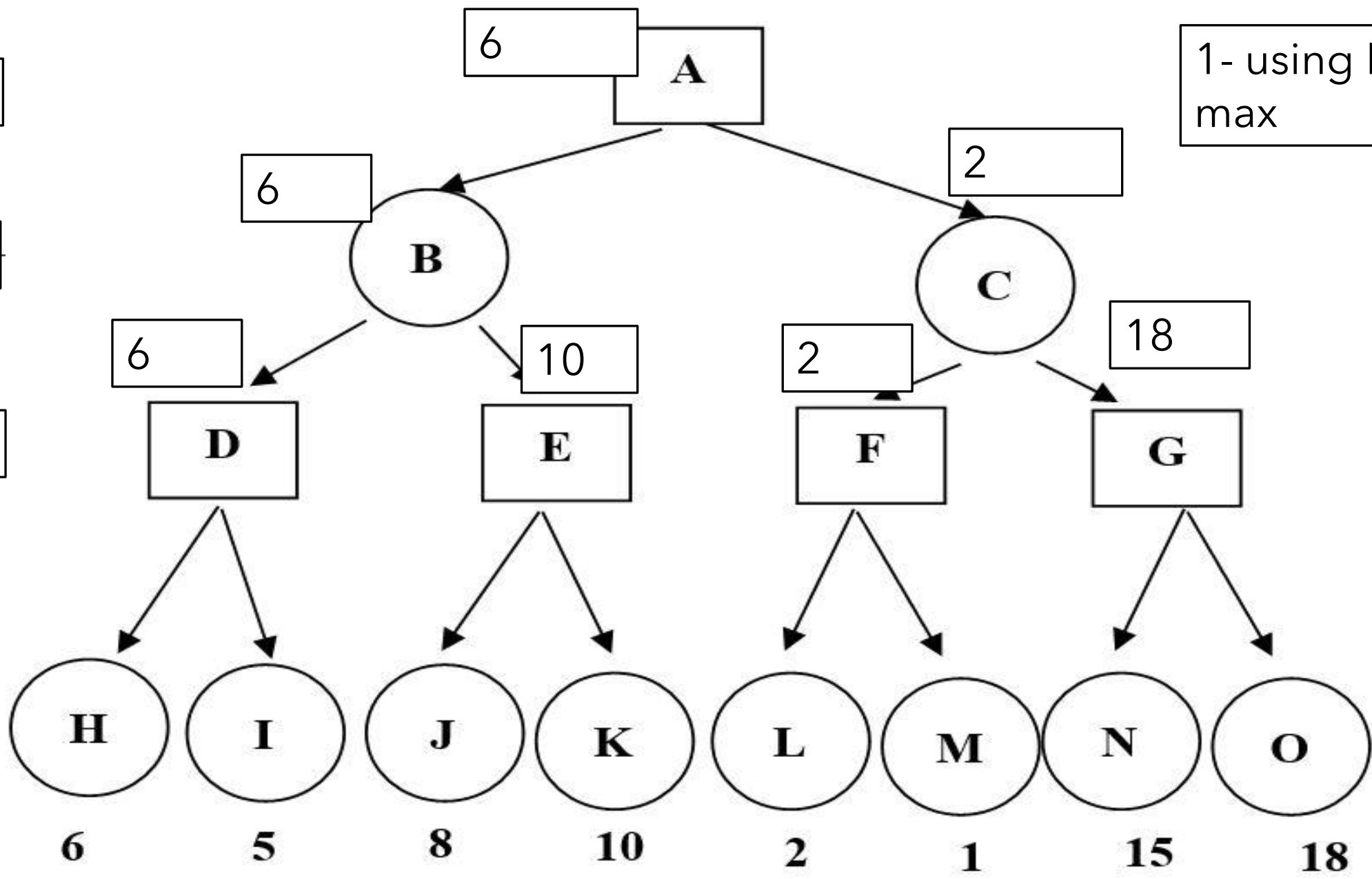
More
Examples

max

min

max

1- using Min
max

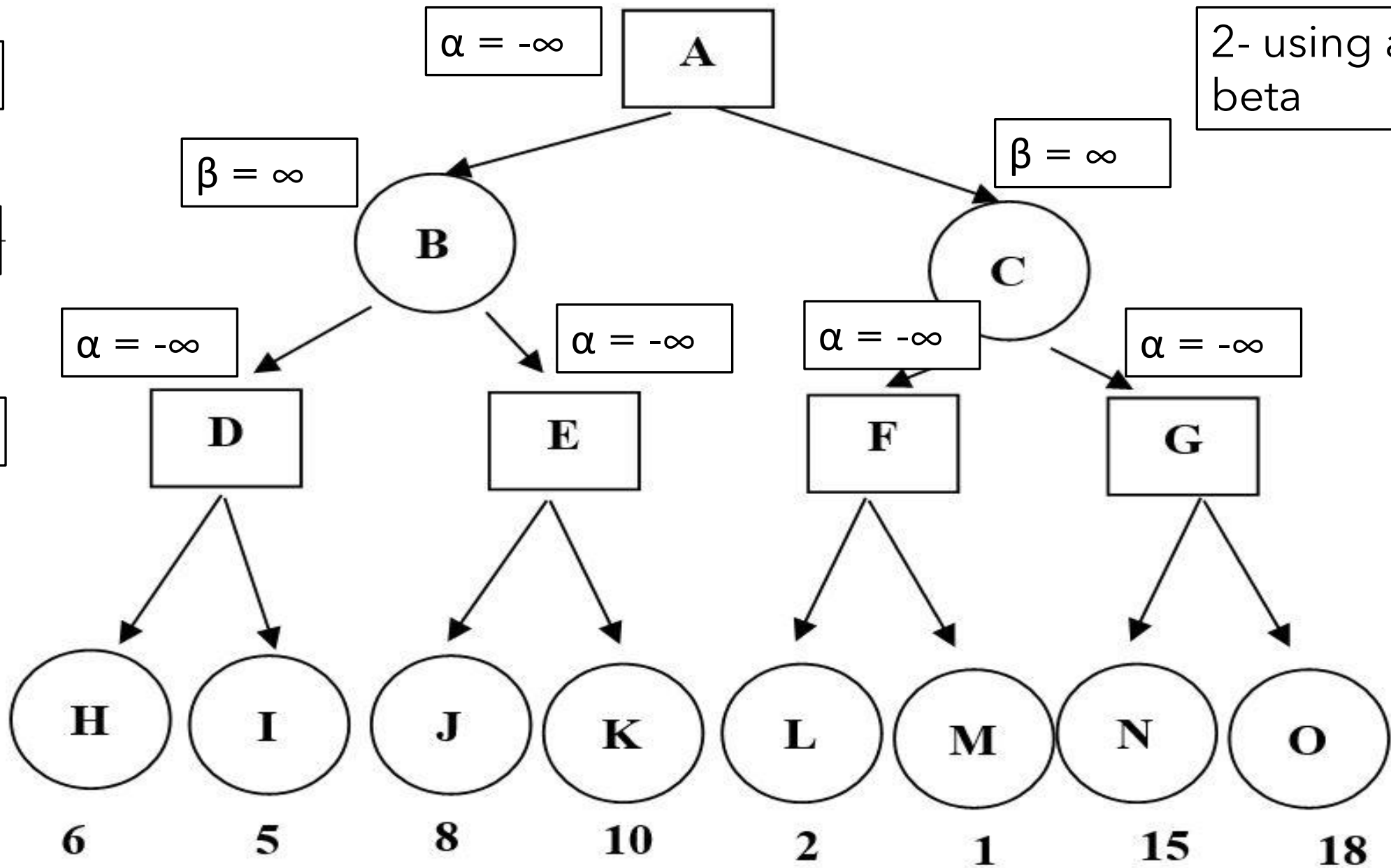


max

min

max

2- using alpha
beta

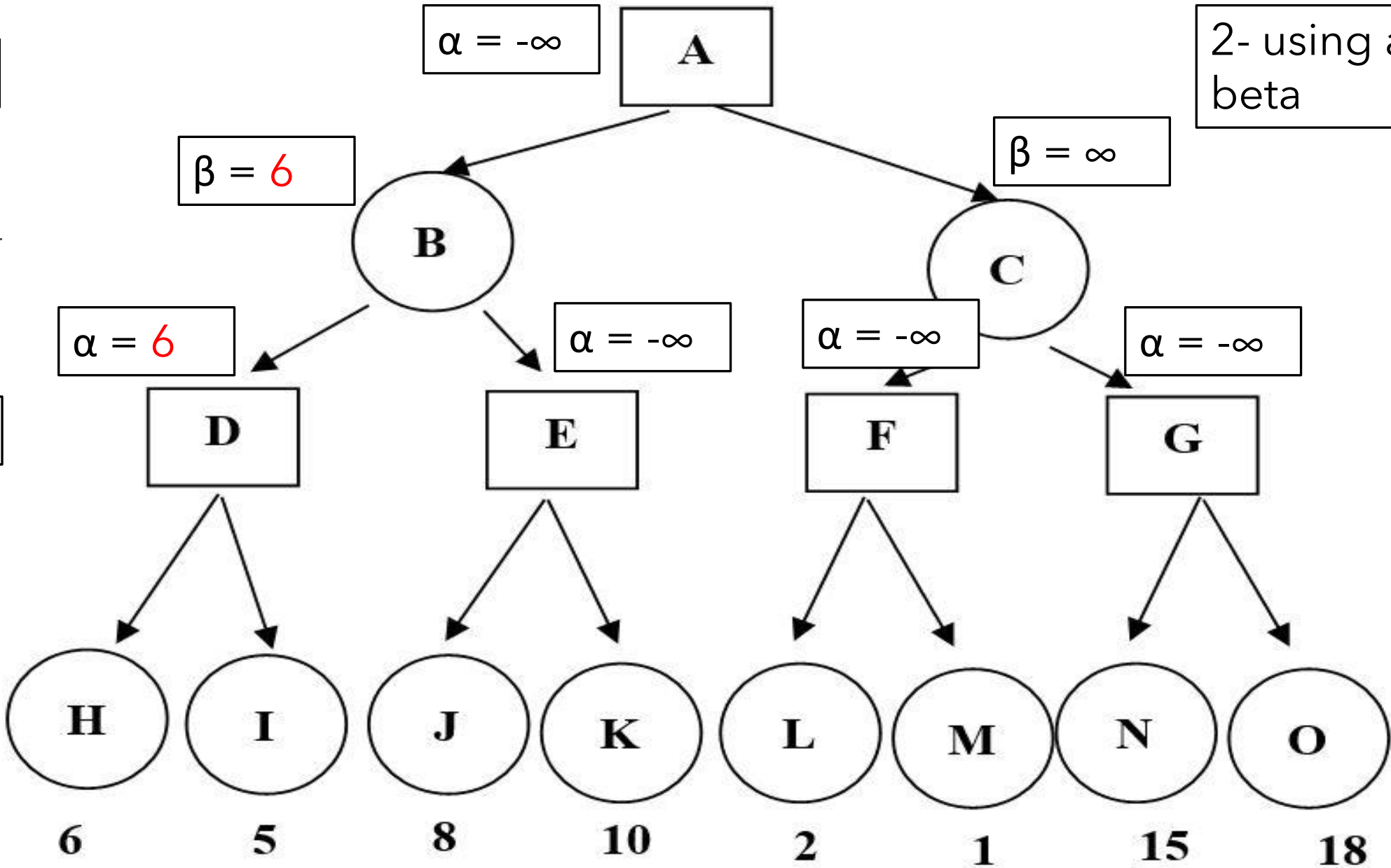


max

min

max

2- using alpha
beta

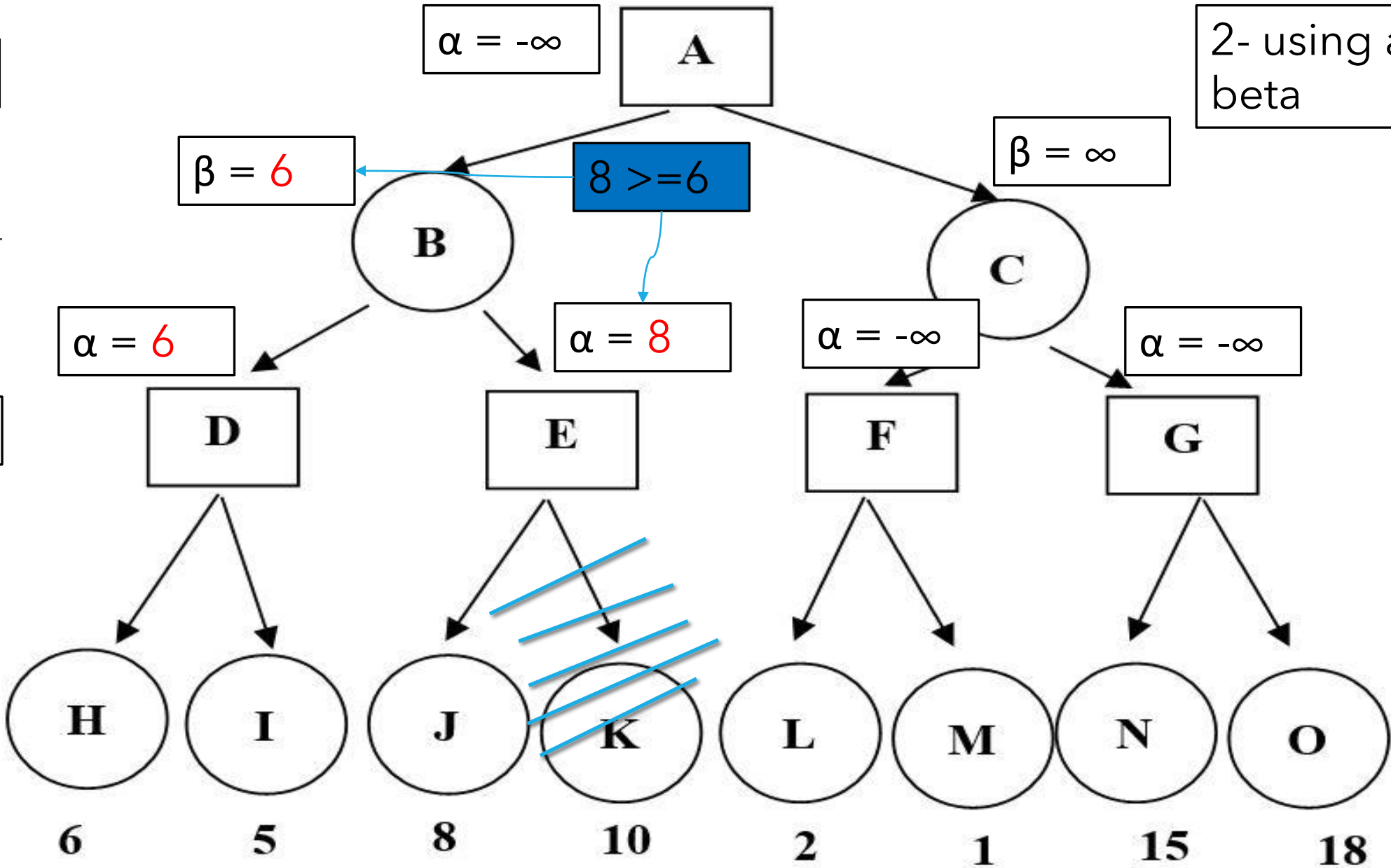


max

min

max

2- using alpha beta

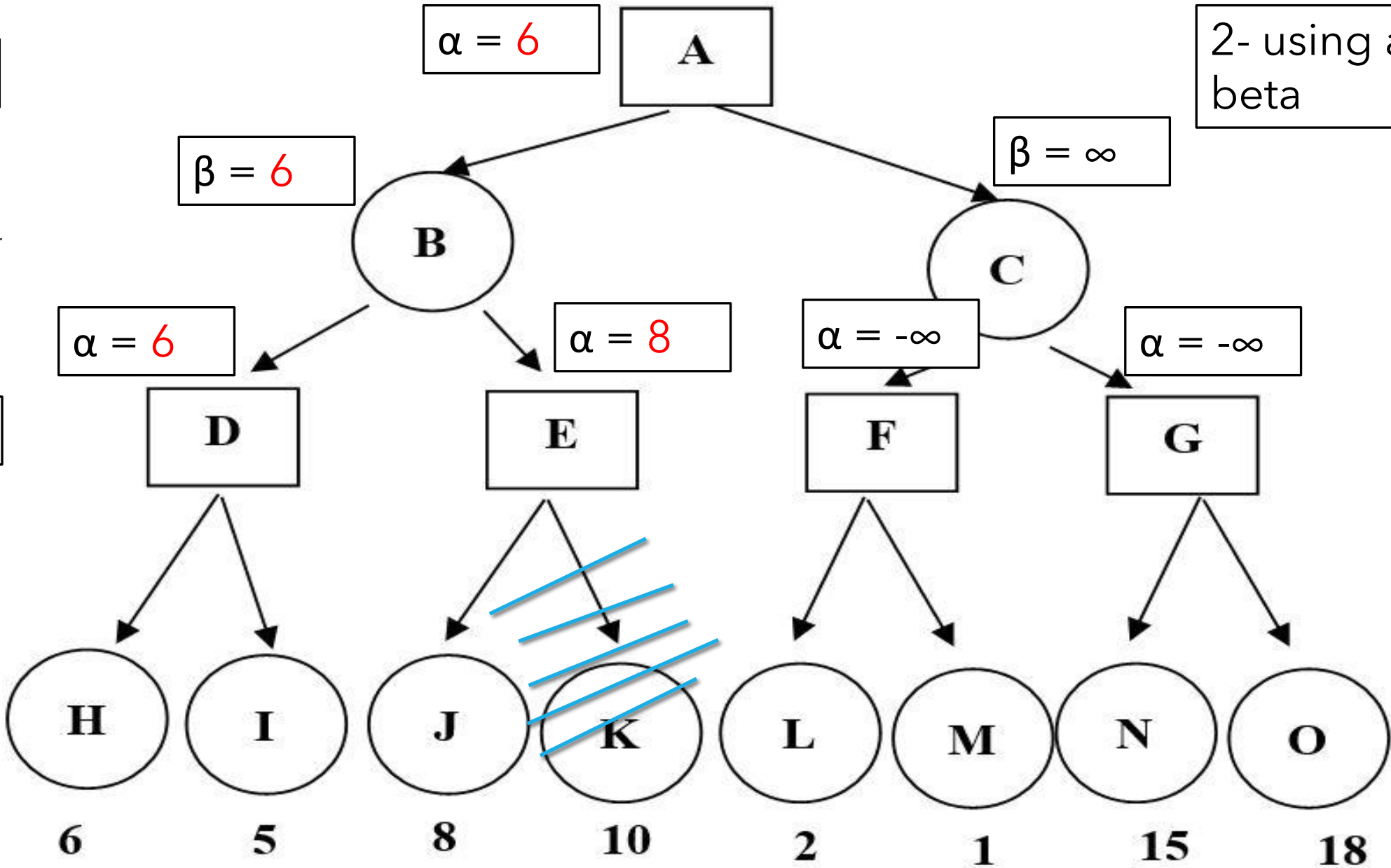


max

min

max

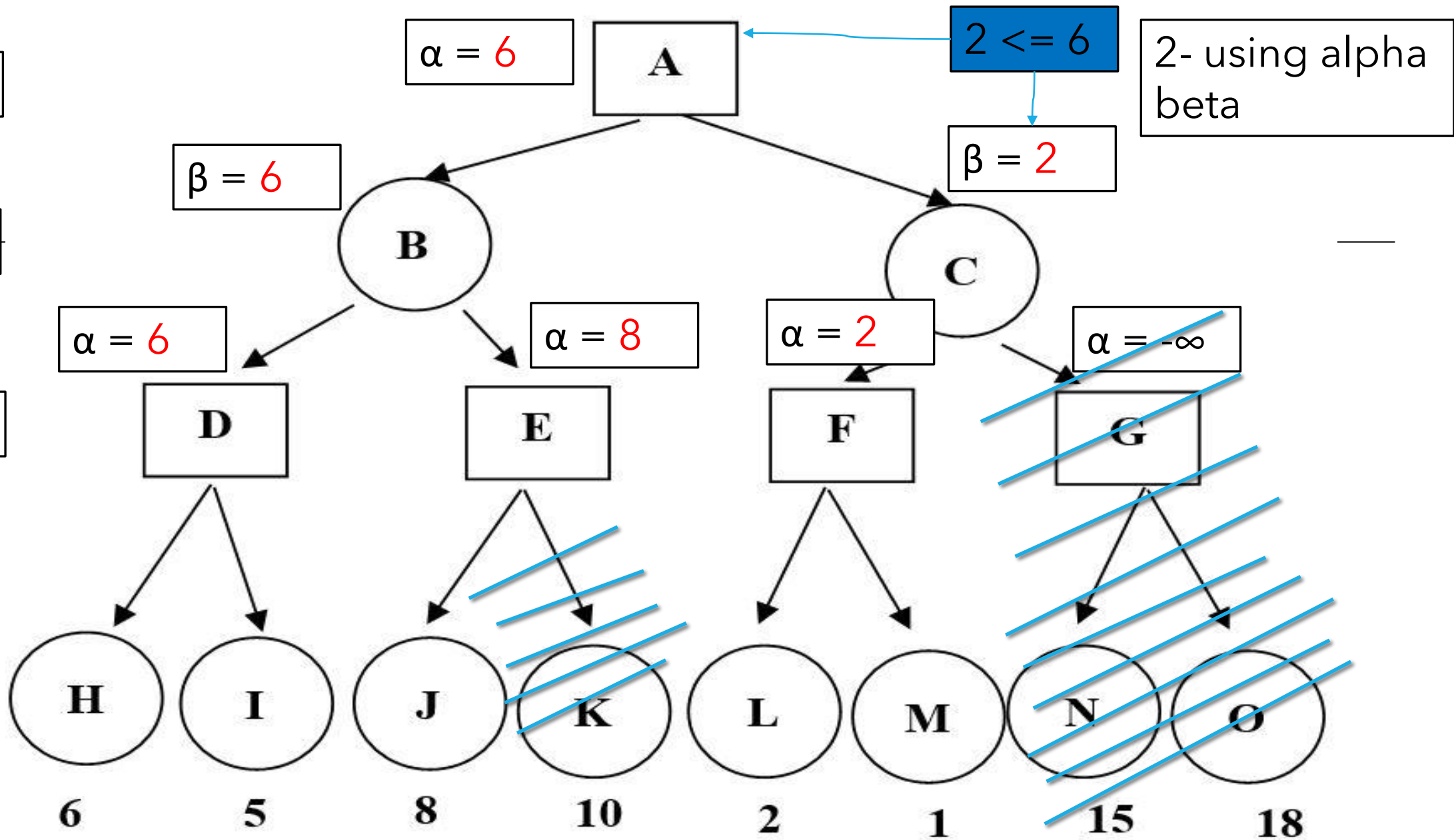
2- using alpha beta



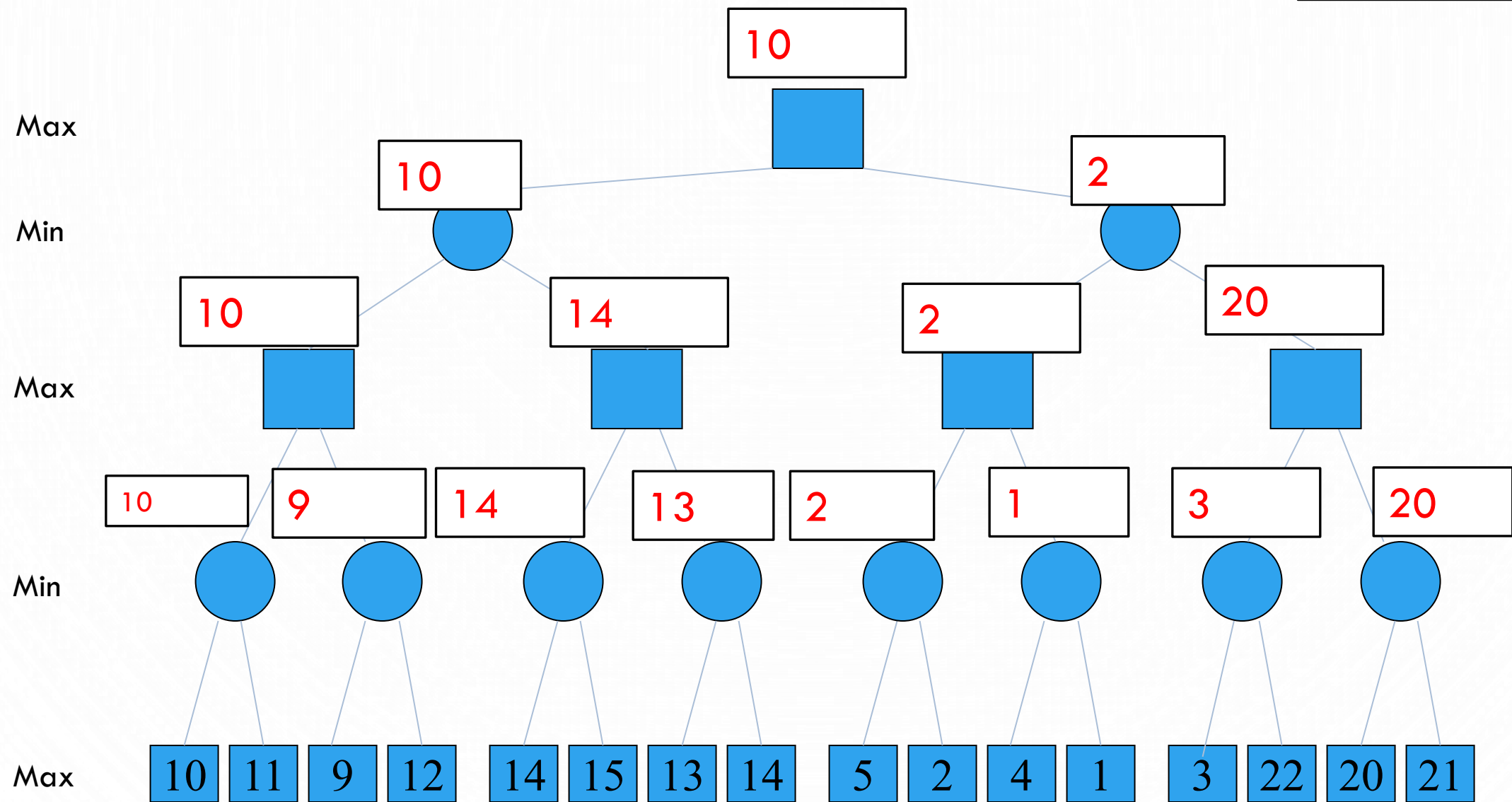
max

min

max



1 - min max



2- using alpha
beta

